UNIVERSITY-LEVEL SELF-STUDY REFERENCE

THE COMPLETE MACHINE LEARNING HANDBOOK

Theory / Mathematics / Python
Visualizations / Real-World Applications

38 CHAPTERS • 12 CODE PROJECTS • 128 INTERVIEW QUESTIONS

Educational reference edition | 14 October 2026

TEACHING EDITION | VERIFIED CODE

Reader's guide

This handbook follows the 38-chapter curriculum supplied in the brief and progresses from basic programming and mathematics to core algorithms, neural networks, probabilistic modeling, real-time sensors, and ML operations. Each chapter supplies a concept explanation, a mathematical anchor, a hand-calculated example, a complete executed Python example, a figure, a troubleshooting note, and exercises.

Scope and limitation: a 250–400+ page fully worked textbook with individual derivations, projects, and exercise solutions for every listed subtopic is considerably larger than this edition. This is a compact, substantive 38-chapter teaching handbook rather than a claim of exhaustive, graduate-textbook treatment of every item. The chapter roadmaps retain the complete requested syllabus, so missing depth is visible rather than silently replaced. All example scripts are included in the companion ZIP.

How to study with this book

- Follow Chapters 1–10 in order before attempting new algorithms.
- For each algorithm, understand the loss, assumptions, decision boundary, and failure modes before changing hyperparameters.
- Recompute every worked numerical example by hand, then run and modify its script.
- Inspect plots skeptically: axis labels, train/test splits, randomness, and units always matter.
- Use the 12 projects as small reproducible lab exercises, not performance evidence for real products.

Reproducibility and safety

Chapter examples have been executed in the local Python environment; corresponding output is included verbatim. Stochastic outputs can vary with library version, BLAS threading, GPU backend, and hardware. Run examples in a fresh virtual environment using the companion requirements guide. Medical and financial examples are educational demonstrations and not clinical or professional decision systems.

Study roadmap

Phase	Chapters	Skill target
Foundations	1–10	Programming, math, statistics, data preparation
Classical ML	11–26	Models, metrics, tuning and responsible use
Neural and specialized ML	27–35	Deep learning, text, vision, RL, graphs, probability
Engineering	36–38	Projects, MLOps and embedded sensor systems

Contents

Chapter and appendix entries are clickable in PDF readers that support internal links.

Reader's guide 2

Contents 3

1. Foundations of AI and Machine Learning . . . 5
2. Python for Machine Learning . . . 8
3. Linear Algebra . . . 11
4. Calculus for Machine Learning . . . 14
5. Probability Theory . . . 17
6. Statistics and Statistical Inference . . . 20
7. Optimization . . . 23
8. Understanding Data . . . 26
9. Exploratory Data Analysis . . . 29
10. Preprocessing and Feature Engineering . . . 32
11. Linear Regression . . . 35
12. Regularized and Advanced Regression . . . 38
13. Classification Fundamentals . . . 41
14. Logistic Regression . . . 44
15. k-Nearest Neighbors . . . 47
16. Naive Bayes . . . 50
17. Support Vector Machines . . . 53
18. Decision Trees . . . 56
19. Ensemble Methods . . . 59
20. Clustering . . . 62
21. Dimensionality Reduction . . . 65
22. Anomaly Detection . . . 68
23. Association Rules and Recommender Systems . . . 71
24. Evaluating Machine Learning Models . . . 74
25. Model Selection and Hyperparameter Tuning . . . 77

26. Explainable and Responsible Machine Learning . . . 80
27. Neural Network Foundations . . . 83
28. Deep Learning Architectures . . . 86
29. Generative Machine Learning . . . 89
30. Time-Series Machine Learning . . . 92
31. Natural Language Processing . . . 95
32. Computer Vision . . . 98
33. Reinforcement Learning . . . 101
34. Graph Machine Learning . . . 104
35. Probabilistic Machine Learning . . . 107
36. End-to-End Machine Learning Projects . . . 110
37. Machine Learning Engineering and MLOps . . . 112
38. Machine Learning in Electrical Engineering and Embedded Systems . . . 115

Hands-on machine learning projects 118

Appendix A — Mathematical formula sheet 130

Appendix B — Algorithm comparison 133

Appendix C — Python machine learning cheat sheet 136

Appendix D — Evaluation metrics 137

Appendix E — Optimization method reference 139

Appendix F — Machine learning glossary 140

Appendix G — 128 interview questions and answers 144

Appendix H — Derivations and worked checks 151

Appendix I — Debugging and troubleshooting 155

Appendix J — Further learning and references 156

Appendix K — Reproducibility and chapter map 157

CHAPTER 01 / 38 · CORE NOTES

1. Foundations of AI and Machine Learning

Learning objective. Explain the purpose of foundations of ai and machine learning, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Artificial intelligence is the broader discipline of constructing agents that perceive, reason, plan, and act. Machine learning is the part of AI that improves a model from experience rather than requiring a rule for every case. Supervised learning uses labeled targets; unsupervised learning discovers structure; reinforcement learning learns decisions from feedback. Deep learning uses multilayer neural networks, while generative modeling learns to produce plausible new observations. Begin every problem by specifying the input, target, feedback, and deployment environment.

When and where this matters

Fraud detection uses labels from past transactions; grouping similar customer behavior can use clustering without labels.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Artificial Intelligence vs. Machine Learning vs. Deep Learning.
- Traditional programming vs. machine learning.
- Historical development of machine learning.
- Types of AI and machine learning systems.
- Supervised learning.
- Unsupervised learning.
- Semi-supervised learning.
- Self-supervised learning.
- Reinforcement learning.
- Online learning.
- Batch learning.
- Transfer learning.
- Multitask learning.
- Generative vs. discriminative models.
- Parametric vs. nonparametric models.
- Classification, regression, clustering, ranking, and anomaly detection.
- Training, validation, and testing.
- The complete machine learning lifecycle.

3. Mathematical model and notation

$$\hat{y} = f_\theta(x)$$

x is an input, theta denotes learned parameters, and f maps the input to a prediction. In unsupervised learning, an explicit labeled y may not exist.

Worked numerical example — follow every step

Suppose four labeled emails contain two spam messages and two legitimate messages. A model that predicts spam for three emails, two correctly, has 3/4 = 75% overall accuracy if it also correctly rejects the fourth. Always calculate the confusion matrix; accuracy hides some class-specific mistakes.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
# Supervised learning predicts known labels; clustering discovers groups.
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.cluster import KMeans
from sklearn.metrics import accuracy_score
X, y = load_iris(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, stratify=y, random_state=7)
clf = LogisticRegression(max_iter=1000).fit(Xtr, ytr)
cluster = KMeans(n_clusters=3, n_init=10, random_state=7).fit(X)
print('Labeled examples:', len(y))
print('Test accuracy:', round(accuracy_score(yte, clf.predict(Xte)), 3))
print('Cluster sizes:', sorted(__import__('numpy').bincount(cluster.labels_)))
```

Verified console output

```
Labeled examples: 150
Test accuracy: 0.974
Cluster sizes: [np.int64(38), np.int64(50), np.int64(62)]
```

5. Visual reasoning

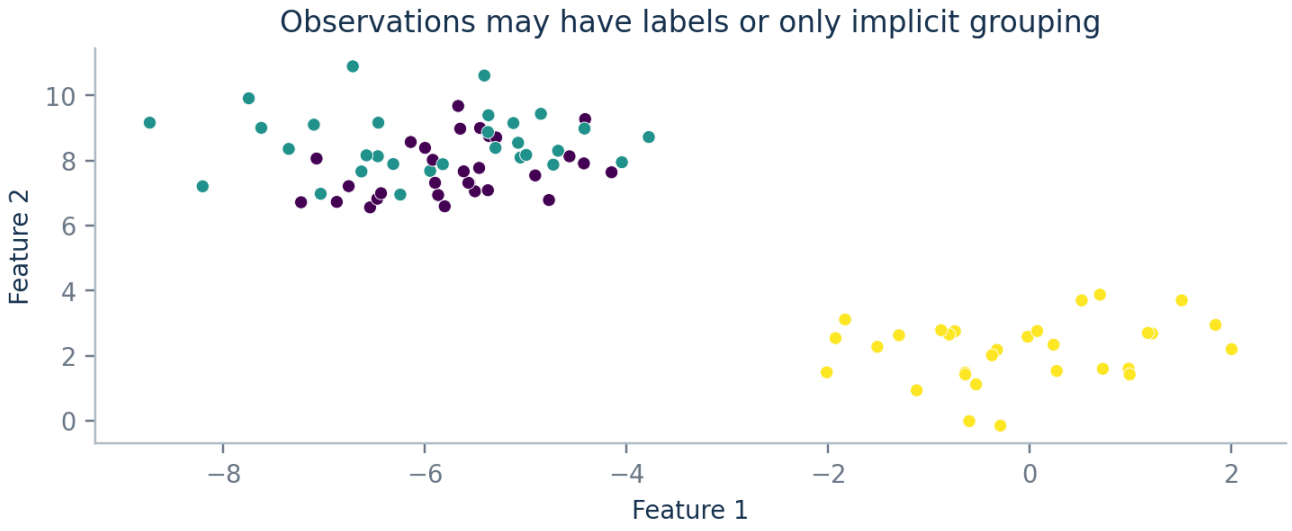


Figure 1. Artificial intelligence is the broader discipline of constructing agents that perceive, reason, plan, and act.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. Do not call every statistical rule an AI model, or assume an accurate training score predicts real-world performance.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when foundations of ai and machine learning would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Artificial intelligence is the broader discipline of constructing agents that perceive, reason, plan, and act.

Formula. x is an input, theta denotes learned parameters, and f maps the input to a prediction. In unsupervised learning, an explicit labeled y may not exist.

Practical example. Fraud detection uses labels from past transactions; grouping similar customer behavior can use clustering without labels.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for foundations of ai and machine learning.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 02 / 38 · CORE NOTES

2. Python for Machine Learning

Learning objective. Explain the purpose of python for machine learning, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Python is the dominant practical language for machine learning because it combines readable syntax with mature numerical libraries. NumPy operations act on whole arrays; pandas stores labeled tables; scikit-learn supplies estimators and preprocessing pipelines; PyTorch provides automatic differentiation for neural networks. Vectorization usually improves clarity and speed, but shape errors can silently distort results. Reproducibility requires environment information, seeds, and a record of the dataset split.

When and where this matters

Scientists prepare sensor tables in pandas; engineers implement batch inference using NumPy matrices.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Python syntax and data types.
- Variables, functions, loops, and conditionals.
- Lists, dictionaries, sets, and tuples.
- List comprehensions.
- Object-oriented programming fundamentals.
- NumPy arrays.
- Vectorization and broadcasting.
- pandas DataFrames.
- Data visualization with Matplotlib.
- Seaborn and Plotly introductions.
- SciPy fundamentals.
- scikit-learn fundamentals.
- Jupyter Notebook.
- Virtual environments and package management.
- Debugging and testing Python code.
- Reproducibility and random seeds.

3. Mathematical model and notation

$$s = Xw$$

If X has shape (n,p) and w has shape (p,), the prediction vector s has shape (n,).

Worked numerical example — follow every step

For two rows [1,2] and [2,3] and weights [0.6,0.4], scores are 1(0.6)+2(0.4)=1.4 and 2(0.6)+3(0.4)=2.4. Their average is 1.9.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
01  import numpy as np
02  import pandas as pd
03  features = np.array([[1., 2.], [2., 3.], [3., 4.]])
04  weights = np.array([0.6, 0.4])
05  scores = features @ weights
06  frame = pd.DataFrame(features, columns=['feature_a', 'feature_b'])
07  frame['score'] = scores
08  print(frame.to_string(index=False))
09  print('Mean score:', round(float(scores.mean()), 3))
```

Verified console output

```
feature_a  feature_b  score
       1.0        2.0    1.4
       2.0        3.0    2.4
       3.0        4.0    3.4
Mean score: 2.4
```

5. Visual reasoning

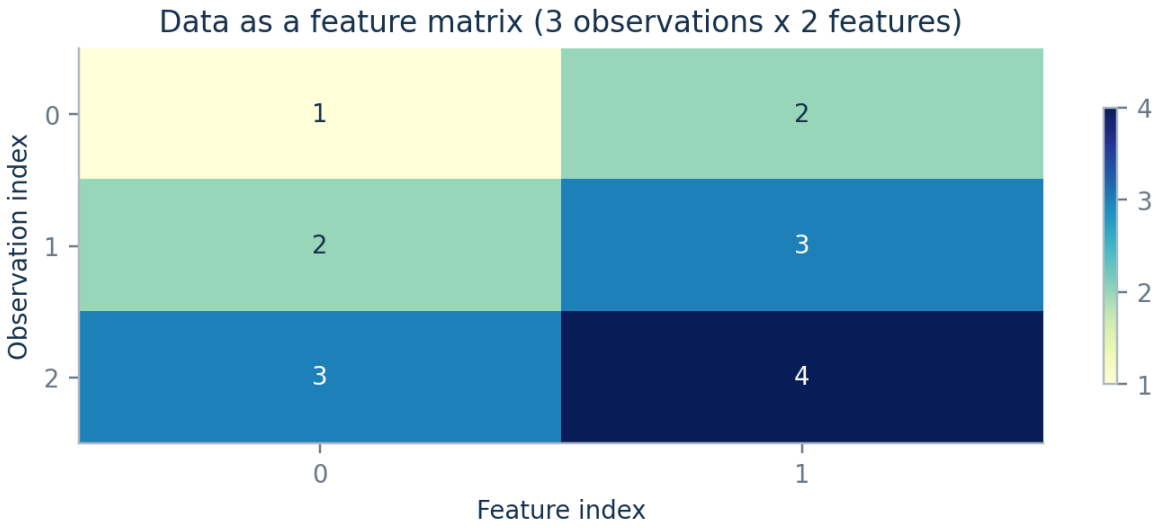


Figure 2. Python is the dominant practical language for machine learning because it combines readable syntax with mature numerical libraries.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. Do not use Python loops where array operations suffice, or assume broadcasting is safe without checking shapes.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when python for machine learning would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.

- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Python is the dominant practical language for machine learning because it combines readable syntax with mature numerical libraries.

Formula. If X has shape (n,p) and w has shape (p,), the prediction vector s has shape (n,).

Practical example. Scientists prepare sensor tables in pandas; engineers implement batch inference using NumPy matrices.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for python for machine learning.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 03 / 38 · CORE NOTES

3. Linear Algebra

Learning objective. Explain the purpose of linear algebra, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Linear algebra describes how machine learning represents and transforms data. Each observation is a vector, a dataset is a matrix, and a neural network layer is a matrix multiplication followed by a nonlinear function. Dot products measure alignment; projections reduce dimensionality; eigenvectors and singular vectors reveal important directions. Know the shapes of every matrix before computing: many conceptual bugs are dimension mismatches.

When and where this matters

PCA extracts dominant variation; linear layers combine thousands of input features with trainable weights.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Scalars, vectors, matrices, and tensors.
- Matrix dimensions and shapes.
- Matrix addition and multiplication.
- Dot products and inner products.
- Norms and distances.
- Matrix transpose.
- Matrix inverse and pseudoinverse.
- Rank and linear independence.
- Orthogonality and projections.
- Eigenvalues and eigenvectors.
- Eigendecomposition.
- Singular Value Decomposition.
- Positive definite and semidefinite matrices.
- Covariance matrices.
- Trace and determinant.
- Matrix calculus fundamentals.
- Applications in PCA, regression, neural networks, and optimization.

3. Mathematical model and notation

$$X = U\Sigma V^T$$

The singular value decomposition expresses an m-by-n matrix in orthogonal directions with nonnegative singular values.

Worked numerical example — follow every step

For A = [[3,1],[1,3]], the vector [1,1] has eigenvalue 4 because A[1,1]=[4,4]. The vector [1,-1] has eigenvalue 2. The matrix is symmetric and positive definite.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
01  import numpy as np
02  A = np.array([[3., 1.], [1., 3.]])
03  values, vectors = np.linalg.eigh(A)
04  U, s, Vt = np.linalg.svd(A)
05  print('Eigenvalues:', values.round(3))
06  print('Singular values:', s.round(3))
07  print('Reconstruction error:', round(float(np.linalg.norm(U @ np.diag(s) @ Vt-A)),
9))
```

Verified console output

```
Eigenvalues: [2. 4.]
Singular values: [4. 2.]
Reconstruction error: 0.0
```

5. Visual reasoning

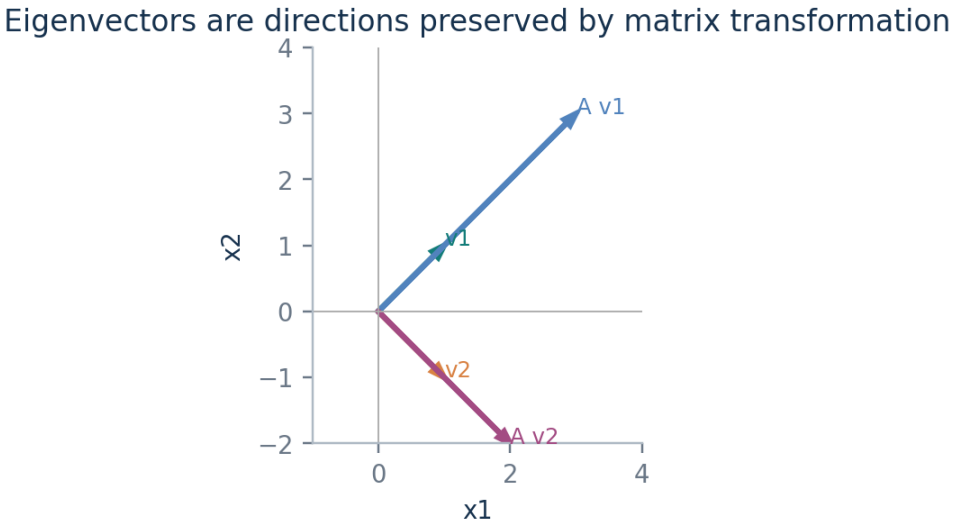


Figure 3. Linear algebra describes how machine learning represents and transforms data.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. A matrix inverse exists only for square, full-rank matrices; numerical least squares is often more stable than explicitly inverting matrices.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when linear algebra would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Linear algebra describes how machine learning represents and transforms data.

Formula. The singular value decomposition expresses an m-by-n matrix in orthogonal directions with nonnegative singular values.

Practical example. PCA extracts dominant variation; linear layers combine thousands of input features with trainable weights.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for linear algebra.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 04 / 38 · CORE NOTES

4. Calculus for Machine Learning

Learning objective. Explain the purpose of calculus for machine learning, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Calculus gives machine learning a language for changes and optimization. A derivative measures how an output varies with one input; a partial derivative varies one coordinate while holding the others fixed. The gradient points toward local increase. The chain rule differentiates composite functions, which is exactly what backpropagation automates. Hessians describe local curvature, helping explain why optimization can be easy, slow, or unstable.

When and where this matters

Backpropagation trains networks; gradient methods fit physical model parameters to laboratory measurements.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Functions and graphs.
- Limits and continuity.
- Derivatives.
- Partial derivatives.
- Chain rule.
- Product and quotient rules.
- Gradient vectors.
- Directional derivatives.
- Jacobian matrices.
- Hessian matrices.
- Taylor series.
- Multivariable optimization.
- Critical points and stationary points.
- Local and global minima.
- Convex and nonconvex functions.
- Lagrange multipliers.
- Automatic differentiation.
- Backpropagation as an application of the chain rule.

3. Mathematical model and notation

$$\nabla f(x) = \left[\frac{\partial f}{\partial x_1}, \ldots, \frac{\partial f}{\partial x_p}\right]^T$$

For p input coordinates the gradient has p entries, each a partial derivative.

Worked numerical example — follow every step

For f(x,y)=x^2+3xy+y^2, f_x=2x+3y and f_y=3x+2y. At (1,2), the gradient is [8,7]. The steepest local increase is along this vector.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
# Central difference approximates the derivative of f(x) = x^3.
def f(x): return x**3
x, h = 2., 1e-4
numerical = (f(x+h)-f(x-h))/(2*h)
exact = 3*x*x
print('Numerical derivative:', round(numerical, 6))
print('Exact derivative:', exact)
print('Absolute error:', round(abs(numerical-exact), 10))
```

Verified console output

```
Numerical derivative: 12.0
Exact derivative: 12.0
Absolute error: 1e-08
```

5. Visual reasoning

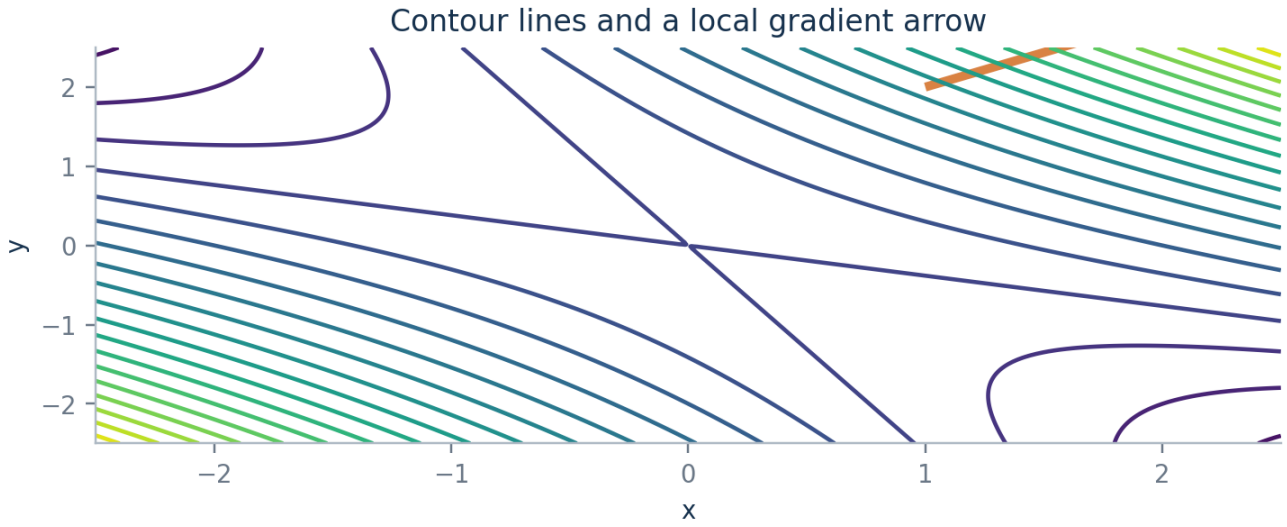


Figure 4. Calculus gives machine learning a language for changes and optimization.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. Do not confuse f_xy with f_x f_y; a second mixed partial differentiates twice, first with respect to one variable and then the other.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when calculus for machine learning would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.

- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Calculus gives machine learning a language for changes and optimization.

Formula. For p input coordinates the gradient has p entries, each a partial derivative.

Practical example. Backpropagation trains networks; gradient methods fit physical model parameters to laboratory measurements.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for calculus for machine learning.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 05 / 38 · CORE NOTES

5. Probability Theory

Learning objective. Explain the purpose of probability theory, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Probability describes uncertainty in data and predictions. Random variables represent outcomes; distributions allocate probability to possible values; conditional probabilities adjust beliefs after evidence. Bayes' rule connects prior plausibility, observation likelihood, and posterior plausibility. Expectation measures an average; variance measures spread. Machine learning frequently turns a prediction into a probability distribution rather than one certain answer.

When and where this matters

Medical screening needs base rates; predictive maintenance estimates the probability a device will fail.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Random variables.
- Probability distributions.
- Joint, marginal, and conditional probabilities.
- Independence.
- Bayes' theorem.
- Law of total probability.
- Expectation.
- Variance and standard deviation.
- Covariance and correlation.
- Bernoulli and Binomial distributions.
- Uniform, Gaussian, and Poisson distributions.
- Exponential and Multinomial distributions.
- Sampling distributions.
- Central Limit Theorem.
- Law of Large Numbers.
- Maximum Likelihood Estimation.
- Maximum A Posteriori Estimation.
- Information theory fundamentals.
- Entropy, cross-entropy, and KL divergence.

3. Mathematical model and notation

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}$$

The formula assumes P(B)>0. P(A) is the prior and P(A|B) is the posterior.

Worked numerical example — follow every step

If a condition affects 1% of people, a test detects 90% of cases and falsely flags 10% of others, then P(condition|positive)=0.9(0.01)/[0.9(0.01)+0.1(0.99)]=0.0833, or about 8.33%.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
01  import numpy as np
02  rng = np.random.default_rng(42)
03  flips = rng.binomial(n=1, p=0.30, size=10000)
04  print('Analytical mean:', 0.30)
05  print('Empirical mean:', round(float(flips.mean()), 4))
06  print('Analytical variance:', round(0.3*0.7, 4))
07  print('Estimated variance:', round(float(flips.var()), 4))
```

Verified console output

```
Analytical mean: 0.3
Empirical mean: 0.2942
Analytical variance: 0.21
Estimated variance: 0.2076
```

5. Visual reasoning

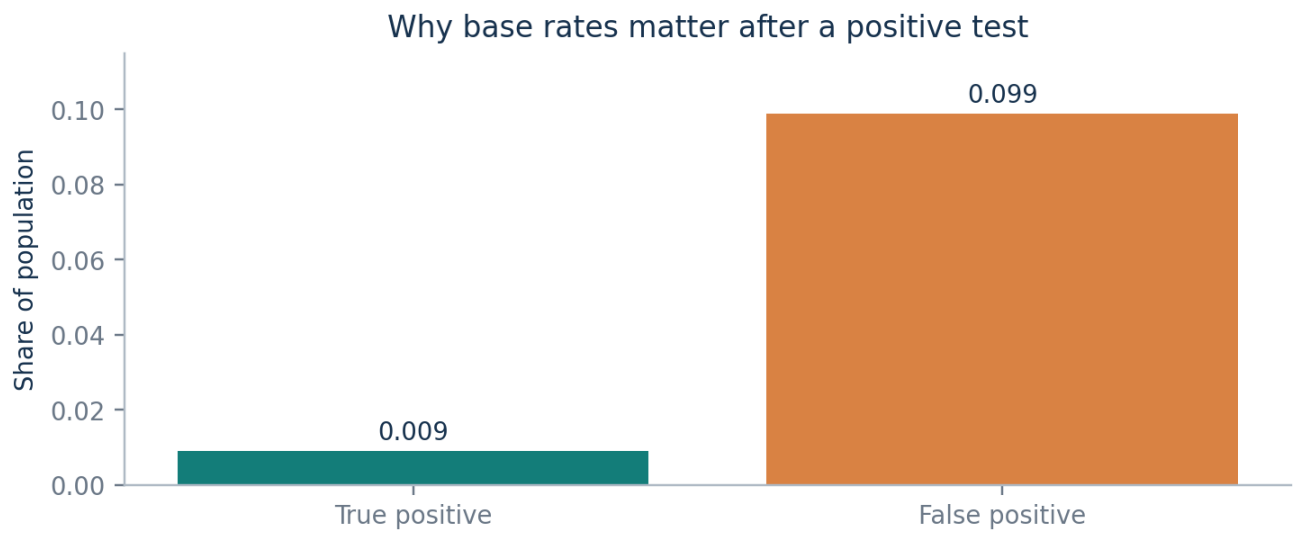


Figure 5. Probability describes uncertainty in data and predictions.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. Never reverse conditional probabilities: P(A|B) generally differs from P(B|A).

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when probability theory would be appropriate and when it would be a poor choice.

- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Probability describes uncertainty in data and predictions.

Formula. The formula assumes P(B)>0. P(A) is the prior and P(A|B) is the posterior.

Practical example. Medical screening needs base rates; predictive maintenance estimates the probability a device will fail.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for probability theory.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 06 / 38 · CORE NOTES

6. Statistics and Statistical Inference

Learning objective. Explain the purpose of statistics and statistical inference, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Statistics uses samples to estimate population behavior and quantify uncertainty. A confidence interval is a procedure with a coverage property, not the probability that a fixed parameter moves around. Hypothesis tests measure how unusual data are under a specified null model. A small p-value is not an effect size, and correlation cannot establish causation without supporting design or assumptions. Good ML evaluation uses statistical thinking when comparing models across samples.

When and where this matters

A/B tests compare website variants; model validation assesses uncertainty in estimated accuracy.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Descriptive statistics.
- Population vs. sample.
- Sample bias.
- Sampling techniques.
- Confidence intervals.
- Hypothesis testing.
- p-values.
- Type I and Type II errors.
- Statistical power.
- Effect size.
- Correlation vs. causation.
- Bootstrap sampling.
- Permutation tests.
- Bias and variance.
- Regression assumptions.
- Statistical uncertainty.
- Multiple comparisons.
- A/B testing fundamentals.

3. Mathematical model and notation

bar xpm t_{1-alpha/2,n-1}frac{s}{sqrt n}

A two-sided t interval uses sample mean, sample standard deviation, and a critical t value under suitable sampling assumptions.

Worked numerical example — follow every step

For observations 4, 6, and 8, the mean is 6 and the sample standard deviation is 2. Standard error is 2/sqrt(3), approximately 1.155; a 95% t interval with two degrees of freedom is wide because the sample is tiny.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
01  import numpy as np
02  from scipy.stats import t
03  x = np.array([4, 7, 5, 6, 8, 5, 7], dtype=float)
04  se = x.std(ddof=1)/np.sqrt(len(x))
05  margin = t.ppf(0.975, df=len(x)-1)*se
06  print('Sample mean:', round(x.mean(), 3))
07  print('95% CI:', tuple(round(z, 3) for z in (x.mean()-margin, x.mean()+margin)))
08  print('Interpretation: repeated intervals cover true means ~95% in the long run.')
```

Verified console output

```
Sample mean: 6.0
95% CI: (np.float64(4.692), np.float64(7.308))
Interpretation: repeated intervals cover true means ~95% in the long run.
```

5. Visual reasoning

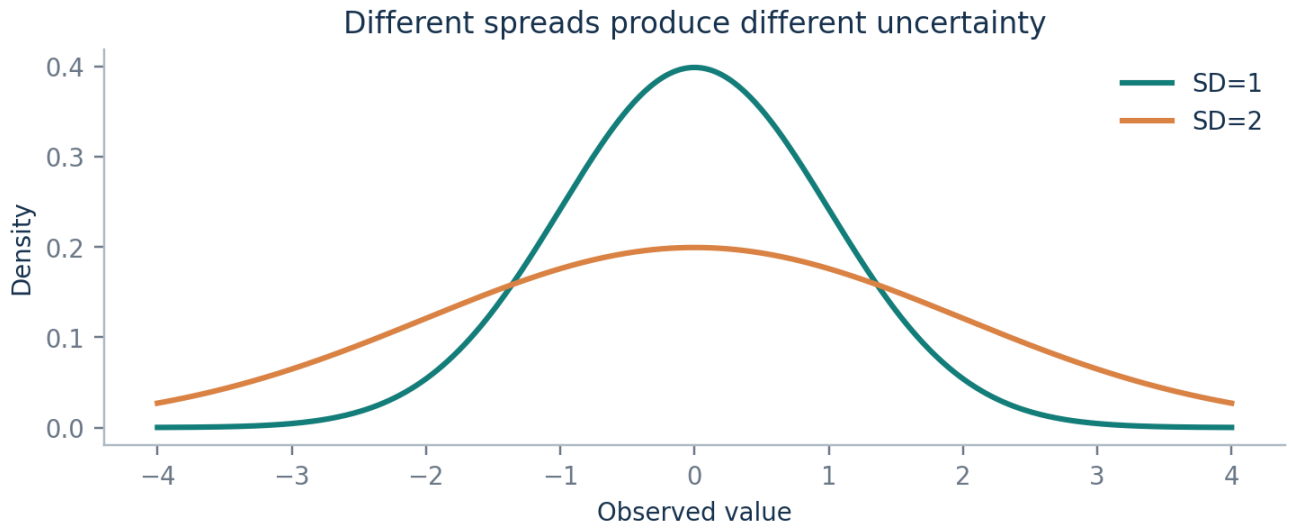


Figure 6. Statistics uses samples to estimate population behavior and quantify uncertainty.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. Do not say a p-value is the probability the null hypothesis is true.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when statistics and statistical inference would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Statistics uses samples to estimate population behavior and quantify uncertainty.

Formula. A two-sided t interval uses sample mean, sample standard deviation, and a critical t value under suitable sampling assumptions.

Practical example. A/B tests compare website variants; model validation assesses uncertainty in estimated accuracy.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for statistics and statistical inference.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 07 / 38 · CORE NOTES

7. Optimization

Learning objective. Explain the purpose of optimization, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Optimization searches for model parameters that minimize a loss or maximize an objective. Batch methods use all examples for each update; stochastic methods use individual or mini-batch gradients. Learning rates govern step size: very large steps diverge, very small steps waste compute. Momentum and adaptive optimizers can accelerate training, but no method removes the need for a sensible objective, normalization, and independent evaluation.

When and where this matters

Fit linear models by gradient descent; train neural networks with AdamW and scheduled learning rates.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Objective functions.
- Loss functions.
- Convex optimization.
- Gradient descent.
- Batch gradient descent.
- Stochastic gradient descent.
- Mini-batch gradient descent.
- Momentum.
- Nesterov acceleration.
- AdaGrad, RMSProp, and Adam.
- Learning rate selection.
- Learning rate scheduling.
- Newton's method.
- Quasi-Newton methods.
- Coordinate descent.
- Constraints and regularization.
- Convergence.
- Saddle points.
- Numerical stability.
- Gradient clipping.

3. Mathematical model and notation

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

eta is the learning rate, J is the objective, and the gradient is evaluated at the current parameters.

Worked numerical example — follow every step

For J(w)=(w-3)^2, start w=0 and choose eta=0.1. The first gradient is 2(0-3)=-6, so w_new=0-0.1(-6)=0.6. The objective falls from 9 to 5.76.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
# Optimize J(w) = (w - 3)^2 with gradient 2(w-3).
w = -1.0
for _ in range(30):
    gradient = 2*(w-3)
    w -= 0.1*gradient
print('Final weight:', round(w, 5))
print('Final loss:', round((w-3)**2, 9))
```

Verified console output

```
Final weight: 2.99505
Final loss: 2.452e-05
```

5. Visual reasoning

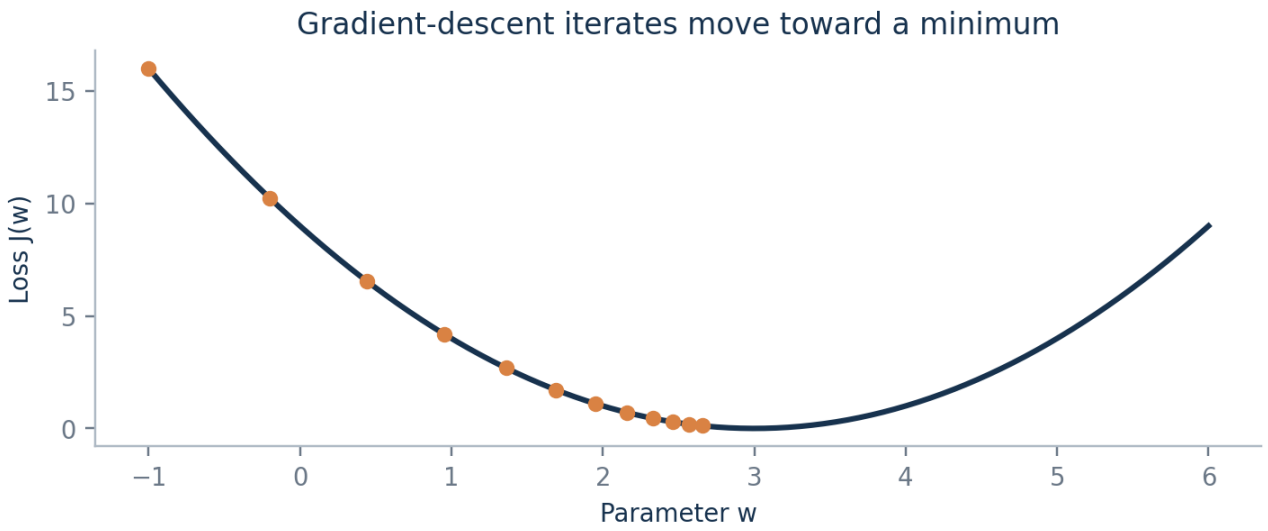


Figure 7. Optimization searches for model parameters that minimize a loss or maximize an objective.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. Do not treat the smallest training loss as proof of the best generalization; monitor held-out performance.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when optimization would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.

- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Optimization searches for model parameters that minimize a loss or maximize an objective.

Formula. eta is the learning rate, J is the objective, and the gradient is evaluated at the current parameters.

Practical example. Fit linear models by gradient descent; train neural networks with AdamW and scheduled learning rates.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for optimization.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 08 / 38 · CORE NOTES

8. Understanding Data

Learning objective. Explain the purpose of understanding data, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Real data contain missing values, duplicates, inconsistent units, unusual values, and biased collection patterns. A useful dataset description identifies observations, features, labels, time stamps, acquisition methods, and known exclusions. The meaning of missingness matters: a sensor may stop reporting exactly when it overheats. Data cleaning must preserve the information relevant to the task without leaking unseen target values into training.

When and where this matters

Electronic sensor streams need unit validation; patient records often require careful handling of missing measurements.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Structured and unstructured data.
- Numerical and categorical variables.
- Ordinal and nominal features.
- Text, images, time series, and sensor data.
- Data collection.
- Data cleaning.
- Data quality.
- Missing values.
- Outliers.
- Duplicate data.
- Class imbalance.
- Data leakage.
- Dataset documentation.

3. Mathematical model and notation

$$r_{missing} = \frac{N_{missing}}{N_{total}}$$

The missingness rate counts unavailable entries divided by all relevant entries.

Worked numerical example — follow every step

If a temperature column has 4 missing readings out of 20, its missingness rate is 4/20=0.20 or 20%. A median from the training set can be an initial imputation baseline.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
01  import numpy as np
02  import pandas as pd
03  df = pd.DataFrame({'temperature':[20., np.nan, 24., 22.], 'fault':[0,0,1,0]})
04  print('Missing values:', df.isna().sum().to_dict())
05  print('Median temperature:', df.temperature.median())
06  print('Class counts:', df.fault.value_counts().sort_index().to_dict())
```

Verified console output

```
Missing values: {'temperature': 1, 'fault': 0}
Median temperature: 22.0
Class counts: {0: 3, 1: 1}
```

5. Visual reasoning

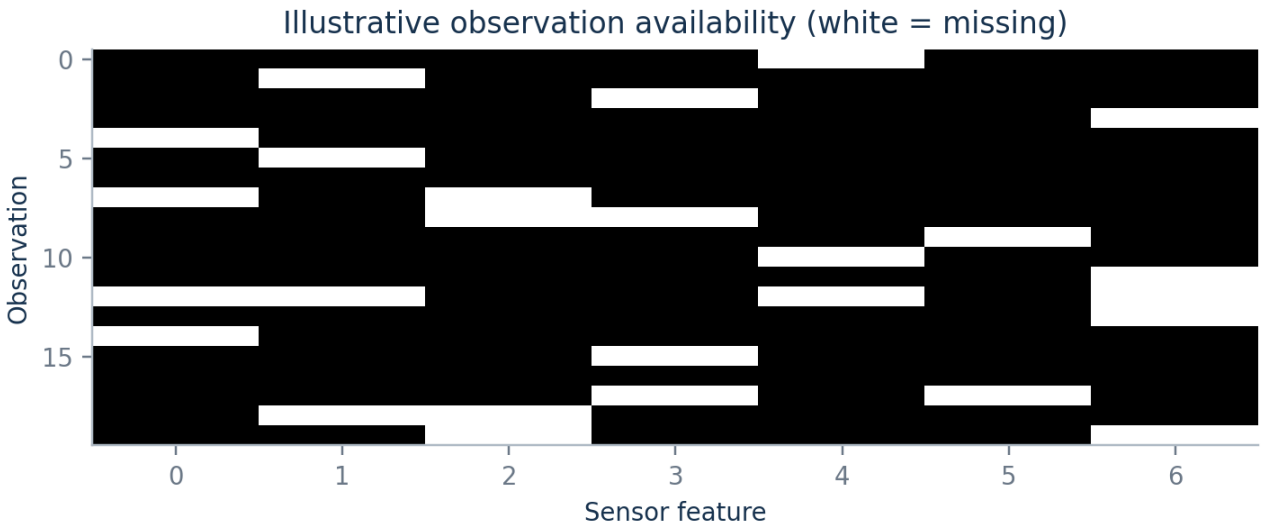


Figure 8. Real data contain missing values, duplicates, inconsistent units, unusual values, and biased collection patterns.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. Do not delete outliers simply because their values are large; in fault detection those rare measurements may be the entire target.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when understanding data would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Real data contain missing values, duplicates, inconsistent units, unusual values, and biased collection patterns.

Formula. The missingness rate counts unavailable entries divided by all relevant entries.

Practical example. Electronic sensor streams need unit validation; patient records often require careful handling of missing measurements.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for understanding data.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 09 / 38 · CORE NOTES

9. Exploratory Data Analysis

Learning objective. Explain the purpose of exploratory data analysis, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Exploratory data analysis (EDA) examines distributions, relationships, unusual values, and potential data problems before model fitting. Histograms reveal skew and multimodality; scatter plots reveal trends and nonlinearities; box plots show spread and outlying observations. Correlation measures linear association but misses many nonlinear structures. EDA is exploratory: patterns chosen after seeing the data should not be presented as pre-registered causal findings.

When and where this matters

Inspect machine vibration distributions; examine retail spend patterns before customer segmentation.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Summary statistics.
- Histograms.
- Scatter plots.
- Box plots.
- Violin plots.
- Pair plots.
- Correlation matrices.
- Feature distributions.
- Grouped analysis.
- Detecting skewness.
- Detecting outliers.
- Visualizing relationships.
- Avoiding misleading correlations.

3. Mathematical model and notation

$$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2}\sqrt{\sum_i (y_i - \bar{y})^2}}$$

Pearson's sample correlation is between -1 and 1 when both sample variances are positive.

Worked numerical example — follow every step

For x=[1,2,3] and y=[2,4,6], each y equals 2x, so their sample Pearson correlation is exactly +1. Correlation of +1 does not establish that x causes y.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
01  import numpy as np
02  import pandas as pd
03  x = np.array([1, 2, 3, 4, 5], dtype=float)
04  y = np.array([2, 4, 5, 8, 11], dtype=float)
05  df = pd.DataFrame({'x': x, 'y': y})
06  print('Feature means:', df.mean().round(2).to_dict())
07  print('Pearson correlation:', round(df.corr().loc['x','y'], 3))
08  print('Caution: correlation alone does not establish causation.')
```

Verified console output

```
Feature means: {'x': 3.0, 'y': 6.0}
Pearson correlation: 0.984
Caution: correlation alone does not establish causation.
```

5. Visual reasoning

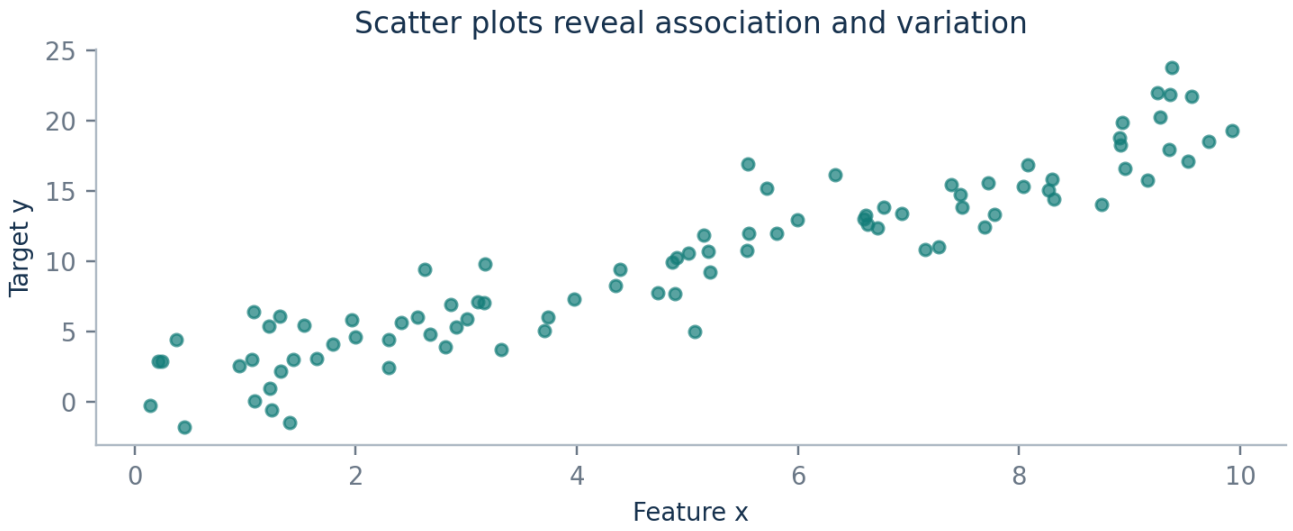


Figure 9. Exploratory data analysis (EDA) examines distributions, relationships, unusual values, and potential data problems before model fitting.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. Plot raw data before trusting a correlation coefficient; one influential outlier can change the summary dramatically.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when exploratory data analysis would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Exploratory data analysis (EDA) examines distributions, relationships, unusual values, and potential data problems before model fitting.

Formula. Pearson's sample correlation is between -1 and 1 when both sample variances are positive.

Practical example. Inspect machine vibration distributions; examine retail spend patterns before customer segmentation.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for exploratory data analysis.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 10 / 38 · CORE NOTES

10. Preprocessing and Feature Engineering

Learning objective. Explain the purpose of preprocessing and feature engineering, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Preprocessing makes raw features suitable for a model. Standardization centers and scales numerical columns; one-hot encoding maps nominal categories into indicator features; imputation handles missing entries; pipelines preserve transformation order. Crucially, every learned preprocessing statistic must be estimated on training folds only. Combining preprocessing and an estimator in one Pipeline reduces leakage and makes deployment reproducible.

When and where this matters

Combine numerical sensor measurements and device categories; build leakage-safe credit risk models.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Train-validation-test splitting.
- Random and stratified sampling.
- Temporal splits.
- Standardization.
- Min-max normalization.
- Robust scaling.
- Missing-value imputation.
- One-hot encoding.
- Ordinal encoding.
- Target encoding and leakage prevention.
- Polynomial features.
- Interaction features.
- Feature selection.
- Feature extraction.
- Dimensionality reduction.
- Data augmentation.
- Imbalanced data handling.
- SMOTE and its limitations.
- scikit-learn Pipelines and ColumnTransformers.

3. Mathematical model and notation

$$z = \frac{x - \mu_{train}}{\sigma_{train}}$$

mu_train and sigma_train are computed only on the training partition and applied unchanged to validation and test data.

Worked numerical example — follow every step

Training temperatures [10,20,30] have mean 20 and population standard deviation approximately 8.165. A new test temperature 40 becomes z=(40-20)/8.165 approximately 2.45 using training statistics.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
01  import numpy as np
02  from sklearn.compose import ColumnTransformer
03  from sklearn.preprocessing import StandardScaler, OneHotEncoder
04  from sklearn.pipeline import Pipeline
05  from sklearn.linear_model import LogisticRegression
06  X = np.array([['10','red'],['20','blue'],['30','red'],['40','blue']])
07  y = np.array([0,0,1,1])
08  pre = ColumnTransformer([('num', StandardScaler(), [0]),
09   ('cat', OneHotEncoder(handle_unknown='ignore'), [1])])
10  pipe = Pipeline([('preprocess', pre), ('model', LogisticRegression())]).fit(X,y)
11  print('Predictions:', pipe.predict(X).tolist())
12  print('Transformed feature count:', pipe.named_steps['preprocess'].transform(X)
.shape[1])
```

Verified console output

```
Predictions: [0, 0, 1, 1]
Transformed feature count: 3
```

5. Visual reasoning

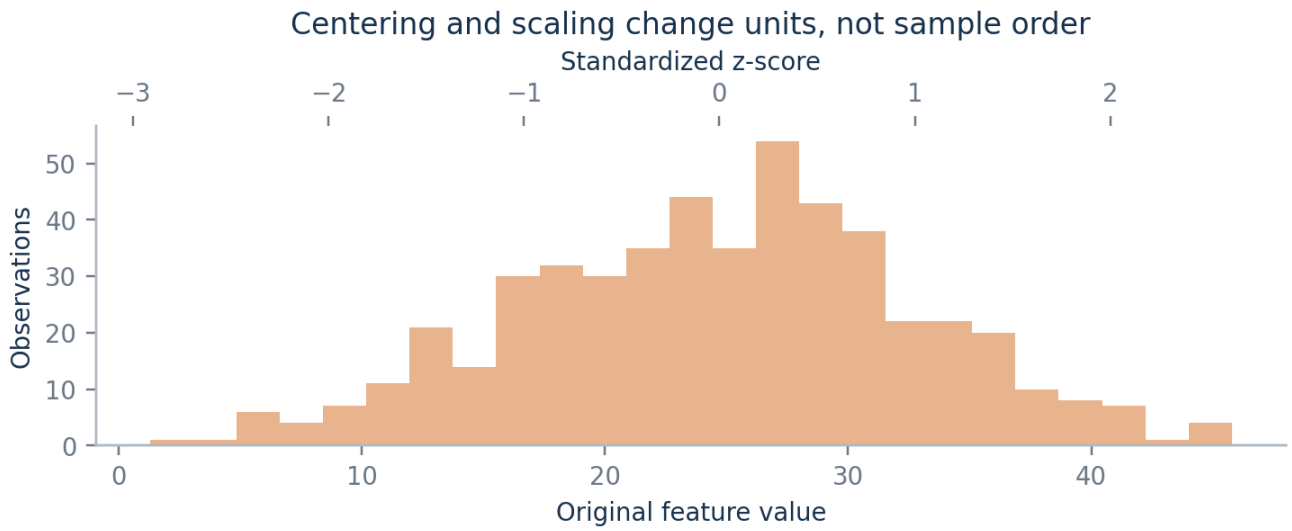


Figure 10. Preprocessing makes raw features suitable for a model.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. Do not fit a scaler, imputer, PCA, or feature selector on the complete dataset before cross-validation.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when preprocessing and feature engineering would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Preprocessing makes raw features suitable for a model.

Formula. mu_train and sigma_train are computed only on the training partition and applied unchanged to validation and test data.

Practical example. Combine numerical sensor measurements and device categories; build leakage-safe credit risk models.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for preprocessing and feature engineering.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 11 / 38 · CORE NOTES

11. Linear Regression

Learning objective. Explain the purpose of linear regression, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Linear regression predicts a continuous response as a weighted sum of inputs. Ordinary least squares chooses coefficients that minimize squared residuals, making large errors disproportionately costly. The model is easy to interpret, fast to train, and a strong baseline. Its assumptions concern how errors behave for statistical inference; predictive usefulness can still be assessed on held-out data when these assumptions fail.

When and where this matters

Estimate home prices from floor area; relate power consumption to operating conditions.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Simple linear regression.
- Multiple linear regression.
- Least squares.
- Mean Squared Error.
- Mean Absolute Error.
- Root Mean Squared Error.
- Coefficient of determination.
- Normal equation.
- Gradient descent implementation.
- Residual analysis.
- Regression assumptions.
- Multicollinearity.
- Confidence and prediction intervals.

3. Mathematical model and notation

J(w,b)=frac1nsum_{i=1}^{n}(y_i-w^Tx_i-b)^2

J is mean squared error, w is the coefficient vector, b is intercept, and n is the number of observations.

Worked numerical example — follow every step

For points (1,3),(2,5),(3,7), the exact line is y=2x+1. At x=4 it predicts 9; each training residual is zero, so training MSE is 0. This ideal fit says nothing about noisy future data.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
01  import numpy as np
02  from sklearn.linear_model import LinearRegression
```

```
x = np.array([[1.],[2.],[3.],[4.]])
y = np.array([3.,5.,7.,9.])
model = LinearRegression().fit(x,y)
print('Slope:', round(float(model.coef_[0]), 4))
print('Intercept:', round(float(model.intercept_), 4))
print('Prediction for x=5:', round(float(model.predict([[5.]])[0]), 4))
```

Verified console output

```
Slope: 2.0
Intercept: 1.0
Prediction for x=5: 11.0
```

5. Visual reasoning

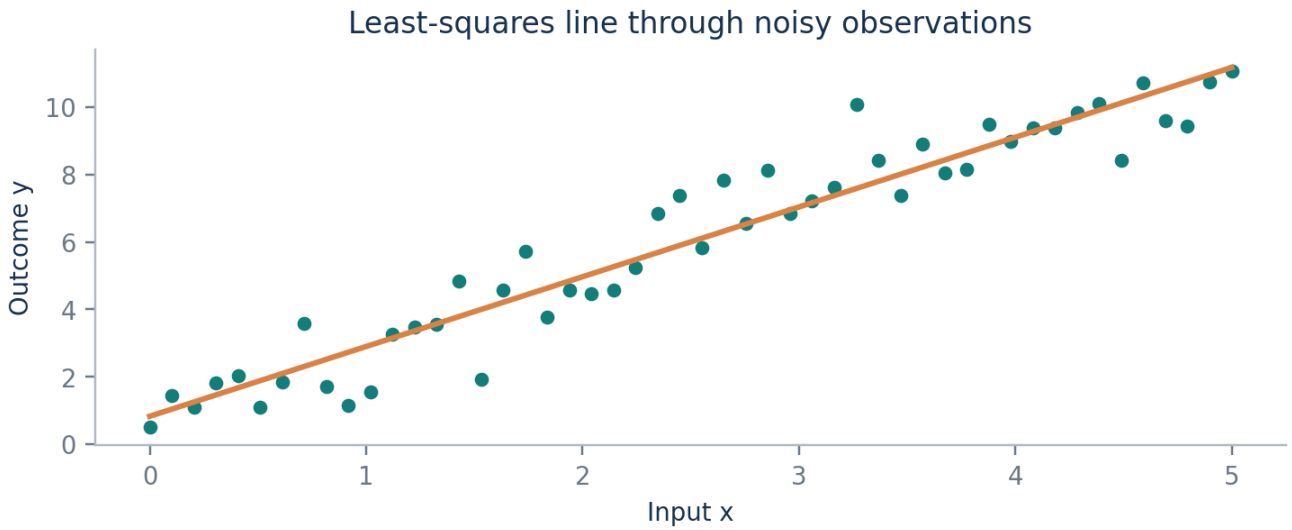


Figure 11. Linear regression predicts a continuous response as a weighted sum of inputs.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. An R-squared value near 1 on training data does not establish an unbiased or reliable causal model.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when linear regression would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Linear regression predicts a continuous response as a weighted sum of inputs.

Formula. J is mean squared error, w is the coefficient vector, b is intercept, and n is the number of observations.

Practical example. Estimate home prices from floor area; relate power consumption to operating conditions.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for linear regression.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 12 / 38 · CORE NOTES

12. Regularized and Advanced Regression

Learning objective. Explain the purpose of regularized and advanced regression, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Regularized regression modifies the loss to discourage overly complex coefficients. Ridge penalizes squared coefficient size and usually shrinks weights continuously; Lasso penalizes absolute magnitude and can set some coefficients exactly to zero. Elastic Net combines both behaviors. Nonlinear and robust alternatives address curvature, outliers, count targets, and heteroscedasticity. Compare alternatives with properly nested preprocessing and validation.

When and where this matters

Choose stable coefficients among correlated biomarkers; forecast loads from many correlated sensor channels.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Ridge regression.
- Lasso regression.
- Elastic Net.
- Polynomial regression.
- Generalized linear models.
- Poisson regression.
- Robust regression.
- Quantile regression.
- Support Vector Regression.
- Kernel regression.
- Regression trees.
- Random Forest Regression.
- Gradient Boosted Regression.
- Model comparison and interpretation.

3. Mathematical model and notation

J_{ridge}(w)=frac1nlVert y-XwrVert_2^2+lambdalVert wrVert_2^2

lambda >= 0 controls the L2 penalty; larger penalties generally shrink learned coefficients more strongly.

Worked numerical example — follow every step

If weights are [3,4], their L2 squared penalty is 3^2+4^2=25. At lambda=0.1 the regularization term adds 2.5 to the data-fit loss.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
01  import numpy as np
02  from sklearn.linear_model import Ridge, Lasso
```

```
X = np.array([[1.,1.],[2.,1.],[3.,2.],[4.,3.],[5.,5.]])
y = np.array([2.,3.,5.,7.,9.])
for model in [Ridge(alpha=1.0), Lasso(alpha=0.1)]:
    model.fit(X,y)
    print(type(model).__name__, 'coefficients:', model.coef_.round(3).tolist())
```

Verified console output

```
Ridge coefficients: [0.982, 0.719]
Lasso coefficients: [1.251, 0.499]
```

5. Visual reasoning

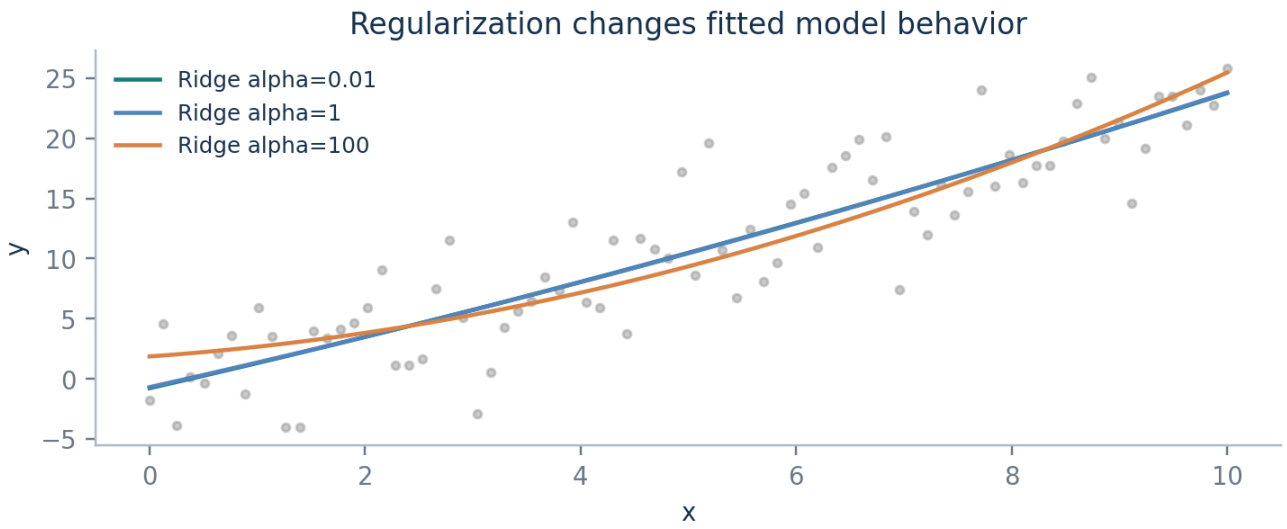


Figure 12. Regularized regression modifies the loss to discourage overly complex coefficients.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. Do not compare regularization strengths before standardizing predictors with very different scales.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when regularized and advanced regression would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Regularized regression modifies the loss to discourage overly complex coefficients.

Formula. lambda >= 0 controls the L2 penalty; larger penalties generally shrink learned coefficients more strongly.

Practical example. Choose stable coefficients among correlated biomarkers; forecast loads from many correlated sensor channels.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for regularized and advanced regression.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 13 / 38 · CORE NOTES

13. Classification Fundamentals

Learning objective. Explain the purpose of classification fundamentals, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Classification predicts a category or a probability for each category. Binary tasks distinguish two labels; multiclass tasks select one of several; multilabel tasks allow several labels simultaneously. Decision thresholds translate probabilities into actions, and the right threshold depends on the cost of false positives and false negatives. Imbalanced tasks require precision-recall analysis and calibrated probabilities, not accuracy alone.

When and where this matters

Identify defective products; prioritize screening tests for follow-up review.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Binary classification.
- Multiclass classification.
- Multilabel classification.
- Decision thresholds.
- Class probabilities.
- Sigmoid and softmax.
- Logistic loss.
- Class imbalance.
- Cost-sensitive classification.
- Probability calibration.

3. Mathematical model and notation

hat y=mathbf1[hat pgetau]

The decision is positive when predicted probability p reaches threshold tau; altering tau changes the precision-recall tradeoff.

Worked numerical example — follow every step

For probabilities [0.2,0.6,0.8] and threshold 0.5, predictions are [0,1,1]. Raising the threshold to 0.7 changes them to [0,0,1]. No model parameter was retrained.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
01  import numpy as np
02  from sklearn.metrics import precision_score, recall_score
03  truth = np.array([0,0,0,1,1,1])
04  prob = np.array([0.1,0.4,0.55,0.45,0.75,0.9])
05  for threshold in [0.5,0.7]:
```

```
06      pred = (prob >= threshold).astype(int)
07      print('Threshold', threshold, 'precision',round(precision_score(truth,pred),3),
08            'recall',round(recall_score(truth,pred),3))
```

Verified console output

```
Threshold 0.5 precision 0.667 recall 0.667
Threshold 0.7 precision 1.0 recall 0.667
```

5. Visual reasoning

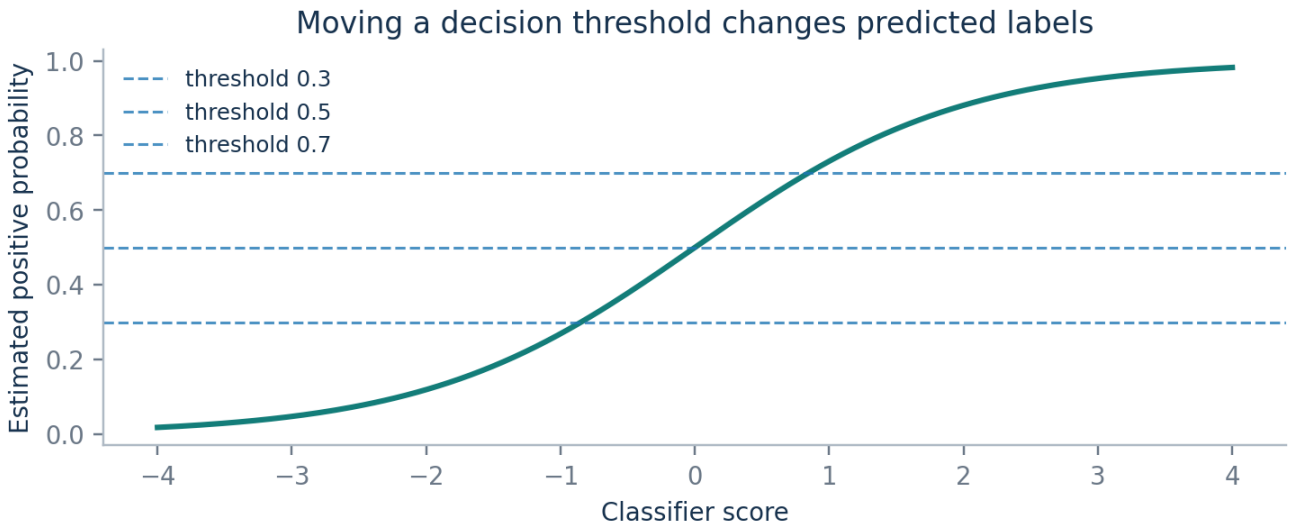


Figure 13. Classification predicts a category or a probability for each category.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. Do not call classifier confidence a calibrated probability without checking calibration on relevant held-out data.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when classification fundamentals would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Classification predicts a category or a probability for each category.

Formula. The decision is positive when predicted probability p reaches threshold tau; altering tau changes the precision-recall tradeoff.

Practical example. Identify defective products; prioritize screening tests for follow-up review.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for classification fundamentals.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 14 / 38 · CORE NOTES

14. Logistic Regression

Learning objective. Explain the purpose of logistic regression, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Logistic regression is a linear classifier in log-odds space. It calculates a weighted sum of inputs, converts that score through the sigmoid, and learns parameters by minimizing log loss. The model is efficient, often interpretable, and gives a strong classification baseline. Its decision boundary is linear in the supplied feature representation, although feature engineering can create nonlinear boundaries in the original space.

When and where this matters

Predict equipment pass/fail status; estimate click-through likelihood for a campaign.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Logistic function.
- Log-odds.
- Logistic regression model.
- Bernoulli likelihood.
- Cross-entropy loss.
- Gradient derivation.
- Binary classification.
- Multinomial logistic regression.
- Regularization.
- Decision boundaries.
- Probability interpretation.

3. Mathematical model and notation

$$P(y = 1 \mid x) = \sigma(w^T x + b) = \frac{1}{1 + e^{-(w^T x + b)}}$$

The sigmoid maps any real-valued score into (0,1). Cross-entropy fits probabilities to observed labels.

Worked numerical example — follow every step

If the logit score is 0, sigmoid(0)=1/2. If score is log(3), the probability is 3/(1+3)=0.75 and the odds are 3:1.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
01  import numpy as np
02  from sklearn.linear_model import LogisticRegression
```

```
03  X = np.array([[0.],[1.],[2.],[3.],[4.],[5.]])
04  y = np.array([0,0,0,1,1,1])
05  model = LogisticRegression().fit(X,y)
06  print('Class probabilities at x=2.5:', model.predict_proba([[2.5]]).round(3)
.tolist())
07  print('Prediction at x=4:', model.predict([[4]]).tolist())
```

Verified console output

```
Class probabilities at x=2.5: [[0.5, 0.5]]
Prediction at x=4: [1]
```

5. Visual reasoning

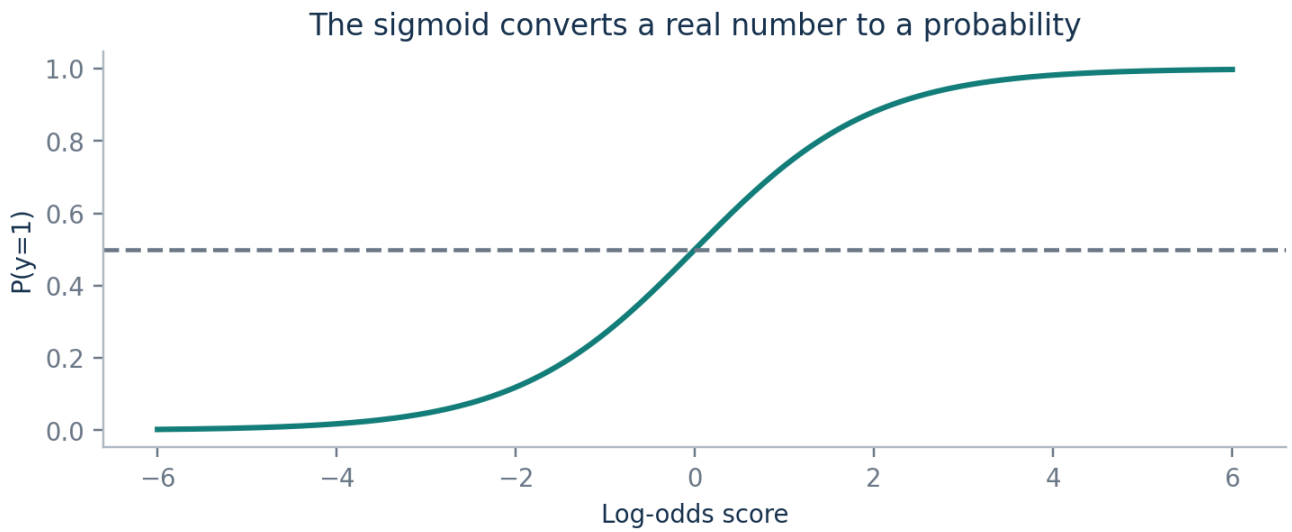


Figure 14. Logistic regression is a linear classifier in log-odds space.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. Coefficients change log-odds, not probability by the same fixed amount across all input values.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when logistic regression would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Logistic regression is a linear classifier in log-odds space.

Formula. The sigmoid maps any real-valued score into (0,1). Cross-entropy fits probabilities to observed labels.

Practical example. Predict equipment pass/fail status; estimate click-through likelihood for a campaign.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for logistic regression.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 15 / 38 · CORE NOTES

15. k-Nearest Neighbors

Learning objective. Explain the purpose of k-nearest neighbors, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

k-nearest neighbors (kNN) predicts from nearby labeled examples under a chosen distance. Small k captures fine local boundaries but is sensitive to noise; large k smooths predictions and can miss rare groups. Distances must be meaningful: features with huge numeric scales can dominate, and high-dimensional data can make neighborhoods less informative. kNN stores training examples and performs much of its work at prediction time.

When and where this matters

Classify small image embeddings; flag unusual nearby operating conditions in factory sensors.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Distance metrics.
- Euclidean and Manhattan distances.
- Minkowski distance.
- Feature scaling.
- Choosing k.
- Weighted kNN.
- Classification and regression.
- Curse of dimensionality.
- Computational efficiency.

3. Mathematical model and notation

$$d_2(x, z) = \sqrt{\sum_{j=1}^{p} (x_j - z_j)^2}$$

Euclidean distance measures the straight-line separation of two p-dimensional vectors.

Worked numerical example — follow every step

For points x=[1,2] and z=[4,6], Euclidean distance is sqrt((4-1)^2+(6-2)^2)=sqrt(9+16)=5.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
01  import numpy as np
02  from sklearn.neighbors import KNeighborsClassifier
03  X = np.array([[0,0],[1,0],[0,1],[4,4],[5,4],[4,5]])
```

```
y = np.array([0,0,0,1,1,1])
model = KNeighborsClassifier(n_neighbors=3).fit(X,y)
print('Neighbor distances:',np.linalg.norm(X-[0.5,0.5],axis=1).round(3).tolist())
print('Predicted class:', model.predict([[0.5,0.5]]).tolist())
```

Verified console output

```
Neighbor distances: [0.707, 0.707, 0.707, 4.95, 5.701, 5.701]
Predicted class: [0]
```

5. Visual reasoning

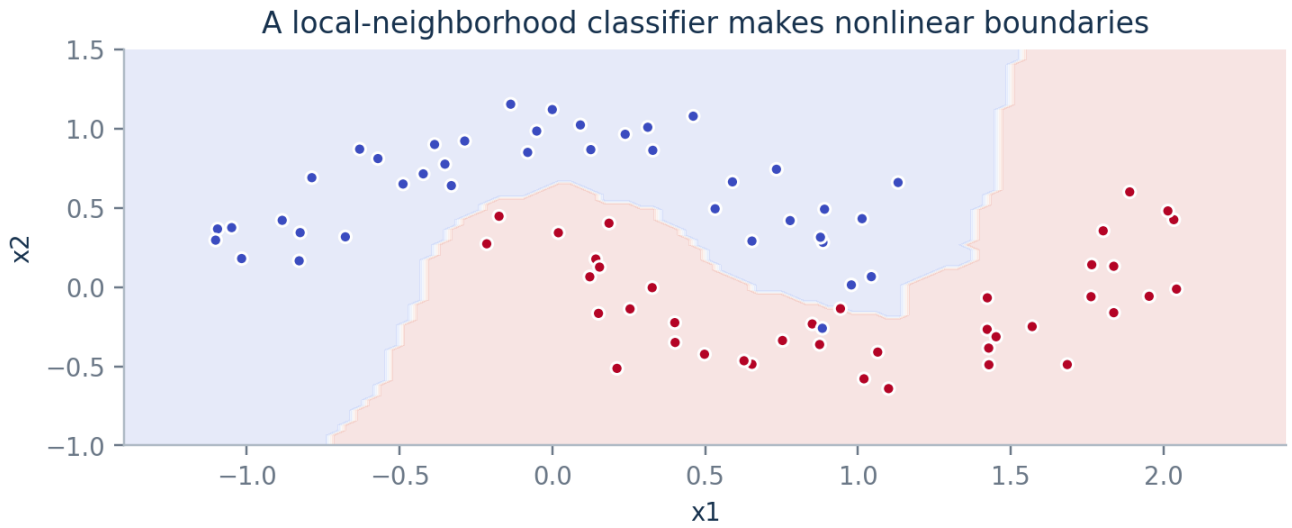


Figure 15. k-nearest neighbors (kNN) predicts from nearby labeled examples under a chosen distance.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. Always scale numerical features appropriately; otherwise distance can reflect units more than useful similarity.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when k-nearest neighbors would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. k-nearest neighbors (kNN) predicts from nearby labeled examples under a chosen distance.

Formula. Euclidean distance measures the straight-line separation of two p-dimensional vectors.

Practical example. Classify small image embeddings; flag unusual nearby operating conditions in factory sensors.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for k-nearest neighbors.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 16 / 38 · CORE NOTES

16. Naive Bayes

Learning objective. Explain the purpose of naive bayes, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Naive Bayes applies Bayes' rule with a simplifying assumption that features are conditionally independent given a class. This assumption is often unrealistic yet can yield effective, fast baselines, especially for text counts. Gaussian Naive Bayes models continuous features; Multinomial Naive Bayes handles nonnegative counts; Bernoulli Naive Bayes handles binary indicators. Smoothing prevents probabilities from becoming zero when a word is absent in one training class.

When and where this matters

Filter unsolicited email; classify technical support tickets.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Bayes' theorem.
- Conditional independence assumption.
- Gaussian Naive Bayes.
- Multinomial Naive Bayes.
- Bernoulli Naive Bayes.
- Laplace smoothing.
- Text classification example.

3. Mathematical model and notation

$$P(y \mid x) \propto P(y)\prod_{j} P(x_j \mid y)$$

The product assumes conditional feature independence within each class; proportionality suppresses the common evidence term.

Worked numerical example — follow every step

Suppose P(spam)=0.4, P(free|spam)=0.8, P(free|not spam)=0.1. Then P(spam|free)=0.4(0.8)/[0.4(0.8)+0.6(0.1)]=0.8421.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
01  from sklearn.feature_extraction.text import CountVectorizer
02  from sklearn.naive_bayes import MultinomialNB
03  texts = ['free money now','claim free prize','meeting at noon','project meeting
today']
04  y = [1,1,0,0]
```

```
05  vectorizer = CountVectorizer()
06  model = MultinomialNB().fit(vectorizer.fit_transform(texts), y)
07  print('Vocabulary size:',len(vectorizer.vocabulary_))
08  print('Spam predictions:', model.predict(vectorizer.transform(['free prize','team
meeting'])).tolist())
```

Verified console output

```
Vocabulary size: 10
Spam predictions: [1, 0]
```

5. Visual reasoning

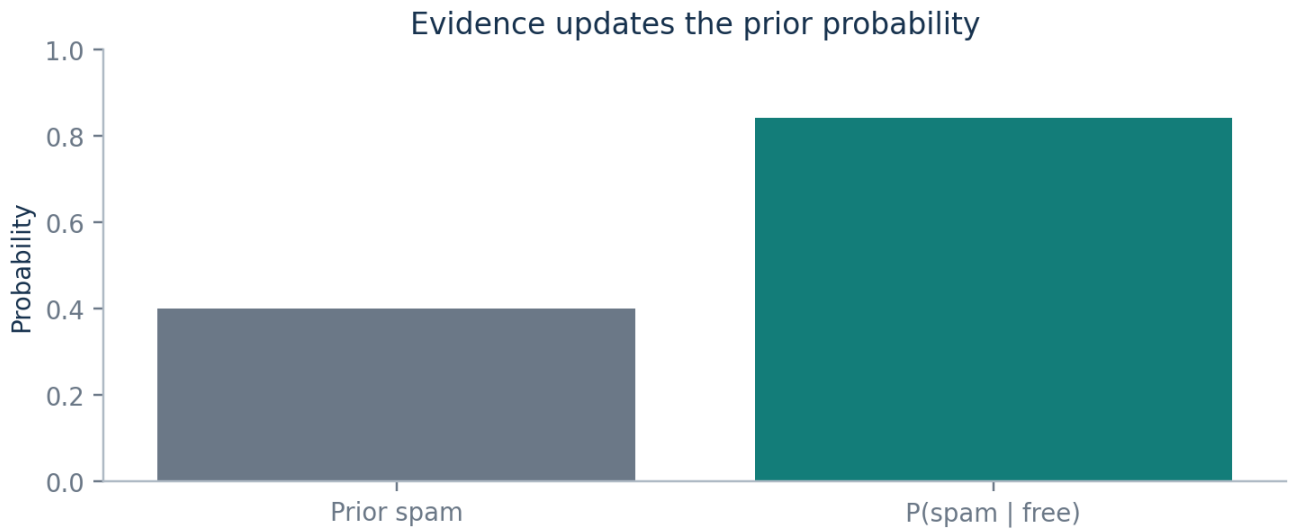


Figure 16. Naive Bayes applies Bayes' rule with a simplifying assumption that features are conditionally independent given a class.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. Independence is a modeling assumption, not a claim that words are actually independent.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when naive bayes would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Naive Bayes applies Bayes' rule with a simplifying assumption that features are conditionally independent given a class.

Formula. The product assumes conditional feature independence within each class; proportionality suppresses the common evidence term.

Practical example. Filter unsolicited email; classify technical support tickets.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for naive bayes.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 17 / 38 · CORE NOTES

17. Support Vector Machines

Learning objective. Explain the purpose of support vector machines, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

A support vector machine searches for a decision boundary with a large geometric margin. The soft-margin formulation balances margin width with classification violations. Kernels compare inputs in an implicit feature space, allowing nonlinear boundaries without explicitly constructing all expanded features. SVMs can work well on modest-sized datasets; training and tuning may become expensive on very large datasets.

When and where this matters

Classify spectral signatures; identify medical diagnostic categories in small feature sets.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Hyperplanes.
- Maximum margin classification.
- Hard-margin SVM.
- Soft-margin SVM.
- Hinge loss.
- Kernel trick.
- Linear, polynomial, and RBF kernels.
- Support vectors.
- Hyperparameter selection.
- Support Vector Regression.

3. Mathematical model and notation

min_{w,b,xi}frac12lVert wrVert_2^2+Csum_ixi_i

The soft-margin objective minimizes squared weight norm plus penalty-weighted slack variables under margin constraints.

Worked numerical example — follow every step

For hyperplane 2x+0y-4=0, the Euclidean distance from point (3,1) is |2(3)-4|/sqrt(2^2)=1.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
from sklearn.datasets import make_moons
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
X,y = make_moons(n_samples=300, noise=0.15, random_state=7)
Xtr,Xte,ytr,yte = train_test_split(X,y,random_state=7)
for kernel in ['linear','rbf']:
    clf=SVC(kernel=kernel,gamma='scale').fit(Xtr,ytr)
```

```
08      print(kernel, 'test accuracy:', round(clf.score(Xte,yte),3), 'support vectors:',
len(clf.support_))
```

Verified console output

```
linear test accuracy: 0.92 support vectors: 78
rbf test accuracy: 0.973 support vectors: 51
```

5. Visual reasoning

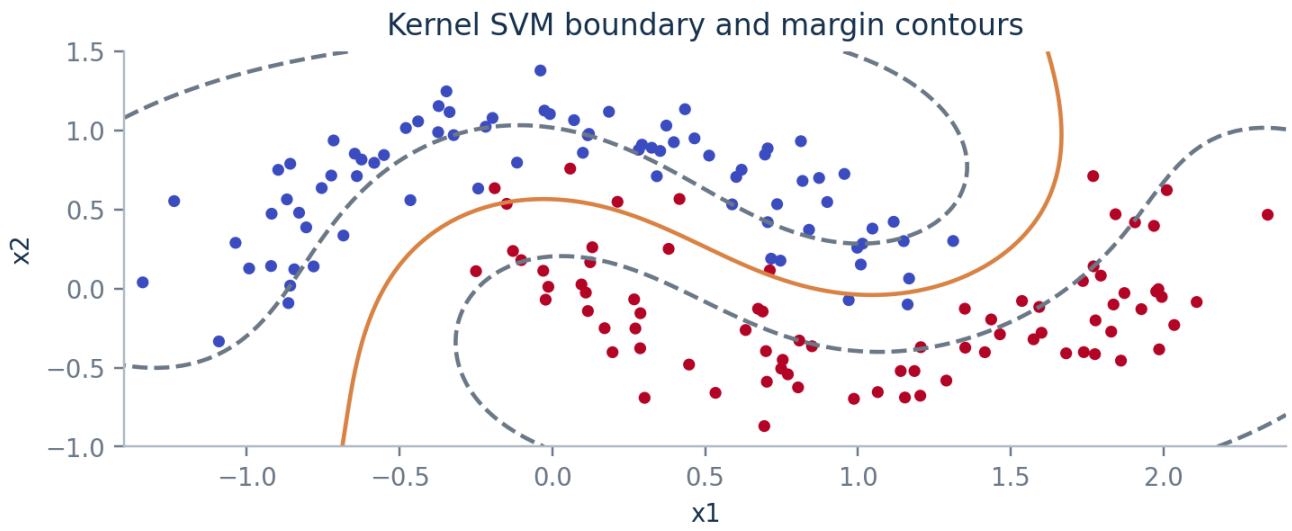


Figure 17. A support vector machine searches for a decision boundary with a large geometric margin.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. A larger C generally reduces allowed training violations but can increase overfitting; C is not a learning rate.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when support vector machines would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. A support vector machine searches for a decision boundary with a large geometric margin.

Formula. The soft-margin objective minimizes squared weight norm plus penalty-weighted slack variables under margin constraints.

Practical example. Classify spectral signatures; identify medical diagnostic categories in small feature sets.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for support vector machines.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 18 / 38 · CORE NOTES

18. Decision Trees

Learning objective. Explain the purpose of decision trees, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Decision trees recursively split data into increasingly homogeneous regions. For classification, split quality can use Gini impurity or entropy; for regression, splits usually reduce within-leaf prediction error. Trees encode readable if/else rules and handle nonlinear interactions without explicit polynomial terms. Deep trees can memorize noise, so depth limits, minimum leaf sizes, pruning, and validation are important.

When and where this matters

Create interpretable service-triage rules; predict manufacturing defects from machine settings.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Decision rules.
- Recursive partitioning.
- Entropy.
- Information gain.
- Gini impurity.
- Tree construction.
- Regression trees.
- Overfitting.
- Pruning.
- Maximum depth.
- Feature importance.
- Decision tree visualization.

3. Mathematical model and notation

$$Gini = 1 - \sum_{k=1}^{K} p_k^2$$

p_k is the share of class k in a node; a pure node has Gini impurity zero.

Worked numerical example — follow every step

In a node containing 3 positive and 1 negative examples, class proportions are 0.75 and 0.25. Gini=1-(0.75^2+0.25^2)=0.375.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
01  from sklearn.tree import DecisionTreeClassifier, export_text
02  X=[[1],[2],[3],[4],[5],[6]]
03  y=[0,0,0,1,1,1]
04  tree=DecisionTreeClassifier(max_depth=2,random_state=0).fit(X,y)
05  print(export_text(tree,feature_names=['signal']).strip())
06  print('Prediction at signal=5:',tree.predict([[5]]).tolist())
```

Verified console output

```
|--- signal <= 3.50
|   |--- class: 0
|--- signal >  3.50
|   |--- class: 1
Prediction at signal=5: [1]
```

5. Visual reasoning

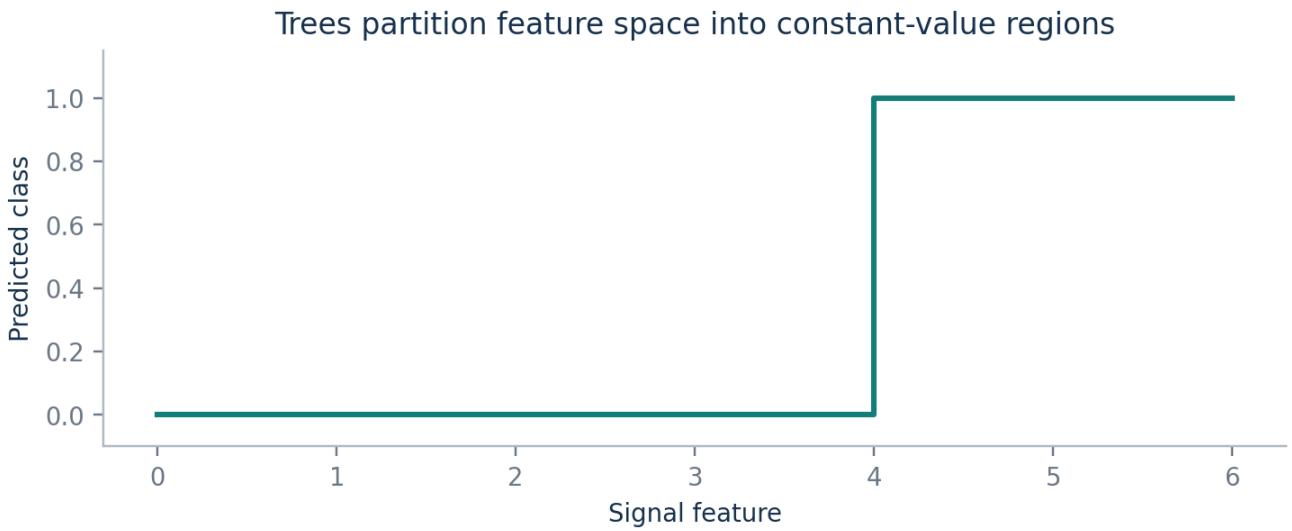


Figure 18. Decision trees recursively split data into increasingly homogeneous regions.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. Feature importance based on impurity decrease can favor high-cardinality features; inspect holdout permutation importance too.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when decision trees would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Decision trees recursively split data into increasingly homogeneous regions.

Formula. p_k is the share of class k in a node; a pure node has Gini impurity zero.

Practical example. Create interpretable service-triage rules; predict manufacturing defects from machine settings.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for decision trees.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 19 / 38 · CORE NOTES

19. Ensemble Methods

Learning objective. Explain the purpose of ensemble methods, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Ensembles aggregate multiple models to improve performance or reliability. Bagging fits learners on resampled data and reduces variance; random forests add randomness in feature selection. Boosting adds successive weak learners that correct earlier errors, often giving strong performance on tabular data. Stacking trains a meta-model on out-of-fold base predictions. Proper evaluation matters because a complex ensemble can quietly overfit.

When and where this matters

Tabular credit-risk classification; sensor-based predictive maintenance.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Bagging.
- Random Forests.
- Extra Trees.
- Boosting.
- AdaBoost.
- Gradient Boosting.
- XGBoost.
- LightGBM.
- CatBoost.
- Voting classifiers.
- Stacking.
- Out-of-bag evaluation.
- Bias-variance tradeoffs.

3. Mathematical model and notation

hat f(x)=frac1Bsum_{b=1}^{B}f_b(x)

A regression bagging ensemble averages predictions from B component models; classifiers may average probabilities or vote.

Worked numerical example — follow every step

If three regression trees predict 12, 16, and 14, the bagged mean is (12+16+14)/3=14.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
01  from sklearn.datasets import load_wine
02  from sklearn.model_selection import train_test_split
```

```
03  from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
04  X,y=load_wine(return_X_y=True)
05  a,b,c,d=train_test_split(X,y,random_state=7,stratify=y)
06  for clf in [RandomForestClassifier(n_estimators=50,random_state=7),
GradientBoostingClassifier(random_state=7)]:
07      clf.fit(a,c)
08      print(type(clf).__name__, 'test accuracy:', round(clf.score(b,d),3))
```

Verified console output

```
RandomForestClassifier test accuracy: 0.978
GradientBoostingClassifier test accuracy: 0.978
```

5. Visual reasoning

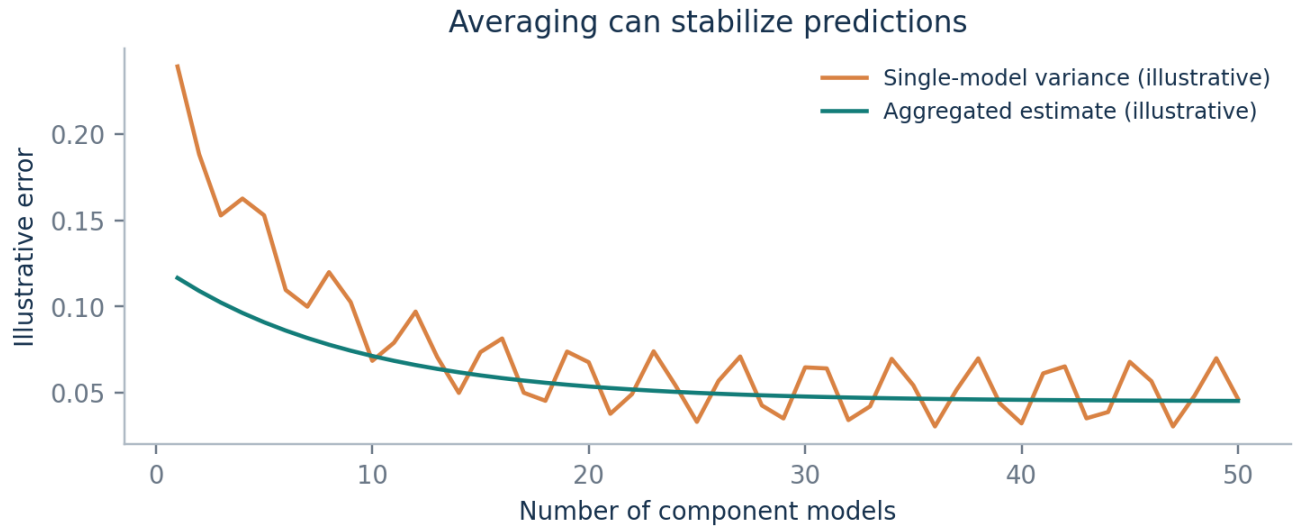


Figure 19. Ensembles aggregate multiple models to improve performance or reliability.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. Never train a stacking meta-model on the same in-sample base predictions used to fit its component models.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when ensemble methods would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Ensembles aggregate multiple models to improve performance or reliability.

Formula. A regression bagging ensemble averages predictions from B component models; classifiers may average probabilities or vote.

Practical example. Tabular credit-risk classification; sensor-based predictive maintenance.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for ensemble methods.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 20 / 38 · CORE NOTES

20. Clustering

Learning objective. Explain the purpose of clustering, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Clustering groups observations based on a similarity model without requiring labels. K-means minimizes squared distance to cluster centers and assumes meaningful Euclidean geometry. Density-based methods find connected dense regions and can label noise; mixture models assign soft probabilistic cluster membership. Different algorithms can disagree because clustering is not a single objectively defined truth.

When and where this matters

Group customers by purchasing patterns; discover different operational states in sensor signals.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- k-Means clustering.
- k-Means++ initialization.
- Hierarchical clustering.
- Agglomerative clustering.
- DBSCAN.
- HDBSCAN overview.
- Gaussian Mixture Models.
- Expectation-Maximization.
- Spectral clustering.
- Density-based clustering.
- Choosing the number of clusters.
- Silhouette score.
- Davies-Bouldin index.
- Clustering limitations.

3. Mathematical model and notation

$$J=\sum_{i=1}^{n}\lVert x_i-\mu_{c_i}\rVert_2^2$$

K-means minimizes the within-cluster sum of squared distances to assigned center mu.

Worked numerical example — follow every step

For one-dimensional observations 1 and 3, their mean center is 2, giving squared errors $(1-2)^2+(3-2)^2=2$.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
01  import numpy as np
02  from sklearn.cluster import KMeans
03  X=np.array([[0,0],[0,1],[1,0],[8,8],[8,9],[9,8]], dtype=float)
04  km=KMeans(n_clusters=2,random_state=42,n_init=10).fit(X)
```

```
print('Cluster centers:',km.cluster_centers_.round(2).tolist())
print('Inertia:',round(km.inertia_,3))
print('Assignments:',km.labels_.tolist())
```

Verified console output

```
Cluster centers: [[0.33, 0.33], [8.33, 8.33]]
Inertia: 2.667
Assignments: [0, 0, 0, 1, 1, 1]
```

5. Visual reasoning

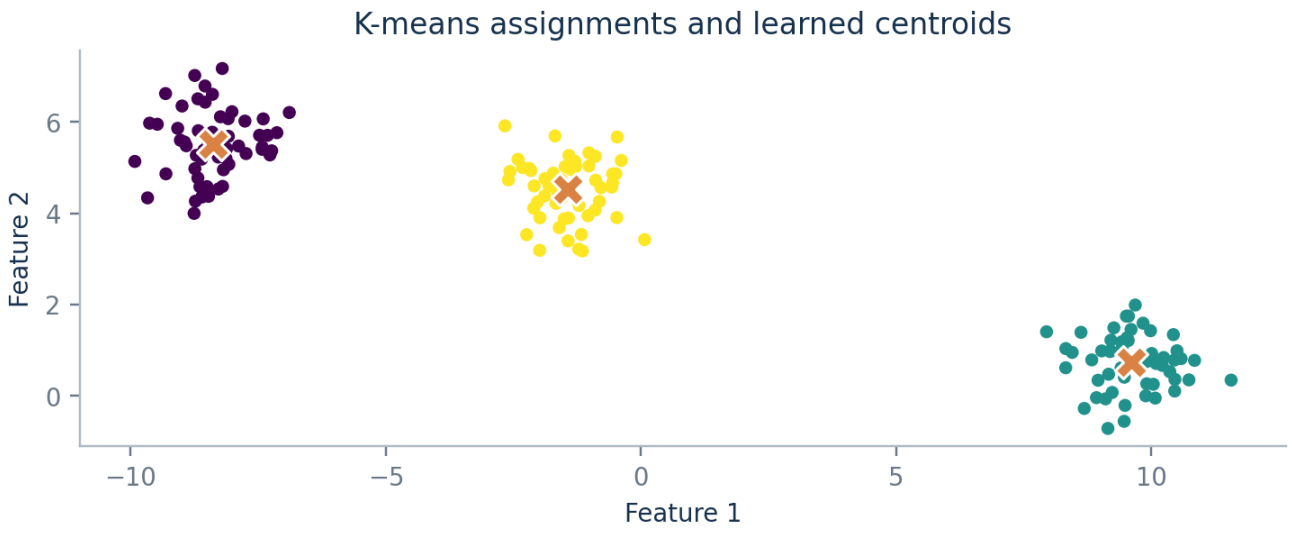


Figure 20. Clustering groups observations based on a similarity model without requiring labels.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. Cluster labels are arbitrary identifiers; cluster 0 is not inherently better or larger than cluster 1.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when clustering would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Clustering groups observations based on a similarity model without requiring labels.

Formula. K-means minimizes the within-cluster sum of squared distances to assigned center mu.

Practical example. Group customers by purchasing patterns; discover different operational states in sensor signals.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for clustering.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 21 / 38 · CORE NOTES

21. Dimensionality Reduction

Learning objective. Explain the purpose of dimensionality reduction, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Dimensionality reduction converts many variables into fewer representations while preserving selected properties. PCA finds orthogonal directions that explain maximum variance after suitable centering. t-SNE and UMAP emphasize local neighborhoods for visualization, but plot distances and cluster separations may be misleading. Select the method based on whether you need compression, noise reduction, visual exploration, or supervised prediction.

When and where this matters

Compress spectral data; visualize image-feature embeddings.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Curse of dimensionality.
- Principal Component Analysis.
- Covariance matrices.
- Eigenvectors and eigenvalues.
- SVD-based PCA.
- Explained variance.
- Linear Discriminant Analysis.
- t-SNE.
- UMAP.
- Autoencoders.
- Random projections.
- Reconstruction error.
- Visualization pitfalls.

3. Mathematical model and notation

$$z = X_c v_1$$

The first PCA score projects centered observations X_c onto the leading unit-variance direction v_1.

Worked numerical example — follow every step

The centered points (-1,-1),(0,0),(1,1) lie along the diagonal. The leading PCA direction is [1,1]/sqrt(2), retaining all nonzero variance in one component.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
01  import numpy as np
02  from sklearn.decomposition import PCA
03  X=np.array([[1,1],[2,2],[3,3],[4,4]],dtype=float)
04  pca=PCA(n_components=1).fit(X)
05  print('Explained variance ratio:',pca.explained_variance_ratio_.round(4).tolist())
06  print('First direction:',np.abs(pca.components_[0]).round(4).tolist())
07  print('Reduced shape:',pca.transform(X).shape)
```

Verified console output

```
Explained variance ratio: [1.0]
First direction: [0.7071, 0.7071]
Reduced shape: (4, 1)
```

5. Visual reasoning

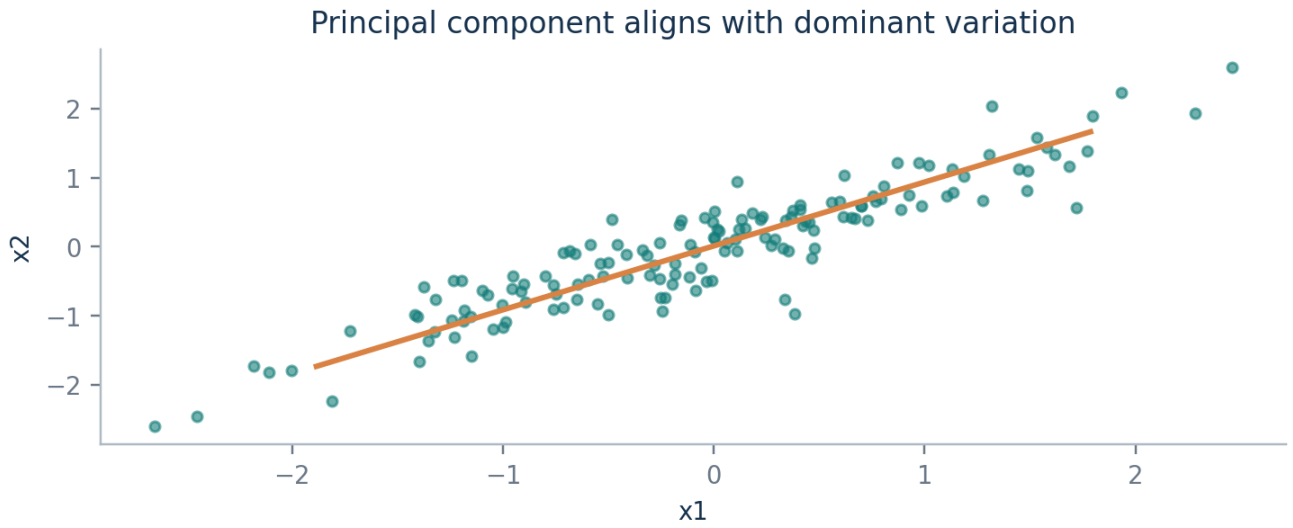


Figure 21. Dimensionality reduction converts many variables into fewer representations while preserving selected properties.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. Fit PCA on the training split only; and do not interpret t-SNE axes as meaningful original measurements.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when dimensionality reduction would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Dimensionality reduction converts many variables into fewer representations while preserving selected properties.

Formula. The first PCA score projects centered observations X_c onto the leading unit-variance direction v_1.

Practical example. Compress spectral data; visualize image-feature embeddings.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for dimensionality reduction.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 22 / 38 · CORE NOTES

22. Anomaly Detection

Learning objective. Explain the purpose of anomaly detection, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Anomaly detection identifies rare or unusual observations relative to a reference distribution. It differs from ordinary classification because labeled anomalies may be scarce or unrepresentative. Statistical thresholds, density models, Isolation Forest, One-Class SVM, and reconstruction models embody different assumptions. Novelty detection fits normal data, then judges new points; outlier detection may include contaminated training records.

When and where this matters

Detect credit-card fraud; identify unusual vibration patterns in rotating motors.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Statistical outlier detection.
- Z-score.
- Interquartile range.
- Isolation Forest.
- One-Class SVM.
- Local Outlier Factor.
- Reconstruction-based methods.
- Novelty detection.
- Evaluation with rare events.

3. Mathematical model and notation

$$z = \frac{x - \mu}{\sigma}$$

A z-score measures deviation from a reference mean in units of reference standard deviation.

Worked numerical example — follow every step

If baseline temperature has mean 50 C and standard deviation 5 C, a 65 C reading has z=(65-50)/5=3, warranting investigation under a three-sigma heuristic.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
01  import numpy as np
02  from sklearn.ensemble import IsolationForest
03  X=np.array([0.,0.1,-0.2,0.2,-0.1,10.]).reshape(-1,1)
```

```
model=IsolationForest(contamination=1/6,random_state=42).fit(X)
print('Anomaly labels (-1 means outlier):',model.predict(X).tolist())
print('Anomaly values:',X[model.predict(X)==-1].ravel().tolist())
```

Verified console output

```
Anomaly labels (-1 means outlier): [1, 1, 1, 1, 1, -1]
Anomaly values: [10.0]
```

5. Visual reasoning

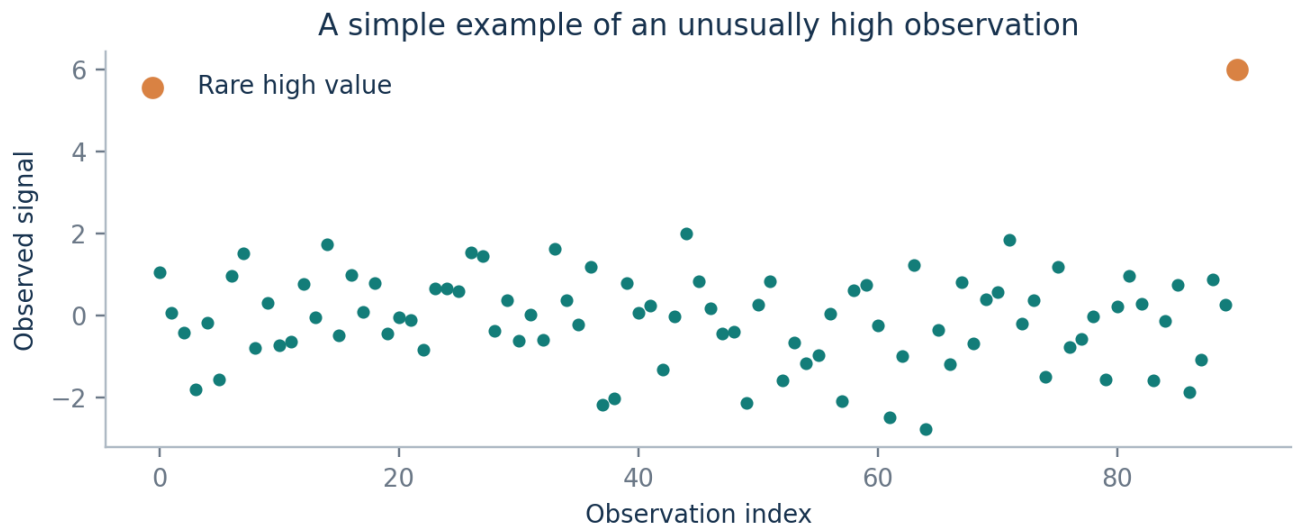


Figure 22. Anomaly detection identifies rare or unusual observations relative to a reference distribution.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. An anomalous value is not necessarily a fault, and a real fault may still appear statistically typical.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when anomaly detection would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Anomaly detection identifies rare or unusual observations relative to a reference distribution.

Formula. A z-score measures deviation from a reference mean in units of reference standard deviation.

Practical example. Detect credit-card fraud; identify unusual vibration patterns in rotating motors.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for anomaly detection.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 23 / 38 · CORE NOTES

23. Association Rules and Recommender Systems

Learning objective. Explain the purpose of association rules and recommender systems, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Association rules find frequently co-occurring items, while recommenders rank options for individual users. Support measures how often an itemset occurs; confidence measures conditional co-occurrence; lift compares confidence with the consequent's baseline frequency. Collaborative filtering uses interactions among users and items; content-based filtering uses item or user features. Recommenders need evaluation on future or hidden interactions and attention to popularity bias and cold starts.

When and where this matters

Recommend products; find component failure modes frequently observed together.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Frequent itemsets.
- Apriori.
- FP-Growth.
- Support, confidence, and lift.
- Collaborative filtering.
- Content-based filtering.
- Matrix factorization.
- Implicit feedback.
- Ranking metrics.
- Cold-start problems.

3. Mathematical model and notation

$$lift(A \to B) = \frac{P(B \mid A)}{P(B)}$$

Lift above 1 indicates positive co-occurrence relative to independent occurrence, not causal influence.

Worked numerical example — follow every step

If bread appears in 60% of baskets and 75% of milk baskets include bread, lift(milk->bread)=0.75/0.60=1.25.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
# Manually compute association-rule support, confidence, lift.
transactions=[{'milk','bread'},{'milk','eggs'},{'bread','milk'},{'bread'}]
a=sum('milk' in t for t in transactions)
b=sum('bread' in t for t in transactions)
ab=sum({'milk','bread'}.issubset(t) for t in transactions)
support=ab/len(transactions)
confidence=ab/a
lift=confidence/(b/len(transactions))
print('Support:',support)
print('Confidence milk->bread:',round(confidence,3))
print('Lift:',round(lift,3))
```

Verified console output

```
Support: 0.5
Confidence milk->bread: 0.667
Lift: 0.889
```

5. Visual reasoning

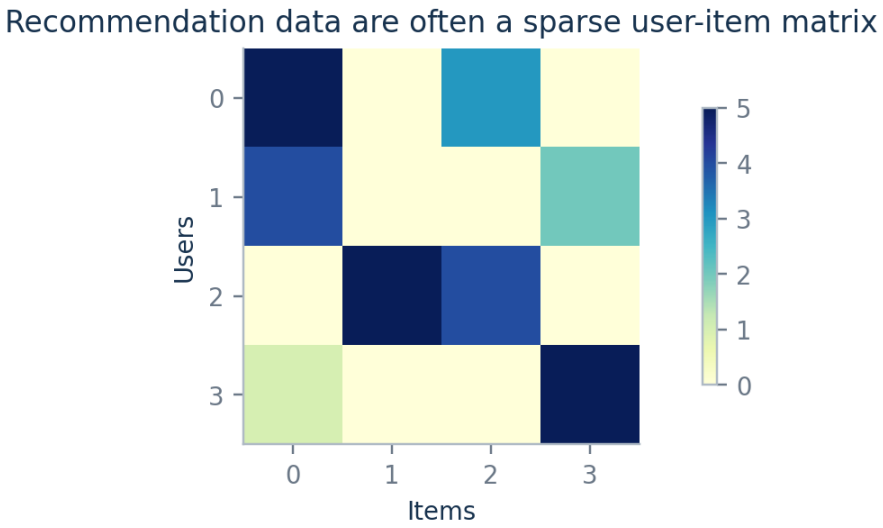


Figure 23. Association rules find frequently co-occurring items, while recommenders rank options for individual users.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. A high confidence rule may only reflect a universally common item; consider lift and statistical support together.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when association rules and recommender systems would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Association rules find frequently co-occurring items, while recommenders rank options for individual users.

Formula. Lift above 1 indicates positive co-occurrence relative to independent occurrence, not causal influence.

Practical example. Recommend products; find component failure modes frequently observed together.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for association rules and recommender systems.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 24 / 38 · CORE NOTES

24. Evaluating Machine Learning Models

Learning objective. Explain the purpose of evaluating machine learning models, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Model evaluation estimates behavior on unseen examples using task-appropriate metrics. Accuracy can hide poor performance on rare positive cases; precision describes correctness among positive predictions, while recall describes coverage of true positives. Regression metrics quantify errors in target units or squared units. Calibration asks whether predicted probabilities correspond to observed frequencies. Always isolate the final test set from model tuning.

When and where this matters

Evaluate screening models with sensitivity and specificity; assess sensor prediction with MAE in physical units.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Training vs. validation vs. test sets.
- Confusion matrix.
- Accuracy.
- Precision.
- Recall.
- Specificity.
- F1-score.
- Balanced accuracy.
- ROC curves and AUC.
- Precision-recall curves.
- Log loss.
- Calibration.
- MAE, MSE, RMSE, MAPE, and R^2.
- Ranking metrics.
- Clustering evaluation metrics.
- Statistical uncertainty in performance.
- Fair comparisons between models.

3. Mathematical model and notation

$$F_1 = \frac{2PR}{P+R}$$

The harmonic mean of precision P and recall R, defined as zero in common conventions when both are zero.

Worked numerical example — follow every step

With TP=8, FP=2, FN=4, precision=8/10=0.8 and recall=8/12=0.6667. F1=2(0.8)(0.6667)/(0.8+0.6667)=0.7273.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
01  from sklearn.metrics import confusion_matrix, precision_score, recall_score,
f1_score
02  y=[1,0,1,1,0,0,1,0]
03  pred=[1,0,0,1,1,0,1,0]
04  print('Confusion matrix:',confusion_matrix(y,pred).tolist())
05  print('Precision:',round(precision_score(y,pred),3))
06  print('Recall:',round(recall_score(y,pred),3))
07  print('F1:',round(f1_score(y,pred),3))
```

Verified console output

```
Confusion matrix: [[3, 1], [1, 3]]
Precision: 0.75
Recall: 0.75
F1: 0.75
```

5. Visual reasoning

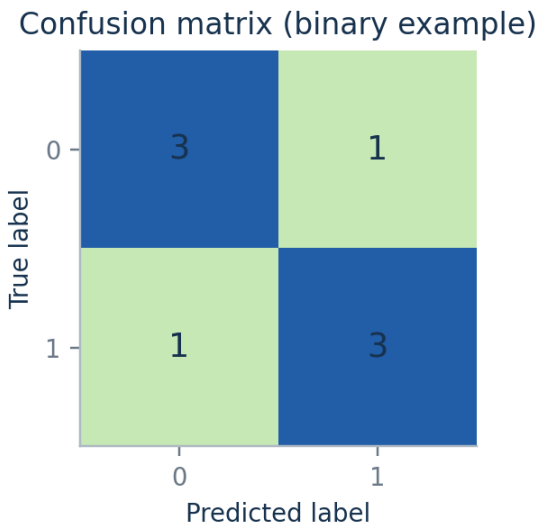


Figure 24. Model evaluation estimates behavior on unseen examples using task-appropriate metrics.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. ROC-AUC can look reassuring under severe class imbalance even when precision at useful thresholds is low.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when evaluating machine learning models would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.

- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Model evaluation estimates behavior on unseen examples using task-appropriate metrics.

Formula. The harmonic mean of precision P and recall R, defined as zero in common conventions when both are zero.

Practical example. Evaluate screening models with sensitivity and specificity; assess sensor prediction with MAE in physical units.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for evaluating machine learning models.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 25 / 38 · CORE NOTES

25. Model Selection and Hyperparameter Tuning

Learning objective. Explain the purpose of model selection and hyperparameter tuning, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Hyperparameters control model structure or training behavior, while fitted parameters are learned from data. Validation selects hyperparameters and compares candidate pipelines. Cross-validation rotates holdout folds for more reliable estimates; grouping, time order, and nested tuning are important when examples are correlated. Random search often covers broad spaces efficiently; Bayesian optimization uses a surrogate of performance to guide new trials.

When and where this matters

Tune defect classifiers across production batches; choose sensor model complexity with group-aware folds.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Holdout validation.
- k-Fold cross-validation.
- Stratified cross-validation.
- Group-aware cross-validation.
- Time-series cross-validation.
- Nested cross-validation.
- Grid Search.
- Random Search.
- Bayesian Optimization.
- Early stopping.
- Regularization.
- Learning curves.
- Validation curves.
- Bias-variance decomposition.
- Overfitting prevention.

3. Mathematical model and notation

hat R_{CV}=frac1Ksum_{k=1}^{K}R_k

K-fold CV aggregates loss or performance across K validation folds; use suitable weighted averaging if fold sizes differ.

Worked numerical example — follow every step

If three validation accuracies are 0.80, 0.85, and 0.90, their mean is (0.80+0.85+0.90)/3=0.85. Variation across folds signals uncertainty.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
from sklearn.datasets import load_iris
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV
X,y=load_iris(return_X_y=True)
pipe=make_pipeline(StandardScaler(),SVC())
search=GridSearchCV(pipe,{'svc__C':[0.1,1,10],'svc__kernel':['linear','rbf']},cv=5)
search.fit(X,y)
print('Best settings:',search.best_params_)
print('Mean CV accuracy:',round(search.best_score_,3))
```

Verified console output

```
Best settings: {'svc__C': 10, 'svc__kernel': 'rbf'}
Mean CV accuracy: 0.973
```

5. Visual reasoning

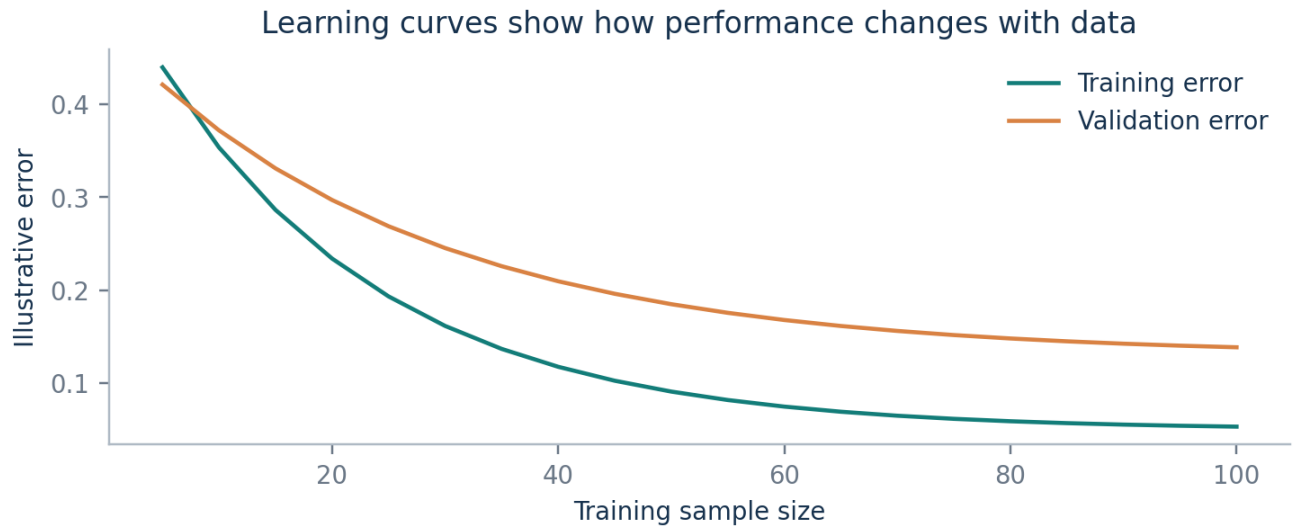


Figure 25. Hyperparameters control model structure or training behavior, while fitted parameters are learned from data.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. Time-series rows should not be randomly shuffled into future-to-past training unless the use case truly permits that information.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when model selection and hyperparameter tuning would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.

- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Hyperparameters control model structure or training behavior, while fitted parameters are learned from data.

Formula. K-fold CV aggregates loss or performance across K validation folds; use suitable weighted averaging if fold sizes differ.

Practical example. Tune defect classifiers across production batches; choose sensor model complexity with group-aware folds.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for model selection and hyperparameter tuning.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 26 / 38 · CORE NOTES

26. Explainable and Responsible Machine Learning

Learning objective. Explain the purpose of explainable and responsible machine learning, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Interpretability asks how an algorithm works; explainability analyzes why particular outputs occur. Linear coefficients, local explanations, partial dependence, and permutation importance answer different questions and have different limitations. An explanation of a trained model is not automatically a causal account of the world. Responsible modeling evaluates fairness, subgroup performance, privacy, robustness, security, and human oversight.

When and where this matters

Audit clinical decision support for subgroup errors; explain maintenance alarms to technicians.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Interpretability vs. explainability.
- Model coefficients.
- Feature importance.
- Permutation importance.
- Partial Dependence Plots.
- Individual Conditional Expectation.
- SHAP.
- LIME.
- Counterfactual explanations.
- Fairness metrics.
- Dataset bias.
- Privacy.
- Ethical model design.
- Robustness.
- Security.
- Limitations of explanations.

3. Mathematical model and notation

$$I_j = R(f, X_{perm(j)}) - R(f, X)$$

Permutation importance measures the increase in held-out loss after shuffling one feature; correlated features can complicate interpretation.

Worked numerical example — follow every step

If baseline validation error is 0.12 and shuffling a feature raises error to 0.19, that feature's permutation importance under this experiment is 0.07 error units.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
from sklearn.model_selection import train_test_split
X,y=load_breast_cancer(return_X_y=True)
a,b,c,d=train_test_split(X,y,random_state=7,stratify=y)
clf=RandomForestClassifier(n_estimators=40,random_state=7,n_jobs=1).fit(a,c)
result=permutation_importance(clf,b,d,n_repeats=3,random_state=7,n_jobs=1)
ix=result.importances_mean.argmax()
print('Test accuracy:',round(clf.score(b,d),3))
print('Most influential feature index:',int(ix))
print('Permutation accuracy drop:',round(result.importances_mean[ix],3))
```

Verified console output

```
Test accuracy: 0.972
Most influential feature index: 20
Permutation accuracy drop: 0.028
```

5. Visual reasoning

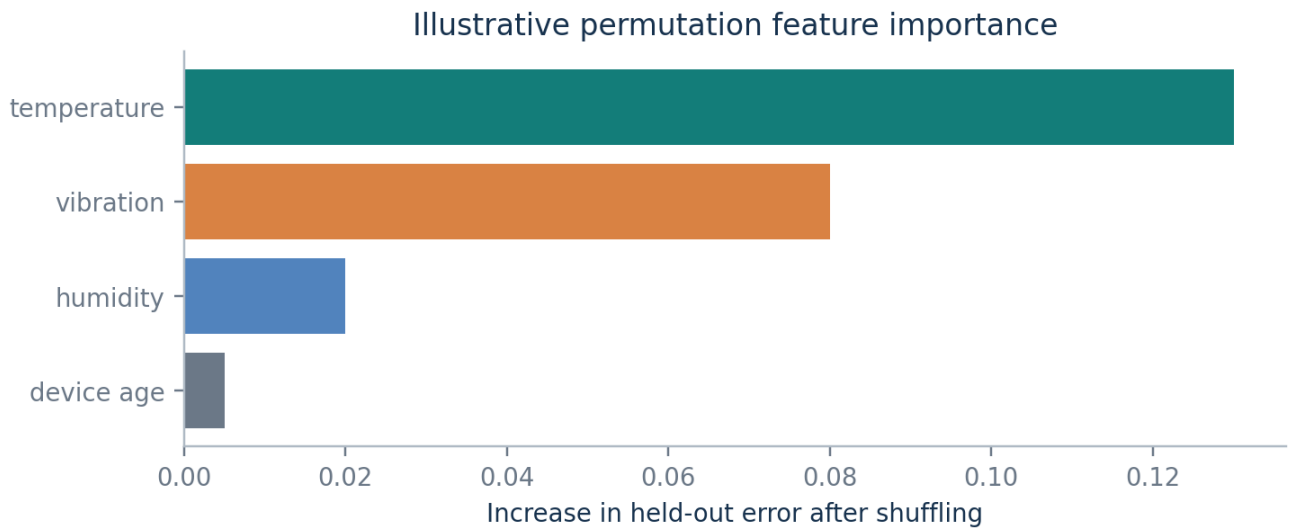


Figure 26. Interpretability asks how an algorithm works; explainability analyzes why particular outputs occur.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. SHAP and feature importance express contributions under assumptions; neither alone proves a variable causes the predicted event.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when explainable and responsible machine learning would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Interpretability asks how an algorithm works; explainability analyzes why particular outputs occur.

Formula. Permutation importance measures the increase in held-out loss after shuffling one feature; correlated features can complicate interpretation.

Practical example. Audit clinical decision support for subgroup errors; explain maintenance alarms to technicians.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for explainable and responsible machine learning.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 27 / 38 · CORE NOTES

27. Neural Network Foundations

Learning objective. Explain the purpose of neural network foundations, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Neural networks compose linear transformations and nonlinear activation functions. A single perceptron makes a linear decision; multilayer networks learn nonlinear mappings. Forward propagation computes predictions, a loss scores them, and backpropagation computes derivatives efficiently using the chain rule. Initialization, normalization, activation choice, regularization, and optimizer settings strongly influence trainability.

When and where this matters

Learn nonlinear signal features; predict outcomes from image or tabular data.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Biological inspiration and its limitations.
- Artificial neurons.
- Perceptrons.
- Multilayer Perceptrons.
- Weights and biases.
- Activation functions.
- Sigmoid, tanh, ReLU, Leaky ReLU, and GELU.
- Forward propagation.
- Loss functions.
- Backpropagation.
- Chain rule derivation.
- Computational graphs.
- Weight initialization.
- Vanishing and exploding gradients.
- Dropout.
- Batch normalization.
- Layer normalization.
- Training neural networks with PyTorch.

3. Mathematical model and notation

$$a^{(l)} = \phi(W^{(l)}a^{(l-1)} + b^{(l)})$$

Each layer applies a weight matrix, bias vector and activation function phi to the prior layer's activations.

Worked numerical example — follow every step

A neuron with weights [2,-1], input [3,4], and bias 1 has preactivation 2(3)-1(4)+1=3. ReLU outputs max(0,3)=3.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
import torch
torch.manual_seed(7)
net=torch.nn.Sequential(torch.nn.Linear(2,4),torch.nn.ReLU(),torch.nn.Linear(4,1))
x=torch.tensor([[1.,2.],[2.,1.]])
y=torch.tensor([[3.],[3.]])
optimizer=torch.optim.SGD(net.parameters(),lr=0.01)
for _ in range(30):
    optimizer.zero_grad()
    loss=((net(x)-y)**2).mean()
    loss.backward()
    optimizer.step()
print('Final MSE:',round(float(((net(x)-y)**2).mean().detach()),4))
print('Number of trainable parameters:',sum(p.numel() for p in net.parameters()))
```

Verified console output

```
Final MSE: 0.0368
Number of trainable parameters: 17
```

5. Visual reasoning

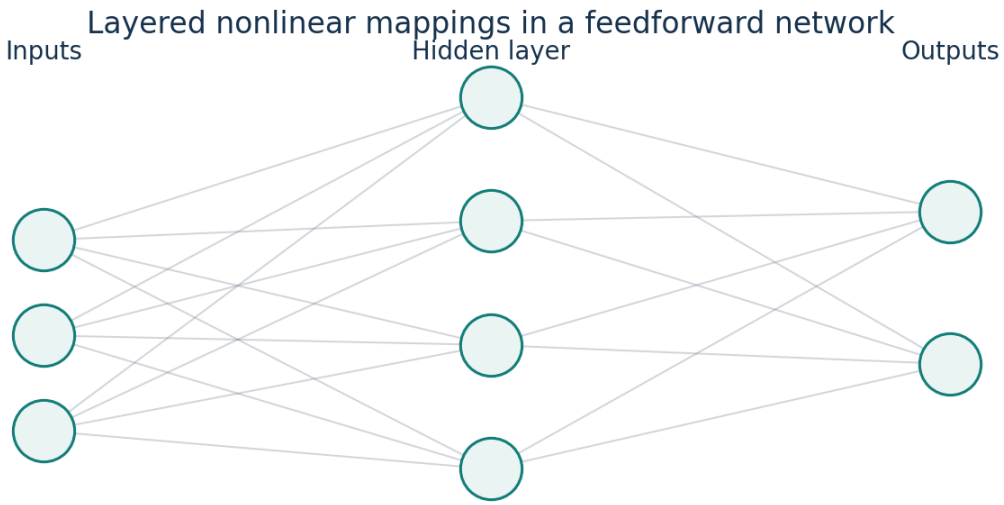


Figure 27. Neural networks compose linear transformations and nonlinear activation functions.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. Calling backward() without clearing accumulated gradients each step can lead to unintended updates in PyTorch.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when neural network foundations would be appropriate and when it would be a poor choice.

- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Neural networks compose linear transformations and nonlinear activation functions.

Formula. Each layer applies a weight matrix, bias vector and activation function phi to the prior layer's activations.

Practical example. Learn nonlinear signal features; predict outcomes from image or tabular data.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for neural network foundations.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 28 / 38 · CORE NOTES

28. Deep Learning Architectures

Learning objective. Explain the purpose of deep learning architectures, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Different network architectures encode different structures. CNNs exploit local patterns and shared filters in images; recurrent networks maintain sequence state; Transformers use attention to combine information across tokens. Residual connections support deep models. Representation transfer lets a pretrained model be fine-tuned for a smaller task, but dataset shift and computational cost still matter. Tensor shapes are essential when tracing models.

When and where this matters

CNNs inspect PCB imagery; Transformers model sequences of maintenance logs.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Convolutional Neural Networks.
- Convolution and pooling.
- Image classification.
- Recurrent Neural Networks.
- LSTM.
- GRU.
- Sequence modeling.
- Attention mechanisms.
- Transformers.
- Positional encoding.
- Encoder and decoder architectures.
- Embeddings.
- Residual connections.
- Transfer learning.
- Fine-tuning.
- Foundation models.
- Large Language Models.
- Model quantization.
- Efficient inference.

3. Mathematical model and notation

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Scaled dot-product attention compares queries and keys, normalizes weights, and combines values.

Worked numerical example — follow every step

With Q=[1,0], K1=[1,0], K2=[0,1], dot-product scores are 1 and 0. Softmax weights are approximately 0.731 and 0.269 before any other scale factor.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
import torch
torch.manual_seed(7)
cnn=torch.nn.Sequential(torch.nn.Conv2d(1,4,kernel_size=3,padding=1),
  torch.nn.ReLU(),torch.nn.MaxPool2d(2))
x=torch.randn(2,1,8,8)
y=cnn(x)
print('Input tensor shape:',tuple(x.shape))
print('CNN feature map shape:',tuple(y.shape))
print('Trainable parameters:',sum(p.numel() for p in cnn.parameters()))
```

Verified console output

```
Input tensor shape: (2, 1, 8, 8)
CNN feature map shape: (2, 4, 4, 4)
Trainable parameters: 40
```

5. Visual reasoning

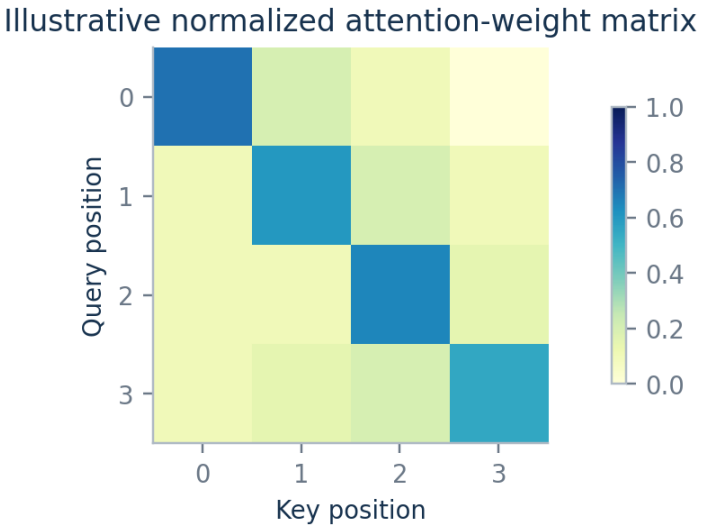


Figure 28. Different network architectures encode different structures.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. Self-attention does not inherently understand token order; order-related information must enter through position representations or architecture.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when deep learning architectures would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Different network architectures encode different structures.

Formula. Scaled dot-product attention compares queries and keys, normalizes weights, and combines values.

Practical example. CNNs inspect PCB imagery; Transformers model sequences of maintenance logs.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for deep learning architectures.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 29 / 38 · CORE NOTES

29. Generative Machine Learning

Learning objective. Explain the purpose of generative machine learning, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Generative models learn patterns in data so they can produce new samples or estimate data likelihoods. VAEs use latent variables and an evidence lower bound; GANs train a generator against a discriminator; diffusion models gradually denoise noisy samples. Their evaluation requires both plausibility and diversity, plus checking memorization and unsafe artifacts. Generating attractive samples is not equivalent to learning a faithful or causal data model.

When and where this matters

Generate product concepts; augment low-resource visual datasets with evaluated synthetic images.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Generative vs. discriminative learning.
- Autoregressive models.
- Variational Autoencoders.
- Generative Adversarial Networks.
- Diffusion models.
- Latent variable models.
- Sampling.
- Likelihood-based and implicit models.
- Generative model evaluation.
- Practical limitations and risks.

3. Mathematical model and notation

$$p_\theta(x) = \int p_\theta(x \mid z) p(z)\, dz$$

Latent-variable generators integrate over hidden variables z drawn from a prior distribution.

Worked numerical example — follow every step

If latent z is standard normal and a toy generator outputs x=2z+1, the generated output has mean 1 and variance 4. This follows from E[2z+1]=1 and Var(2z+1)=4.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
01  import numpy as np
02  rng=np.random.default_rng(7)
03  latent=rng.normal(size=(5,2))
```

```
04  W=np.array([[1.5,0.2],[-0.1,2.0]])
05  samples=latent@W.T
06  print('Generated sample shape:',samples.shape)
07  print('Mean of five synthetic samples:',samples.mean(axis=0).round(3).tolist())
08  print('Note: this is a toy latent-variable generator, not a trained GAN or VAE.')
```

Verified console output

```
Generated sample shape: (5, 2)
Mean of five synthetic samples: [-0.382, -0.322]
Note: this is a toy latent-variable generator, not a trained GAN or VAE.
```

5. Visual reasoning

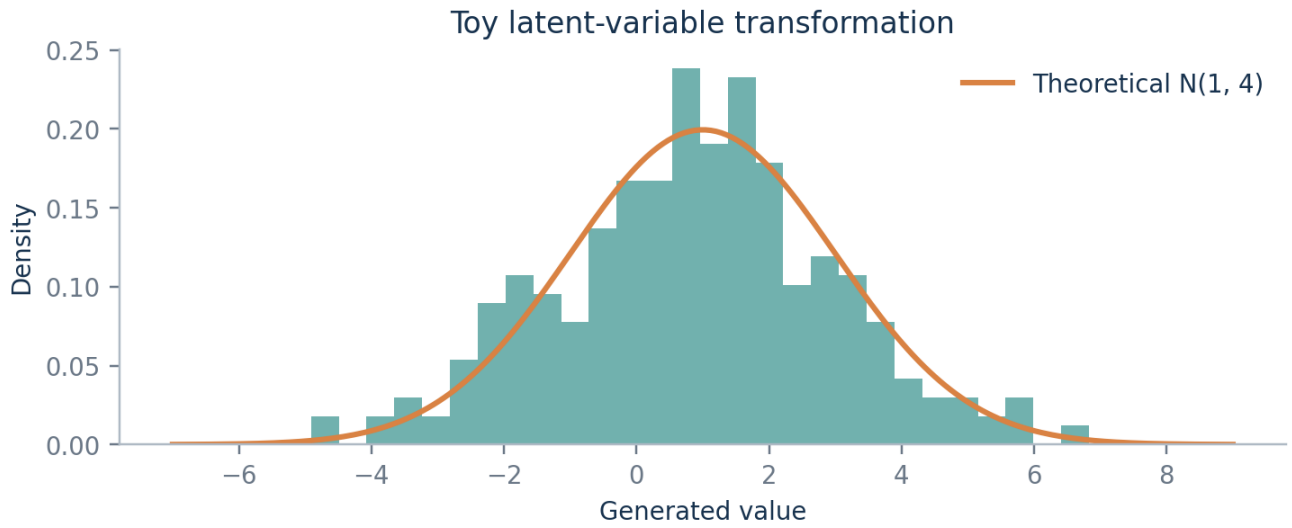


Figure 29. Generative models learn patterns in data so they can produce new samples or estimate data likelihoods.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. Generated data may reveal memorized training samples; synthetic does not automatically mean private.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when generative machine learning would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Generative models learn patterns in data so they can produce new samples or estimate data likelihoods.

Formula. Latent-variable generators integrate over hidden variables z drawn from a prior distribution.

Practical example. Generate product concepts; augment low-resource visual datasets with evaluated synthetic images.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for generative machine learning.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 30 / 38 · CORE NOTES

30. Time-Series Machine Learning

Learning objective. Explain the purpose of time-series machine learning, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Time-series forecasting predicts values using information available before the forecast time. Autocorrelation describes dependence across lags; trend, seasonality, and regime changes complicate evaluation. Classical autoregressive models provide interpretable baselines; machine learning can incorporate lagged values, exogenous predictors, and nonlinear effects. A forecast must specify horizon, cadence, and what information is known at prediction time.

When and where this matters

Forecast power demand; model vibration trends before equipment failure.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Time-series characteristics.
- Trend and seasonality.
- Autocorrelation.
- Stationarity.
- Lag features.
- Rolling statistics.
- AR, MA, ARMA, and ARIMA.
- SARIMA.
- Exponential smoothing.
- Forecasting with supervised models.
- Time-series cross-validation.
- Forecast uncertainty.
- Forecasting metrics.
- Concept drift.

3. Mathematical model and notation

$$\hat{y}_t = c + \sum_{i=1}^{p} \phi_i y_{t-i}$$

An autoregressive model of order p predicts current value from p prior observations.

Worked numerical example — follow every step

If y_hat(t)=2+0.5 y(t-1) and the latest observation is 10, the next prediction is 2+0.5(10)=7.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
01  import numpy as np
02  from sklearn.linear_model import LinearRegression
03  series=np.array([10,12,14,15,18,20,21,24],dtype=float)
04  X=np.column_stack([series[:-2],series[1:-1]])
05  y=series[2:]
06  model=LinearRegression().fit(X,y)
07  print('Lag-window shape:',X.shape)
08  print('Next forecast:',round(float(model.predict([[21,24]])[0]),3))
09  print('Caution: chronological testing is essential for genuine evaluation.')
```

Verified console output

```
Lag-window shape: (6, 2)
Next forecast: 25.31
Caution: chronological testing is essential for genuine evaluation.
```

5. Visual reasoning

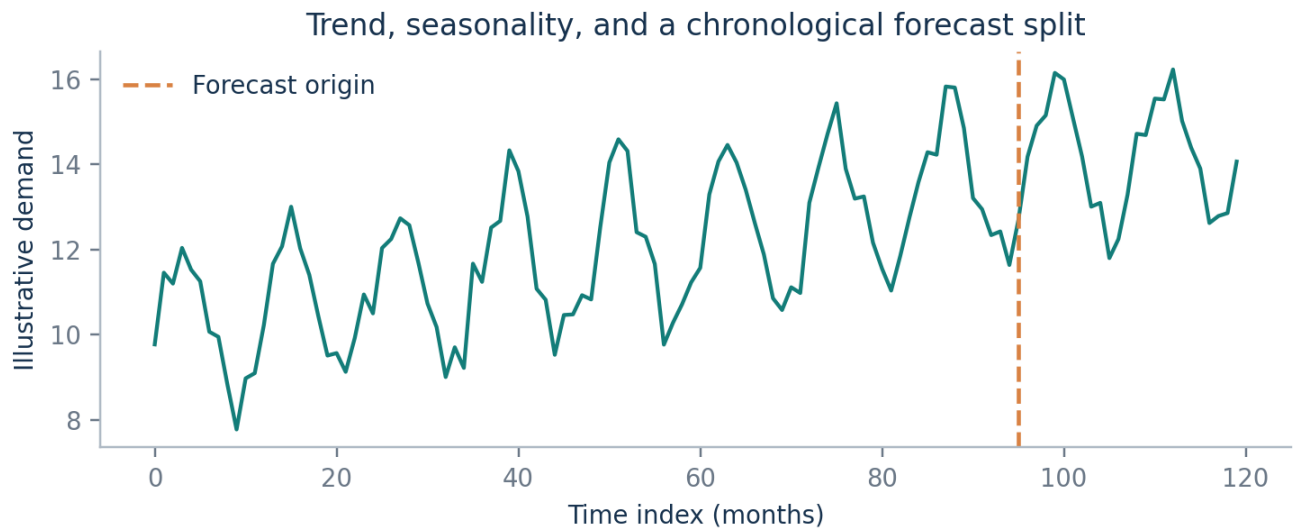


Figure 30. Time-series forecasting predicts values using information available before the forecast time.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. Do not compute rolling features using future values or randomly shuffle chronological test periods.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when time-series machine learning would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Time-series forecasting predicts values using information available before the forecast time.

Formula. An autoregressive model of order p predicts current value from p prior observations.

Practical example. Forecast power demand; model vibration trends before equipment failure.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for time-series machine learning.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 31 / 38 · CORE NOTES

31. Natural Language Processing

Learning objective. Explain the purpose of natural language processing, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Natural language processing converts human language into computable representations and predictions. Classical pipelines tokenize text and construct word counts or TF-IDF; contextual encoders and Transformers model meaning based on neighboring text. Language systems support classification, search, extraction, translation, summarization, and generation. Tokenization, evaluation sets, and domain-specific vocabulary all influence performance.

When and where this matters

Route help-desk messages; retrieve passages in technical manuals.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Text preprocessing.
- Tokenization.
- Bag-of-Words.
- TF-IDF.
- Word embeddings.
- Word2Vec and GloVe.
- Language modeling.
- Attention.
- Transformers.
- BERT-style encoders.
- GPT-style decoders.
- Text classification.
- Sentiment analysis.
- Text generation.
- Semantic search.
- Retrieval-Augmented Generation.
- Evaluation of language systems.

3. Mathematical model and notation

$$TFIDF(t, d) = TF(t, d) \cdot IDF(t)$$

TF-IDF weights a term by its occurrence within a document and its relative rarity across documents; exact formulas vary by implementation.

Worked numerical example — follow every step

If 'sensor' appears three times in a 100-token document, normalized term frequency is 3/100=0.03. If its IDF is 2, the simplified TF-IDF score is 0.06.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
01  from sklearn.pipeline import make_pipeline
02  from sklearn.feature_extraction.text import TfidfVectorizer
03  from sklearn.linear_model import LogisticRegression
04  texts=['excellent experience','very good','terrible service','very bad','good
quality','poor quality']
05  y=[1,1,0,0,1,0]
06  model=make_pipeline(TfidfVectorizer(),LogisticRegression()).fit(texts,y)
07  print('Positive probability:',round(model.predict_proba(['good service'])[0,1],3))
08  print('Predictions:',model.predict(['excellent','terrible']).tolist())
```

Verified console output

```
Positive probability: 0.522
Predictions: [1, 0]
```

5. Visual reasoning

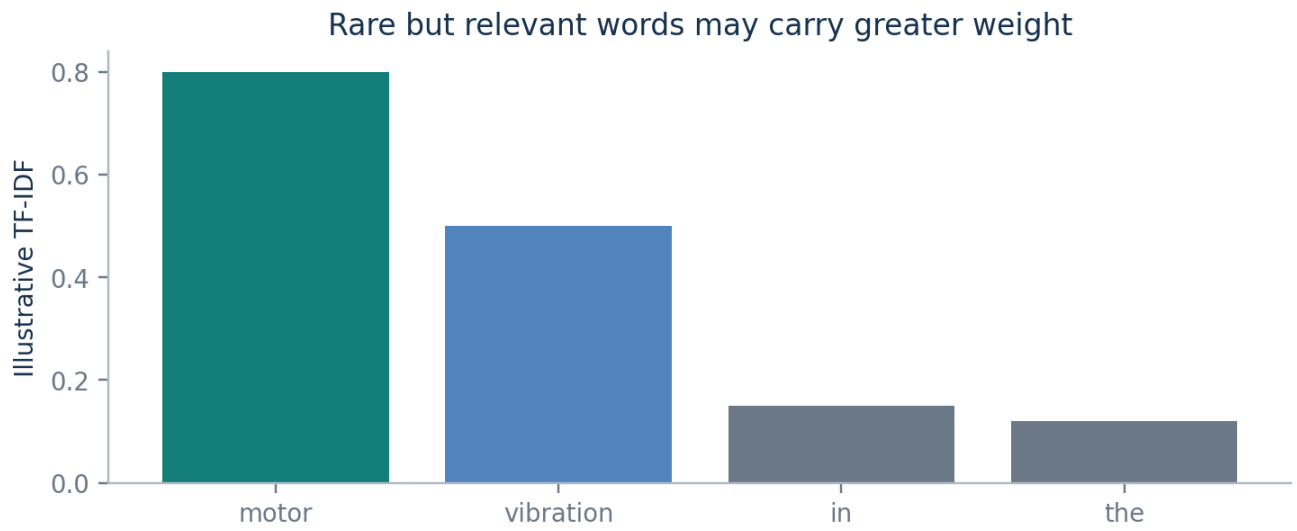


Figure 31. Natural language processing converts human language into computable representations and predictions.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. A model that predicts an associated label does not necessarily understand the statement's intent or truth.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when natural language processing would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Natural language processing converts human language into computable representations and predictions.

Formula. TF-IDF weights a term by its occurrence within a document and its relative rarity across documents; exact formulas vary by implementation.

Practical example. Route help-desk messages; retrieve passages in technical manuals.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for natural language processing.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 32 / 38 · CORE NOTES

32. Computer Vision

Learning objective. Explain the purpose of computer vision, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Computer vision interprets images and videos. Pixel preprocessing controls intensity and color scales; convolutional filters reveal local features; modern deep networks perform image classification, object detection, segmentation, and tracking. Vision models are sensitive to viewpoint, illumination, resolution, camera calibration, and dataset shift. Evaluation must align with the task: classification accuracy, detection precision-recall, or segmentation overlap.

When and where this matters

Inspect circuit boards for defects; identify obstacles for mobile robots.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Digital image representation.
- Image preprocessing.
- Filters and convolution.
- Feature extraction.
- CNN image classification.
- Object detection.
- Image segmentation.
- Transfer learning.
- Vision Transformers.
- Data augmentation.
- Model evaluation.
- Real-world deployment considerations.

3. Mathematical model and notation

$$IoU = \frac{|A \cap B|}{|A \cup B|}$$

Intersection over union compares predicted and true object regions or bounding boxes.

Worked numerical example — follow every step

If a predicted region covers 8 pixels, the reference covers 10, and they overlap in 6, IoU=6/(8+10-6)=6/12=0.5.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
01  from sklearn.datasets import load_digits
02  from sklearn.model_selection import train_test_split
```

```
from sklearn.neighbors import KNeighborsClassifier
X,y=load_digits(return_X_y=True)
a,b,c,d=train_test_split(X,y,test_size=0.2,random_state=7,stratify=y)
model=KNeighborsClassifier(n_neighbors=3).fit(a,c)
print('Image size in pixels:',X.shape[1])
print('Test accuracy:',round(model.score(b,d),3))
print('Classes:',len(set(y)))
```

Verified console output

```
Image size in pixels: 64
Test accuracy: 0.989
Classes: 10
```

5. Visual reasoning

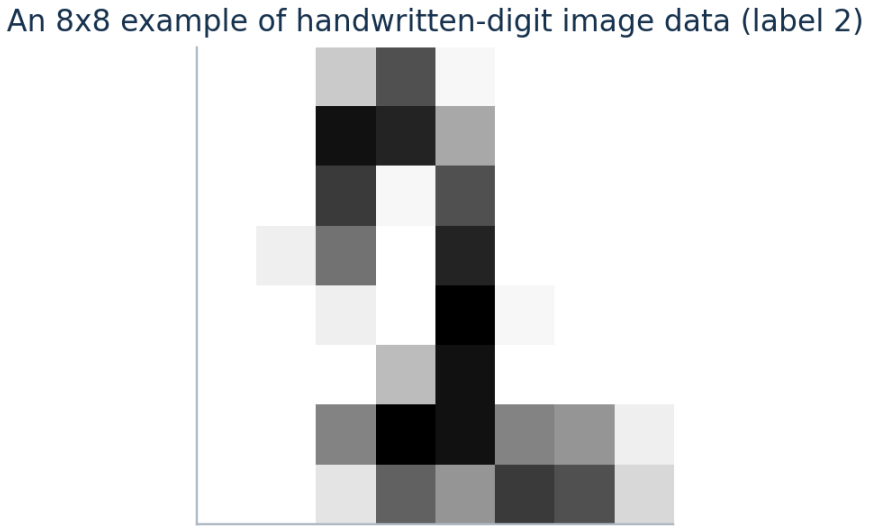


Figure 32. Computer vision interprets images and videos.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. High test accuracy on one camera or lighting condition does not guarantee performance on another.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when computer vision would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Computer vision interprets images and videos.

Formula. Intersection over union compares predicted and true object regions or bounding boxes.

Practical example. Inspect circuit boards for defects; identify obstacles for mobile robots.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for computer vision.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 33 / 38 · CORE NOTES

33. Reinforcement Learning

Learning objective. Explain the purpose of reinforcement learning, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Reinforcement learning studies sequential decisions where actions influence later observations and rewards. An agent chooses actions according to a policy, interacts with an environment, and attempts to maximize expected discounted reward. Value methods learn how good states or actions are; policy methods directly optimize action probabilities. Exploration is essential but can be costly in safety-critical physical systems.

When and where this matters

Learn robot manipulation policies; optimize adaptive resource allocation.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Agents and environments.
- States, actions, and rewards.
- Markov Decision Processes.
- Policies.
- Value functions.
- Bellman equations.
- Dynamic programming.
- Monte Carlo learning.
- Temporal Difference learning.
- Q-Learning.
- SARSA.
- Deep Q-Networks.
- Policy gradients.
- Actor-Critic methods.
- Exploration vs. exploitation.
- Practical training examples.

3. Mathematical model and notation

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Tabular Q-learning updates an action value using immediate reward and estimated best discounted future value.

Worked numerical example — follow every step

If Q_old=2, reward=1, gamma=0.9, next max Q=4, alpha=0.1, the target is 1+0.9(4)=4.6 and Q_new=2+0.1(4.6-2)=2.26.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
import numpy as np
# Deterministic tabular Q-learning in a 5-state corridor.
q=np.zeros((5,2))  # 0=left, 1=right
rng=np.random.default_rng(7)
for episode in range(500):
    state=0
    for _ in range(20):
        action=int(rng.integers(2)) if rng.random()<0.1 else int(np.argmax(q[state])
)
        nxt=max(0,state-1) if action==0 else min(4,state+1)
        reward=1.0 if nxt==4 else -0.01
        q[state,action]+=0.2*(reward+0.9*np.max(q[nxt])-q[state,action])
        state=nxt
        if state==4: break
print('Action values in starting state:',q[0].round(3).tolist())
print('Greedy action at start (1=right):',int(np.argmax(q[0])))
```

Verified console output

```
Action values in starting state: [0.621, 0.702]
Greedy action at start (1=right): 1
```

5. Visual reasoning



Figure 33. Reinforcement learning studies sequential decisions where actions influence later observations and rewards.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. A high reward in simulation can exploit simulator weaknesses rather than transfer to real-world safety.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when reinforcement learning would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Reinforcement learning studies sequential decisions where actions influence later observations and rewards.

Formula. Tabular Q-learning updates an action value using immediate reward and estimated best discounted future value.

Practical example. Learn robot manipulation policies; optimize adaptive resource allocation.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for reinforcement learning.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 34 / 38 · CORE NOTES

34. Graph Machine Learning

Learning objective. Explain the purpose of graph machine learning, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Graph ML represents entities as nodes connected by relationships. An adjacency matrix encodes edges; graph convolution and message passing combine information from neighboring nodes. Graph-level, node-level, and edge-level predictions solve different problems. Structure provides useful inductive bias, but graph construction, homophily assumptions, and leakage across linked entities must be evaluated carefully.

When and where this matters

Detect fraud rings; analyze dependencies in electrical power networks.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Graph representation.
- Nodes and edges.
- Adjacency matrices.
- Graph Laplacians.
- Node embeddings.
- Graph Convolutional Networks.
- Graph Attention Networks.
- Link prediction.
- Node classification.
- Graph classification.

3. Mathematical model and notation

$$h_v^{(l+1)} = \phi\left(W \sum_{u \in N(v)} h_u^{(l)}\right)$$

A simplified message-passing update combines neighbor representations N(v) before applying a learned transformation.

Worked numerical example — follow every step

In an undirected triangle, each node connects to the other two and has degree 2. Its adjacency row contains two 1s, so the row sum equals its degree.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
01  import networkx as nx
02  G=nx.Graph()
03  G.add_edges_from([('a','b'),('b','c'),('c','d'),('b','d')])
```

```
04  print('Number of nodes:',G.number_of_nodes())
05  print('Node degree dictionary:',dict(G.degree()))
06  print('Most connected node:',max(G.degree,key=lambda item:item[1])[0])
```

Verified console output

```
Number of nodes: 4
Node degree dictionary: {'a': 1, 'b': 3, 'c': 2, 'd': 2}
Most connected node: b
```

5. Visual reasoning

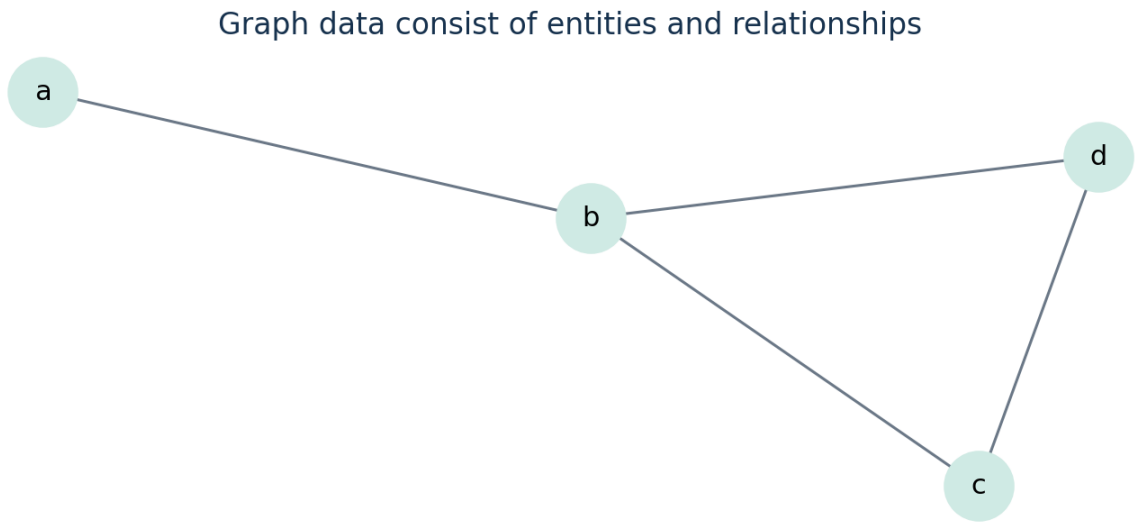


Figure 34. Graph ML represents entities as nodes connected by relationships.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. Random node splits can leak information through overlapping neighborhoods; test the intended graph generalization regime.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when graph machine learning would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Graph ML represents entities as nodes connected by relationships.

Formula. A simplified message-passing update combines neighbor representations N(v) before applying a learned transformation.

Practical example. Detect fraud rings; analyze dependencies in electrical power networks.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for graph machine learning.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 35 / 38 · CORE NOTES

35. Probabilistic Machine Learning

Learning objective. Explain the purpose of probabilistic machine learning, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Probabilistic ML learns or reasons with distributions rather than single-point predictions. Bayesian methods combine prior assumptions with observed likelihoods; Gaussian processes place priors on functions; hidden Markov models describe latent time-varying states. Approximate inference such as MCMC or variational inference becomes necessary when exact posteriors are intractable. Calibration and sensitivity to priors are as important as nominal predictive accuracy.

When and where this matters

Quantify uncertainty in sparse experimental data; track hidden operating modes of machines.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Bayesian inference.
- Bayesian linear regression.
- Gaussian Processes.
- Bayesian networks.
- Hidden Markov Models.
- Latent variable models.
- Expectation-Maximization.
- Variational inference.
- Monte Carlo methods.
- Uncertainty estimation.

3. Mathematical model and notation

$$p(\theta \mid D) \propto p(D \mid \theta)p(\theta)$$

Posterior beliefs about parameters theta are proportional to likelihood of data D times prior beliefs.

Worked numerical example — follow every step

A Beta(2,2) prior and 8 successes with 2 failures give a Beta(10,4) posterior. Its mean is 10/(10+4)=0.7143.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
01  # Bayesian coin learning with Beta prior and Bernoulli outcomes.
02  from scipy.stats import beta
03  alpha_prior,beta_prior=2,2
```

```
successes,failures=8,2
a=alpha_prior+successes
b=beta_prior+failures
print('Posterior Beta parameters:',(a,b))
print('Posterior mean:',round(a/(a+b),3))
print('95% credible interval:',tuple(round(v,3) for v in beta.ppf([0.025,0.975],a,b)
))
```

Verified console output

```
Posterior Beta parameters: (10, 4)
Posterior mean: 0.714
95% credible interval: (np.float64(0.462), np.float64(0.909))
```

5. Visual reasoning

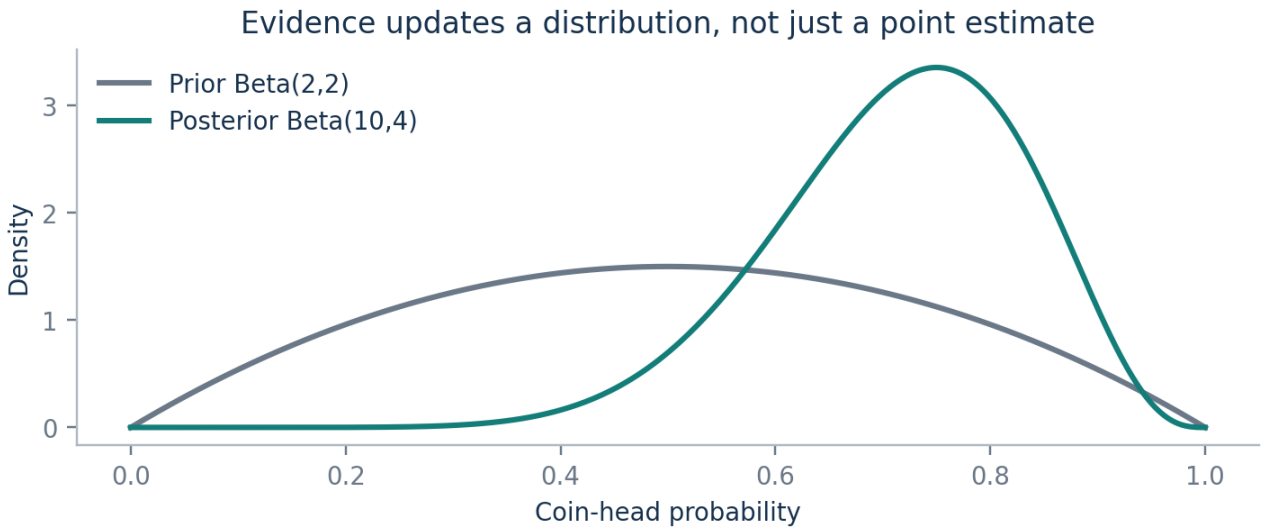


Figure 35. Probabilistic ML learns or reasons with distributions rather than single-point predictions.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. A Bayesian interval is conditional on the chosen model and prior; it is not a guarantee under misspecification.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when probabilistic machine learning would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Probabilistic ML learns or reasons with distributions rather than single-point predictions.

Formula. Posterior beliefs about parameters theta are proportional to likelihood of data D times prior beliefs.

Practical example. Quantify uncertainty in sparse experimental data; track hidden operating modes of machines.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for probabilistic machine learning.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 36 / 38 · CORE NOTES

36. End-to-End Machine Learning Projects

Learning objective. Explain the purpose of end-to-end machine learning projects, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

An end-to-end project begins with a decision problem, not an algorithm. Define an observable target, permitted features, baseline and cost of mistakes. Separate data chronologically or by group where required; build transformations inside a pipeline; evaluate on untouched holdout data. Save model artifacts and document training data, reproducibility, limitations, and monitoring triggers. The lifecycle continues after deployment.

When and where this matters

Build reproducible diabetes-outcome regression; deploy a manufacturing quality predictor.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Define problem, outcome and baseline
- Collect and document the data
- Split by group or time when needed
- EDA and leakage audit
- Fit preprocessing and model pipeline
- Tune on validation folds only
- Evaluate on untouched test data
- Interpret, save and monitor model

3. Mathematical model and notation

hat R_{test}=frac1{n_{test}}sum_i L(y_i,hat y_i)

The holdout estimate averages task-specific loss over test examples not used for training or model selection.

Worked numerical example — follow every step

If four holdout absolute errors are 3, 5, 0, and 4 units, MAE=(3+5+0+4)/4=3 units.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error
X,y=load_diabetes(return_X_y=True)
a,b,c,d=train_test_split(X,y,random_state=42,test_size=0.2)
model=make_pipeline(StandardScaler(),Ridge(alpha=10)).fit(a,c)
print('Training rows:',len(c),'test rows:',len(d))
```

```
11  print('Holdout MAE:',round(mean_absolute_error(d,model.predict(b)),2))
```

Verified console output

```
Training rows: 353 test rows: 89
Holdout MAE: 42.86
```

5. Visual reasoning



Figure 36. An end-to-end project begins with a decision problem, not an algorithm.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. Do not use a held-out test result to tune hyperparameters repeatedly; that gradually makes the test set part of training decisions.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when end-to-end machine learning projects would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. An end-to-end project begins with a decision problem, not an algorithm.

Formula. The holdout estimate averages task-specific loss over test examples not used for training or model selection.

Practical example. Build reproducible diabetes-outcome regression; deploy a manufacturing quality predictor.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for end-to-end machine learning projects.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 37 / 38 · CORE NOTES

37. Machine Learning Engineering and MLOps

Learning objective. Explain the purpose of machine learning engineering and mlops, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

MLOps applies software engineering discipline to data-dependent models. A reliable service versions code, data schemas, fitted models, metrics, and dependencies. Training jobs should be reproducible, deployment should validate request shapes, and monitoring should track quality as well as latency, failures, and input drift. Model retraining needs evaluation gates: automatically updating to a worse or unsafe model is not an improvement.

When and where this matters

Serve models through FastAPI and containers; monitor changing sensor data in production.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Reproducible pipelines.
- Software engineering practices.
- Model serialization.
- REST APIs.
- FastAPI.
- Docker fundamentals.
- Experiment tracking.
- MLflow.
- Data and model versioning.
- Model registries.
- CI/CD concepts.
- Model monitoring.
- Data drift.
- Concept drift.
- Logging.
- Observability.
- Automated testing.
- Model security.
- Cloud deployment fundamentals.

3. Mathematical model and notation

$$drift_j = distance(P_{train,j}, P_{serve,j})$$

A drift statistic compares a feature's training distribution with its serving-time distribution; the distance must be defined for the feature type.

Worked numerical example — follow every step

If 5 of 1000 inference requests fail schema validation, the observed invalid-request rate is 5/1000=0.005, or 0.5%. Logging the causes helps prioritize fixes.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
from pathlib import Path
import joblib
from sklearn.datasets import load_iris
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
X,y=load_iris(return_X_y=True)
model=make_pipeline(StandardScaler(),LogisticRegression(max_iter=300)).fit(X,y)
path=Path(__file__).with_suffix('.joblib')
joblib.dump(model,path)
loaded=joblib.load(path)
print('Serialized model bytes:',path.stat().st_size)
print('Prediction matches:',bool((model.predict(X[:5])==loaded.predict(X[:5])).all()
))
path.unlink()
```

Verified console output

```
Serialized model bytes: 1569
Prediction matches: True
```

5. Visual reasoning



Figure 37. MLOps applies software engineering discipline to data-dependent models.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. A model file loaded from an untrusted source may execute unsafe code in some serialization formats; control artifact provenance.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when machine learning engineering and mlops would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. MLOps applies software engineering discipline to data-dependent models.

Formula. A drift statistic compares a feature's training distribution with its serving-time distribution; the distance must be defined for the feature type.

Practical example. Serve models through FastAPI and containers; monitor changing sensor data in production.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for machine learning engineering and mlops.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

CHAPTER 38 / 38 · CORE NOTES

38. Machine Learning in Electrical Engineering and Embedded Systems

Learning objective. Explain the purpose of machine learning in electrical engineering and embedded systems, reproduce one relevant calculation, implement the example, and identify its key limitations.

1. Intuition, assumptions, and motivation

Embedded machine learning must operate under hard memory, energy, latency, and reliability constraints. Sensor signals often need filtering, windows, and time-frequency features before inference. TinyML runs compact models on microcontrollers; edge AI reduces cloud dependence and communication delay. Quantization and model compression help meet constraints but can reduce accuracy. Hardware-aware validation must include representative sensor noise, sampling behavior, and worst-case processing time.

When and where this matters

Detect motor faults using accelerometers; run real-time event detection on microcontrollers or UAV sensors.

2. Complete topic roadmap

The following entries preserve the requested syllabus for this chapter. They serve as a topic-by-topic checklist; the chapter examples develop the central mechanism rather than pretending that each bullet has been exhaustively proved.

- Sensor data preprocessing.
- Noise filtering.
- Time-domain signals.
- Frequency-domain features.
- Fourier Transform.
- Fast Fourier Transform.
- Signal classification.
- Predictive maintenance.
- Fault detection.
- Anomaly detection in electronics.
- Embedded machine learning.
- TinyML.
- Edge AI.
- Quantization.
- Memory and power constraints.
- Model latency.
- Real-time inference.
- Microcontroller deployment considerations.

3. Mathematical model and notation

$$f_k = \frac{k f_s}{N}$$

For an N-sample DFT from sampling rate f_s, frequency bin k corresponds to k*f_s/N hertz.

Worked numerical example — follow every step

A signal sampled at 1000 Hz for 1 second has N=1000 samples and frequency spacing f_s/N=1 Hz. A 50 Hz sinusoid aligns with bin k=50.

4. Runnable Python implementation

The following standalone script is executed as printed; it can also be run from the chapter scripts directory. The code is a teaching-sized example, not a production-ready workflow.

```
01  import numpy as np
02  # Simulated 50-Hz sensor waveform sampled at 1000 Hz.
03  fs=1000
04  x=np.arange(fs)/fs
05  signal=np.sin(2*np.pi*50*x)+0.3*np.sin(2*np.pi*120*x)
06  frequency=np.fft.rfftfreq(len(signal),d=1/fs)
07  power=np.abs(np.fft.rfft(signal))
08  top=np.argsort(power)[-2:]
09  print('Dominant frequencies (Hz):',sorted(frequency[top].astype(int).tolist()))
10  print('Window duration (s):',round(len(signal)/fs,3))
```

Verified console output

```
Dominant frequencies (Hz): [50, 120]
Window duration (s): 1.0
```

5. Visual reasoning

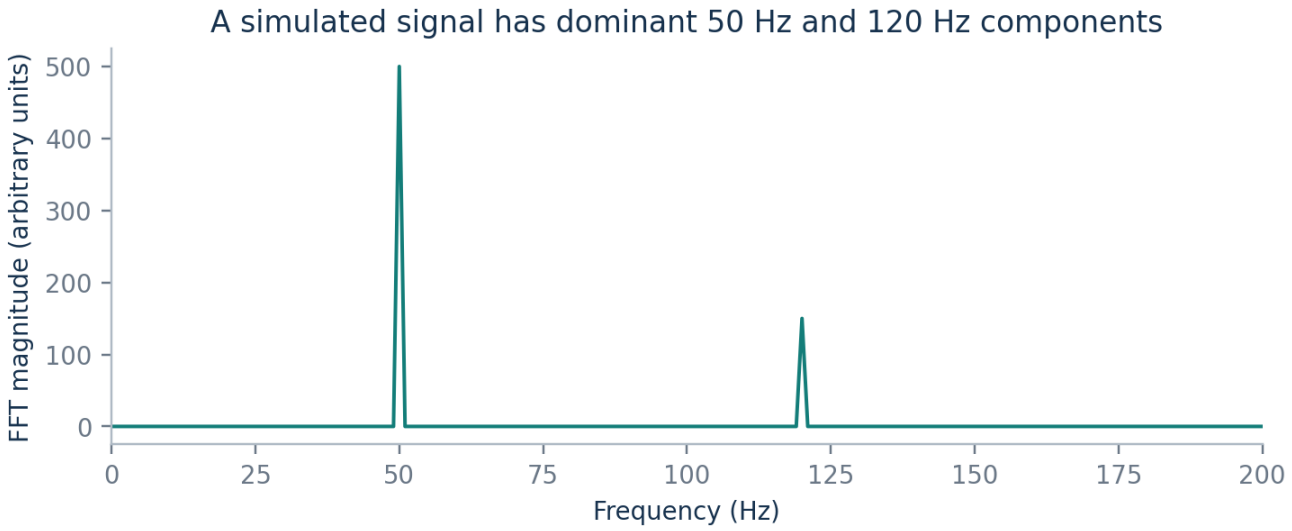


Figure 38. Embedded machine learning must operate under hard memory, energy, latency, and reliability constraints.

6. Interpretation, errors, and limitations

What the calculation and code tell us. They illustrate a specific mechanism on an intentionally small, reproducible dataset. Do not extrapolate a reported score to a different task or population without independent evaluation.

Common mistake. Frequency components above the Nyquist frequency alias unless the acquisition system filters them appropriately.

Implementation check. Verify input shapes, inspect output ranges and metric units, and ensure validation examples were not used while fitting learned transformations.

7. Three-level exercises

- Level 1 — Explain, in your own words, when machine learning in electrical engineering and embedded systems would be appropriate and when it would be a poor choice.
- Level 2 — Reproduce the worked calculation with pencil and paper; verify symbol definitions, substitutions and units.
- Level 3 — Run the Python file, modify one meaningful algorithm setting or input, and explain the difference. For predictive models, use an untouched split.

8. Revision and interview card

Key idea. Embedded machine learning must operate under hard memory, energy, latency, and reliability constraints.

Formula. For an N-sample DFT from sampling rate f_s, frequency bin k corresponds to k*f_s/N hertz.

Practical example. Detect motor faults using accelerometers; run real-time event detection on microcontrollers or UAV sensors.

Interview prompt. Explain the central assumption, identify one mathematical formula, and describe one failure mode for machine learning in electrical engineering and embedded systems.

Self-check solution. The detailed numerical solution appears in Section 3 of this chapter; the companion script and its verified output serve as the Level 3 reference implementation.

PRACTICAL LABS | 12 EXECUTED PROJECTS

Hands-on machine learning projects

These twelve labs are small, reproducible demonstrations spanning a range of tasks. Public datasets are named explicitly; otherwise the data are described as synthetic. Reported metrics come from executing the bundled code, not claims about production validity. Tiny illustrative datasets cannot support generalization conclusions.

Project 01 — House-price baseline

Synthetic square-footage data; baseline is predicting training mean. Demonstrates regression split and MAE.

Baseline and evaluation. The script compares a baseline or reference to a fitted method, or provides a compact sanity-check evaluation. The result is specific to this dataset and split.

```
01  # House-price baseline
02  # Synthetic square-footage data; baseline is predicting training mean. Demonstrates
regression split and MAE.
03  import numpy as np
04  from sklearn.model_selection import train_test_split
05  from sklearn.linear_model import LinearRegression
06  from sklearn.metrics import mean_absolute_error
07  rng=np.random.default_rng(11)
08  area=rng.uniform(800,3200,250).reshape(-1,1)
09  price=60000+210*area[:,0]+rng.normal(0,35000,250)
10  Xtr,Xte,ytr,yte=train_test_split(area,price,test_size=.2,random_state=42)
11  model=LinearRegression().fit(Xtr,ytr)
12  naive=np.full_like(yte,ytr.mean())
13  print('Dataset: 250 synthetic properties')
14  print('Mean baseline MAE:',round(mean_absolute_error(yte,naive)))
15  print('Linear regression MAE:',round(mean_absolute_error(yte,model.predict(Xte))))
16  print('Fitted USD/sq ft:',round(model.coef_[0],2))
```

Verified console output

```
Dataset: 250 synthetic properties
Mean baseline MAE: 139242
Linear regression MAE: 31803
Fitted USD/sq ft: 214.18
```

Extension. Replace the illustrative dataset with a sufficiently large, appropriately licensed dataset; repeat with multiple splits or group-aware evaluation and report uncertainty, error patterns, and deployment limitations.

Project 02 — Diabetes-outcome regression

scikit-learn load_diabetes benchmark (real standardized features; quantitative disease-progression target, not a diagnostic class).

Baseline and evaluation. The script compares a baseline or reference to a fitted method, or provides a compact sanity-check evaluation. The result is specific to this dataset and split.

```
01  # Diabetes-outcome regression
02  # scikit-learn load_diabetes benchmark (real standardized features; quantitative
disease-progression target, not a diagnostic class).
03  from sklearn.datasets import load_diabetes
04  from sklearn.model_selection import train_test_split
05  from sklearn.pipeline import make_pipeline
06  from sklearn.preprocessing import StandardScaler
07  from sklearn.linear_model import Ridge
08  from sklearn.dummy import DummyRegressor
09  from sklearn.metrics import mean_absolute_error
```

```
X,y=load_diabetes(return_X_y=True)
a,b,c,d=train_test_split(X,y,test_size=.2,random_state=42)
models={'mean baseline':DummyRegressor(strategy='mean'),
        'ridge':make_pipeline(StandardScaler(),Ridge(alpha=10.0))}
for name,model in models.items():
    model.fit(a,c)
    print(name,'MAE:',round(mean_absolute_error(d,model.predict(b)),2))
print('Test set size:',len(d))
```

Verified console output

```
mean baseline MAE: 64.01
ridge MAE: 42.86
Test set size: 89
```

Extension. Replace the illustrative dataset with a sufficiently large, appropriately licensed dataset; repeat with multiple splits or group-aware evaluation and report uncertainty, error patterns, and deployment limitations.

Project 03 — Spam detection

Sixteen synthetic messages with spam/legitimate labels. Use only as a teaching demonstration, not a deployment-ready filter.

Baseline and evaluation. The script compares a baseline or reference to a fitted method, or provides a compact sanity-check evaluation. The result is specific to this dataset and split.

```
01  # Spam detection
02  # Sixteen synthetic messages with spam/legitimate labels. Use only as a teaching
demonstration, not a deployment-ready filter.
03  from sklearn.pipeline import make_pipeline
04  from sklearn.feature_extraction.text import CountVectorizer
05  from sklearn.naive_bayes import MultinomialNB
06  from sklearn.dummy import DummyClassifier
07  from sklearn.model_selection import train_test_split
08  from sklearn.metrics import f1_score
09  spam=['claim your free prize','exclusive cash award','win money now','free prize
click',
10        'urgent lottery payment','claim award now','free lottery ticket','big cash
jackpot']
11  legit=['project meeting agenda','invoice attached today','research group
discussion',
12         'meeting notes attached','your project update','team lunch schedule',
13         'lecture slides tomorrow','please review report']
14  texts=spam+legit;y=[1]*8+[0]*8
15  a,b,c,d=train_test_split(texts,y,test_size=.25,stratify=y,random_state=7)
16  clf=make_pipeline(CountVectorizer(),MultinomialNB()).fit(a,c)
17  base=DummyClassifier(strategy='most_frequent').fit([[0]]*len(c),c)
18  print('Baseline F1:',round(f1_score(d,base.predict([[0]]*len(d)),zero_division=0),3)
)
```

CODE CONTINUED

```
19  print('Model F1:',round(f1_score(d,clf.predict(b),zero_division=0),3))
20  print('Test messages:',len(d))
```

Verified console output

```
Baseline F1: 0.0
Model F1: 1.0
Test messages: 4
```

Extension. Replace the illustrative dataset with a sufficiently large, appropriately licensed dataset; repeat with multiple splits or group-aware evaluation and report uncertainty, error patterns, and deployment limitations.

Project 04 — Customer segmentation

Synthetic three-group purchasing profiles; compare k-means against a one-centroid baseline via inertia.

Baseline and evaluation. The script compares a baseline or reference to a fitted method, or provides a compact sanity-check evaluation. The result is specific to this dataset and split.

```
01  # Customer segmentation
02  # Synthetic three-group purchasing profiles; compare k-means against a one-centroid
baseline via inertia.
03  import numpy as np
04  from sklearn.datasets import make_blobs
05  from sklearn.cluster import KMeans
06  from sklearn.preprocessing import StandardScaler
07  X,_=make_blobs(n_samples=240,centers=[[5,10],[30,50],[60,20]],cluster_std=4,
random_state=42)
08  Z=StandardScaler().fit_transform(X)
09  for k in [1,3]:
10      model=KMeans(n_clusters=k,random_state=42,n_init=10).fit(Z)
```

```
    print('Clusters:',k,'inertia:',round(model.inertia_,2))
print('Synthetic customers:',len(X))
```

Verified console output

```
Clusters: 1 inertia: 480.0
Clusters: 3 inertia: 18.61
Synthetic customers: 240
```

Extension. Replace the illustrative dataset with a sufficiently large, appropriately licensed dataset; repeat with multiple splits or group-aware evaluation and report uncertainty, error patterns, and deployment limitations.

Project 05 — Handwritten digits

Public scikit-learn digits benchmark, 8x8 grayscale images; test against a majority-class baseline.

Baseline and evaluation. The script compares a baseline or reference to a fitted method, or provides a compact sanity-check evaluation. The result is specific to this dataset and split.

```
# Handwritten digits
# Public scikit-learn digits benchmark, 8x8 grayscale images; test against a
majority-class baseline.
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.dummy import DummyClassifier
from sklearn.metrics import accuracy_score
X,y=load_digits(return_X_y=True)
a,b,c,d=train_test_split(X,y,test_size=.2,stratify=y,random_state=42)
for name,model in [('majority',DummyClassifier(strategy='most_frequent')),('kNN',
KNeighborsClassifier(n_neighbors=3))]:
    model.fit(a,c)
    print(name,'accuracy:',round(accuracy_score(d,model.predict(b)),3))
print('Test digits:',len(d))
```

Verified console output

```
majority accuracy: 0.1
kNN accuracy: 0.986
Test digits: 360
```

Extension. Replace the illustrative dataset with a sufficiently large, appropriately licensed dataset; repeat with multiple splits or group-aware evaluation and report uncertainty, error patterns, and deployment limitations.

Project 06 — Image classification in PyTorch

Public scikit-learn digits images processed by a small PyTorch MLP (fully connected image classifier for fast reproducibility).

Baseline and evaluation. The script compares a baseline or reference to a fitted method, or provides a compact sanity-check evaluation. The result is specific to this dataset and split.

```
# Image classification in PyTorch
# Public scikit-learn digits images processed by a small PyTorch MLP (fully
connected image classifier for fast reproducibility).
import numpy as np
import torch
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
torch.manual_seed(7)
X,y=load_digits(return_X_y=True)
a,b,c,d=train_test_split(X/16.,y,test_size=.2,stratify=y,random_state=42)
a=torch.tensor(a,dtype=torch.float32);b=torch.tensor(b,dtype=torch.float32)
c=torch.tensor(c,dtype=torch.long);d=torch.tensor(d,dtype=torch.long)
net=torch.nn.Sequential(torch.nn.Linear(64,64),torch.nn.ReLU(),torch.nn.Linear(64,
10))
opt=torch.optim.Adam(net.parameters(),lr=.01)
for _ in range(18):
    opt.zero_grad();loss=torch.nn.functional.cross_entropy(net(a),c)
    loss.backward();opt.step()
with torch.no_grad():
    acc=float((net(b).argmax(1)==d).float().mean())
```

CODE CONTINUED

```
print('Holdout image accuracy:',round(acc,3))
print('Input features:',a.shape[1])
```

Verified console output

```
Holdout image accuracy: 0.889
Input features: 64
```

Extension. Replace the illustrative dataset with a sufficiently large, appropriately licensed dataset; repeat with multiple splits or group-aware evaluation and report uncertainty, error patterns, and deployment limitations.

Project 07 — Chronological time-series forecasting

Synthetic trend-plus-seasonality series, lag-based forecasting vs naive last-value baseline.

Baseline and evaluation. The script compares a baseline or reference to a fitted method, or provides a compact sanity-check evaluation. The result is specific to this dataset and split.

```
# Chronological time-series forecasting
# Synthetic trend-plus-seasonality series, lag-based forecasting vs naive
last-value baseline.
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error
rng=np.random.default_rng(7);t=np.arange(140)
y=10+.06*t+2*np.sin(2*np.pi*t/12)+rng.normal(0,.4,len(t))
X=np.column_stack([y[:-2],y[1:-1]]);target=y[2:]
cut=110;model=LinearRegression().fit(X[:cut],target[:cut])
pred=model.predict(X[cut:]);naive=X[cut:,1]
print('Chronological holdout points:',len(pred))
print('Last-value MAE:',round(mean_absolute_error(target[cut:],naive),3))
print('Linear lag model MAE:',round(mean_absolute_error(target[cut:],pred),3))
```

Verified console output

```
Chronological holdout points: 28
Last-value MAE: 0.815
Linear lag model MAE: 0.763
```

Extension. Replace the illustrative dataset with a sufficiently large, appropriately licensed dataset; repeat with multiple splits or group-aware evaluation and report uncertainty, error patterns, and deployment limitations.

Project 08 — Anomaly detection

Synthetic normal vibration readings with injected spikes; compare a statistical threshold with Isolation Forest.

Baseline and evaluation. The script compares a baseline or reference to a fitted method, or provides a compact sanity-check evaluation. The result is specific to this dataset and split.

```
# Anomaly detection
# Synthetic normal vibration readings with injected spikes; compare a statistical
threshold with Isolation Forest.
import numpy as np
from sklearn.ensemble import IsolationForest
from sklearn.metrics import precision_score,recall_score
rng=np.random.default_rng(7)
X=np.r_[rng.normal(0,1,150),rng.normal(6,.4,10)].reshape(-1,1)
y=np.r_[np.zeros(150,dtype=int),np.ones(10,dtype=int)]
threshold=(X[:,0]>3).astype(int)
forest=(IsolationForest(contamination=10/160,random_state=7).fit_predict(X)==-1)
.astype(int)
for name,pred in [('z-style threshold',threshold),('isolation forest',forest)]:
    print(name,'precision:',round(precision_score(y,pred,zero_division=0),3),
          'recall:',round(recall_score(y,pred,zero_division=0),3))
```

Verified console output

```
z-style threshold precision: 1.0 recall: 1.0
isolation forest precision: 0.5 recall: 0.5
```

Extension. Replace the illustrative dataset with a sufficiently large, appropriately licensed dataset; repeat with multiple splits or group-aware evaluation and report uncertainty, error patterns, and deployment limitations.

Project 09 — Recommendation baseline

Toy user-item rating table with missing entries; compare popularity baseline with nearest-user weighted scoring.

Baseline and evaluation. The script compares a baseline or reference to a fitted method, or provides a compact sanity-check evaluation. The result is specific to this dataset and split.

```
# Recommendation baseline
# Toy user-item rating table with missing entries; compare popularity baseline with
nearest-user weighted scoring.
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
R=np.array([[5,4,0,0],[4,5,1,0],[0,0,5,4],[0,1,4,5]],float)
observed=R>0
popularity=R.sum(0)/np.maximum(observed.sum(0),1)
user=0
similarity=cosine_similarity(R)[user].copy();similarity[user]=0
weighted=np.sum(similarity[:,None]*R,axis=0)/np.maximum(np.sum(similarity[:,
None]*observed,axis=0),1e-9)
unseen=np.where(~observed[user])[0]
print('Unseen item indices:',unseen.tolist())
print('Popularity scores:',popularity[unseen].round(2).tolist())
print('Neighbor-weighted scores:',weighted[unseen].round(2).tolist())
```

Verified console output

```
Unseen item indices: [2, 3]
Popularity scores: [3.33, 4.5]
Neighbor-weighted scores: [1.27, 5.0]
```

Extension. Replace the illustrative dataset with a sufficiently large, appropriately licensed dataset; repeat with multiple splits or group-aware evaluation and report uncertainty, error patterns, and deployment limitations.

Project 10 — Predictive maintenance

Synthetic motor signals transformed to RMS and spectral features; pipeline fits equipment-state labels.

Baseline and evaluation. The script compares a baseline or reference to a fitted method, or provides a compact sanity-check evaluation. The result is specific to this dataset and split.

```
# Predictive maintenance
# Synthetic motor signals transformed to RMS and spectral features; pipeline fits
equipment-state labels.
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.dummy import DummyClassifier
rng=np.random.default_rng(8);n=240;N=128;fs=512
labels=np.r_[np.zeros(n//2,dtype=int),np.ones(n//2,dtype=int)]
features=[]
for y in labels:
    t=np.arange(N)/fs
    wave=np.sin(2*np.pi*40*t)+y*.7*np.sin(2*np.pi*90*t)+rng.normal(0,.15,N)
    mag=np.abs(np.fft.rfft(wave))
    features.append([np.sqrt(np.mean(wave**2)),mag[np.argmin(abs(np.fft.rfftfreq(N,
1/fs)-90))]])
X=np.asarray(features)
a,b,c,d=train_test_split(X,labels,test_size=.25,stratify=labels,random_state=7)
for name,model in [('majority',DummyClassifier(strategy='most_frequent')),('forest',
RandomForestClassifier(n_estimators=30,random_state=7))]:
    model.fit(a,c)
```

CODE CONTINUED

```
    print(name,'test accuracy:',round(model.score(b,d),3))
print('Important: all data are synthetic.')
```

Verified console output

```
majority test accuracy: 0.5
forest test accuracy: 1.0
Important: all data are synthetic.
```

Extension. Replace the illustrative dataset with a sufficiently large, appropriately licensed dataset; repeat with multiple splits or group-aware evaluation and report uncertainty, error patterns, and deployment limitations.

Project 11 — Text sentiment classification

Small labeled illustrative texts, TF-IDF feature learning and logistic classifier with majority baseline.

Baseline and evaluation. The script compares a baseline or reference to a fitted method, or provides a compact sanity-check evaluation. The result is specific to this dataset and split.

```
01  # Text sentiment classification
02  # Small labeled illustrative texts, TF-IDF feature learning and logistic classifier
with majority baseline.
03  from sklearn.pipeline import make_pipeline
04  from sklearn.feature_extraction.text import TfidfVectorizer
05  from sklearn.linear_model import LogisticRegression
06  from sklearn.dummy import DummyClassifier
07  from sklearn.model_selection import train_test_split
08  from sklearn.metrics import accuracy_score
09  texts=['great design','excellent results','very useful','high quality','good
performance','works well',
10         'terrible design','poor results','not useful','low quality','bad
performance','fails often']
11  y=[1]*6+[0]*6
12  a,b,c,d=train_test_split(texts,y,test_size=.33,stratify=y,random_state=7)
13  model=make_pipeline(TfidfVectorizer(ngram_range=(1,2)),LogisticRegression()).fit(a,
c)
14  base=DummyClassifier(strategy='most_frequent').fit([[0]]*len(c),c)
15  print('Baseline accuracy:',round(accuracy_score(d,base.predict([[0]]*len(d))),3))
16  print('Text model accuracy:',round(accuracy_score(d,model.predict(b)),3))
17  print('Test set count:',len(d))
```

Verified console output

```
Baseline accuracy: 0.5
Text model accuracy: 0.25
Test set count: 4
```

Extension. Replace the illustrative dataset with a sufficiently large, appropriately licensed dataset; repeat with multiple splits or group-aware evaluation and report uncertainty, error patterns, and deployment limitations.

Project 12 — Minimal Transformer model

Synthetic token sequences with class equal to whether token 1 appears; a small Transformer encoder is trained and evaluated.

Baseline and evaluation. The script compares a baseline or reference to a fitted method, or provides a compact sanity-check evaluation. The result is specific to this dataset and split.

```
01  # Minimal Transformer model
02  # Synthetic token sequences with class equal to whether token 1 appears; a small
Transformer encoder is trained and evaluated.
03  import numpy as np
04  import torch
05  torch.manual_seed(7)
06  rng=np.random.default_rng(7)
07  X=rng.integers(1,20,size=(250,6));y=(X==1).any(axis=1).astype(int)
08  train=200
09  xtr=torch.tensor(X[:train],dtype=torch.long);xte=torch.tensor(X[train:],
dtype=torch.long)
10  ytr=torch.tensor(y[:train],dtype=torch.long);yte=torch.tensor(y[train:],
dtype=torch.long)
11  class TinyTransformer(torch.nn.Module):
12      def __init__(self):
13          super().__init__()
14          self.emb=torch.nn.Embedding(20,16)
15          self.pos=torch.nn.Embedding(6,16)
16          self.enc=torch.nn.TransformerEncoderLayer(d_model=16,nhead=2,
```

```
dim_feedforward=32,batch_first=True,dropout=0)
17          self.out=torch.nn.Linear(16,2)
18      def forward(self,x):
```

CODE CONTINUED

```
19          h=self.emb(x)+self.pos(torch.arange(x.shape[1]))
20          return self.out(self.enc(h).mean(1))
21  net=TinyTransformer();opt=torch.optim.Adam(net.parameters(),lr=.01)
22  for _ in range(18):
23      opt.zero_grad();loss=torch.nn.functional.cross_entropy(net(xtr),ytr)
24      loss.backward();opt.step()
25  with torch.no_grad():acc=float((net(xte).argmax(1)==yte).float().mean())
26  print('Holdout accuracy:',round(acc,3))
27  print('Baseline negative class accuracy:',round(float((yte==0).float().mean()),3))
28  print('Token sequence length:',xtr.shape[1])
```

Verified console output

```
Holdout accuracy: 1.0
Baseline negative class accuracy: 0.8
Token sequence length: 6
```

Extension. Replace the illustrative dataset with a sufficiently large, appropriately licensed dataset; repeat with multiple splits or group-aware evaluation and report uncertainty, error patterns, and deployment limitations.

Appendix A — Mathematical formula sheet

A compact reference. Symbols and denominator conventions can vary with libraries; each application must verify shapes, assumptions, and units.

Weighted prediction

$$\hat{y} = w^T x + b$$

Linear predictor; feature and coefficient vectors must align.

Mean squared error

MSE=frac1nsum_i(y_i-hat y_i)^2

Regression loss, in squared target units.

Mean absolute error

MAE=frac1nsum_i|y_i-hat y_i|

Regression error in original target units.

Ridge loss

J=frac1n|y-Xw|_2^2+lambda|w|_2^2

Squared penalty shrinks coefficients.

Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Binary logit-to-probability mapping.

Softmax

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Normalized class probabilities.

Cross entropy

$$CE = -\sum_k y_k \log p_k$$

Loss for categorical predictions.

Gradient update

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

Steepest-descent update.

Bayes rule

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}$$

Probability update conditional on evidence.

Sample mean

bar x=frac1nsum_i x_i

Arithmetic mean.

Sample variance

s^2=frac1{n-1}sum_i(x_i-bar x)^2

Bessel-corrected sample variance.

Precision

$$P = \frac{TP}{TP + FP}$$

Correct fraction of predicted positives.

Recall

$$R = \frac{TP}{TP + FN}$$

Detected fraction of actual positives.

F1

$$F_1 = \frac{2PR}{P + R}$$

Harmonic mean of precision and recall.

Euclidean distance

$$d_2(x, z) = \sqrt{\sum_i (x_i - z_i)^2}$$

Two-point straight-line distance.

Gini

$$G = 1 - \sum_k p_k^2$$

Node class impurity.

Entropy

$$H = -\sum_k p_k \log p_k$$

Uncertainty measure.

PCA score

$$z = X_c v_1$$

Projection on first principal axis.

Attention

$$A = softmax(QK^T/\sqrt{d_k})V$$

Scaled dot-product attention.

Q-learning

Qgets Q+alpha(r+gammamax_{aprime}Qprime-Q)

Temporal difference update.

Frequency bins

$$f_k = kf_s/N$$

DFT resolution is sample rate divided by window length.

Appendix B — Algorithm comparison

Key strengths and weaknesses are qualitative. Computational complexity depends on data dimensions, solvers, implementation and hardware; refer to specific libraries when sizing a real system.

Linear regression

Learning type	Mathematical idea	Strength	Limitation
Supervised / regression	Squared residuals	Fast, interpretable	Sensitive to extreme points

Primary hyperparameter(s). fit_intercept

Python reference. LinearRegression

Ridge / Lasso

Learning type	Mathematical idea	Strength	Limitation
Supervised / regression	L2 / L1 penalized loss	Stabilizes coefficients	Tune penalty; scaling matters

Primary hyperparameter(s). alpha

Python reference. Ridge / Lasso

Logistic regression

Learning type	Mathematical idea	Strength	Limitation
Supervised / classification	Logistic likelihood	Good calibrated baseline	Linear log-odds

Primary hyperparameter(s). C, penalty

Python reference. LogisticRegression

kNN

Learning type	Mathematical idea	Strength	Limitation
Supervised / both	Distance and local vote	Nonparametric	Scaling and prediction cost

Primary hyperparameter(s). n_neighbors

Python reference. KNeighborsClassifier

Naive Bayes

Learning type	Mathematical idea	Strength	Limitation
Supervised / classification	Conditional Bayes factorization	Fast and data-efficient	Independence assumption

Primary hyperparameter(s). alpha

Python reference. MultinomialNB

SVM

Learning type	Mathematical idea	Strength	Limitation
Supervised / both	Maximum margin and kernels	Effective in moderate dimensions	Tuning and scaling

Primary hyperparameter(s). C, gamma, kernel

Python reference. SVC / SVR

Decision tree

Learning type	Mathematical idea	Strength	Limitation
Supervised / both	Recursive impurity reduction	Readable nonlinear rules	High variance

Primary hyperparameter(s). max_depth

Python reference. DecisionTreeClassifier

Random forest

Learning type	Mathematical idea	Strength	Limitation
Supervised / both	Bagging of randomized trees	Strong tabular baseline	Less transparent

Primary hyperparameter(s). n_estimators

Python reference. RandomForestClassifier

Gradient boosting

Learning type	Mathematical idea	Strength	Limitation
Supervised / both	Sequential correction	Strong tabular accuracy	Tuning needed

Primary hyperparameter(s). learning_rate

Python reference. HistGradientBoosting*

k-means

Learning type	Mathematical idea	Strength	Limitation
Unsupervised	Within-cluster SSE	Simple and scalable	Assumes numeric geometry

Primary hyperparameter(s). n_clusters

Python reference. KMeans

DBSCAN

Learning type	Mathematical idea	Strength	Limitation
Unsupervised	Density-connected regions	Irregular cluster shapes	Parameter sensitivity

Primary hyperparameter(s). eps, min_samples

Python reference. DBSCAN

PCA

Learning type	Mathematical idea	Strength	Limitation
Unsupervised	Maximum projected variance	Linear compression	Feature scaling important

Primary hyperparameter(s). n_components

Python reference. PCA

Isolation Forest

Learning type	Mathematical idea	Strength	Limitation
Unsupervised	Random-partition path lengths	Scalable anomaly detector	Requires contamination judgement

Primary hyperparameter(s). contamination

Python reference. IsolationForest

MLP

Learning type	Mathematical idea	Strength	Limitation
Supervised / both	Layered nonlinear features	Flexible approximation	Data and tuning needs

Primary hyperparameter(s). hidden_layer_sizes

Python reference. MLPClassifier

CNN

Learning type	Mathematical idea	Strength	Limitation
Deep learning	Spatial convolution and sharing	Strong on images	Domain shift

Primary hyperparameter(s). kernel_size

Python reference. torch.nn.Conv2d

Transformer

Learning type	Mathematical idea	Strength	Limitation
Deep learning	Attention and feedforward blocks	Flexible sequence modeling	Compute and context cost

Primary hyperparameter(s). heads, layers

Python reference. torch.nn.Transformer

Q-learning

Learning type	Mathematical idea	Strength	Limitation
Reinforcement learning	Bellman TD action values	Simple discrete control	State-action explosion

Primary hyperparameter(s). alpha, gamma

Python reference. custom NumPy

Gaussian process

Learning type	Mathematical idea	Strength	Limitation
Probabilistic	Kernel covariance prior	Uncertainty-aware prediction	Cubic scaling naively

Primary hyperparameter(s). kernel

Python reference. GaussianProcessRegressor

Appendix C — Python machine learning cheat sheet

Environment. python -m venv .venv # create isolated environment

Install. python -m pip install -r requirements.txt

Arrays. import numpy as np; a = np.array([[1, 2], [3, 4]])

Matrix product. prediction = X @ weights

Pandas. frame = pd.read_csv("data.csv")

Missingness. frame.isna().sum()

Feature summary. frame.describe(include="all")

Split. train_test_split(X, y, test_size=0.2, random_state=42)

Pipeline. make_pipeline(StandardScaler(), LogisticRegression())

Cross-validation. cross_val_score(pipe, X, y, cv=5)

Grid search. GridSearchCV(pipe, params, cv=5)

Classification. classification_report(y_true, y_pred)

Regression. mean_absolute_error(y_true, y_pred)

PyTorch gradient. loss.backward(); optimizer.step(); optimizer.zero_grad()

Save. joblib.dump(model, "model.joblib")

Reproduce. Store package versions, seeds, data provenance and split rules

Appendix D — Evaluation metrics

Accuracy

Definition. (TP + TN) / total

Misleading under severe imbalance.

Precision

Definition. TP / (TP + FP)

Useful when false alarms are expensive.

Recall

Definition. TP / (TP + FN)

Useful when missed positives are expensive.

Specificity

Definition. TN / (TN + FP)

Measures true-negative detection.

F1 score

Definition. 2 PR / (P + R)

A precision/recall compromise; ignores TN.

ROC AUC

Definition. Area under TPR-vs-FPR curve

Threshold-independent ranking summary.

PR AUC

Definition. Area under precision-recall curve

Helpful under rare positives; baseline depends on prevalence.

MAE

Definition. Mean absolute residual

Same units as target.

MSE

Definition. Mean squared residual

Penalizes large errors; squared target units.

RMSE

Definition. Square root of MSE

Original units; sensitive to large errors.

R-squared

Definition. 1 - SSE/SST

Can be negative on held-out data.

Log loss

Definition. Negative log likelihood of labels

Punishes overconfident incorrect predictions.

Calibration

Definition. Agreement of predicted and observed frequencies

Must be validated on independent data.

Appendix E — Optimization method reference

Method	Core operation	Caution/use
Full-batch GD	Uses all examples per update	Smooth direction; expensive on large datasets.
SGD	Uses one or a mini-batch	Noisy but computationally efficient and often helpful for training.
Momentum	Accumulates smoothed direction	Can accelerate persistent gradients; tune momentum.
AdaGrad	Accumulates squared historical gradients	Adapts coordinate step sizes but may decay too far.
RMSProp	Exponentially averaged squared gradients	Adaptive scaling for nonstationary objectives.
Adam	First and second moment estimates	Practical default but not universally optimal.
AdamW	Adam with decoupled weight decay	Common deep learning optimizer for explicit decay.

Appendix F — Machine learning glossary

Definitions are intentionally concise. Related terms often require distinct definitions even when grouped into one curriculum chapter.

a 95% confidence interval mean. Across repeated samples, the procedure covers the fixed population parameter about 95% of the time under assumptions.

a baseline. A simple reference method that a proposed solution should outperform.

a Bayesian posterior. The parameter or latent distribution after conditioning prior beliefs on observed data.

a Bellman equation. A recursive relationship between a value and immediate reward plus expected future value.

a CNN exploit. Local spatial patterns and shared filter parameters.

a confusion matrix. A table comparing true and predicted classification labels.

a convex objective. An objective whose line segment between two points lies on or above the function, ensuring local minima are global.

a dot product calculate. The sum of corresponding coordinate products; it quantifies alignment and linear scores.

a GAN. A generator and discriminator trained with an adversarial objective.

a Gaussian mixture. A probabilistic combination of Gaussian components assigning soft component responsibilities.

a Gaussian process. A probabilistic distribution over functions specified through mean and covariance functions.

a graph neural network. A model that learns from entities and their relational neighborhood structure.

a learning rate. The scalar controlling the parameter update size.

a logit. The logarithm of probability odds, log(p/(1-p)).

a matrix rank. The number of linearly independent columns or rows.

a p-value. The probability, under the null model, of obtaining a test statistic at least as extreme as observed.

a partial derivative. A derivative with one variable changing while others are held fixed.

a perceptron. A simple weighted-input decision unit, often with threshold activation.

a residual. The observed target minus the fitted prediction.

a saddle point. A stationary point that is neither a local minimum nor a local maximum.

a spiking neural network. A network that represents computation with time-varying discrete spike events.

a tensor. A multidimensional array, often representing model inputs, weights, or activations.

a test set. Previously unused examples used for final evaluation after model selection.

a training set. Examples used to estimate fitted model parameters.

a Transformer. A neural architecture built primarily around attention and feedforward processing of representations.

a VAE. A latent-variable generative model trained with reconstruction and regularized latent distribution objectives.

a validation set. Held-out examples used to select and tune modeling decisions.

Adam. An optimizer that combines running gradient means and second moments into adaptive updates.

an autoregressive model. A model that factorizes a joint distribution into sequential conditional predictions.

an eigenvector. A nonzero direction preserved up to scalar multiplication by a square matrix.

an embedding. A learned or designed numeric vector representation of a discrete or complex entity.

an MDP. A formalization of sequential decision-making using states, actions, transitions, and rewards.

an object detector. A model that predicts object labels and locations in images.

an out-of-bag estimate. Evaluation on examples omitted from a bootstrap sample used to train a component model.

anomaly detection. Identifying observations that deviate from a learned or assumed normal pattern.

ARIMA. A classical autoregressive integrated moving-average forecasting family.

attention. A mechanism that computes context-dependent weighted combinations of representations.

backpropagation calculate. Gradients of network outputs or loss with respect to model parameters using chain-rule composition.

Bayes' theorem. A rule updating a prior probability using observation likelihood and evidence.

calibration. Agreement between stated probabilities and observed event frequencies.

concept drift. A change in the relationship to be predicted, such as $P(y|x)$, over time.

conditional independence. Two variables do not provide extra dependence once a third variable is known.

confidence in association rules. The share of antecedent-containing transactions that also contain the consequent.

correlation. A normalized measure of association, often Pearson linear association.

covariance. The expectation of centered variable products; it measures joint variation.

cross-entropy. Expected negative log probability assigned by a model to a target distribution.

cross-validation. Repeatedly fitting on training folds and assessing on excluded folds.

data drift. A change in input-feature distributions between reference and current data.

diffusion generation. Learning to reverse a progressively noisy transformation to generate samples.

domain shift. A difference between training and deployment data distributions or relationships.

entropy. A measure of uncertainty in a probability distribution.

exploration versus exploitation. Choosing uncertain actions to learn versus choosing current estimated best actions.

F1 score. The harmonic mean of precision and recall.

feature standardization. Subtracting a reference mean and dividing by its reference standard deviation.

fine-tuning. Continuing training of a pretrained model on another task or domain.

Gini impurity. One minus the sum of squared class proportions at a tree node.

gradient clipping. Limiting gradient magnitude to control unusually large updates.

gradient descent. Iteratively subtracting a scaled loss gradient from the current parameters.

imputation. Filling missing entries using a specified rule or fitted estimator.

intersection-over-union. Area overlap divided by area union for predicted and reference regions.

Isolation Forest. An ensemble that detects anomalies through unusually short random-partition paths.

k-means optimize. Within-cluster squared distances to assigned centroids.

KL divergence. A nonnegative measure of discrepancy between probability distributions; it is asymmetric.

kNN require. A meaningful distance or similarity, a neighborhood size k, and stored reference examples.

leakage. Using unavailable future, target, or holdout information during training or selection.

lift. Confidence divided by the baseline frequency of the consequent.

linear regression minimize. In ordinary least squares, the sum or mean of squared residuals.

logistic regression predict. Probabilities via a linear log-odds score transformed by a sigmoid or softmax.

LoRA. Parameter-efficient tuning through learned low-rank updates to selected model weights.

MAP estimation. Maximizing the posterior probability, combining likelihood with a prior.

maximum likelihood estimation. Choosing parameter values that maximize the probability or density of the observed data.

message passing. Updating node representations by combining information from connected nodes.

MLOps. Practices for reproducible training, testing, packaging, deployment, and monitoring of ML systems.

momentum do. Accumulates a direction from past gradients to smooth or accelerate updates.

Naive Bayes' main assumption. Features are conditionally independent given the class.

nested cross-validation. An outer evaluation loop with separate inner hyperparameter tuning.

Nyquist frequency. Half the sampling rate for a uniformly sampled signal.

one-hot encoding. Representing each category using binary indicators, generally one per level.

overfitting. A model learns training-specific patterns that do not transfer well to unseen data.

PCA maximize. Variance captured in each successive orthogonal component, given centered data.

permutation importance. A held-out performance change after disrupting the correspondence of one feature.

precision. TP/(TP+FP): correctness among predicted positives.

quantization. Representing computations or parameters at reduced numeric precision.

R-squared. One minus residual sum of squares divided by target total sum of squares, under a standard definition.

recall. TP/(TP+FN): coverage of actual positives.

regularization. Constraints or penalties intended to favor solutions that generalize better.

reinforcement learning. Learning sequential actions using rewards or costs from an environment.

retrieval-augmented generation. Retrieving external information to condition a generator's response.

self-supervised learning. Constructing training signals from the data itself, for example predicting masked tokens.

SHAP approximate or compute. Feature attribution values defined relative to a specified cooperative-game background and model setup.

SMOTE do. Constructs synthetic minority observations from nearby minority neighbors within training folds.

specificity. TN/(TN+FP): coverage of actual negatives.

statistical power. The probability of rejecting a false null under a specified alternative.

stochastic gradient descent. An update using a randomly sampled training example or mini-batch instead of the full dataset.

supervised learning. Learning a mapping from input features to observed targets.

support in association rules. The fraction of transactions that contain an itemset.

SVD decompose. A matrix into left singular vectors, singular values, and right singular vectors.

TF-IDF measure. A document-specific term weighting incorporating term frequency and corpus rarity.

the bias-variance tradeoff. The balance between systematic model restrictions and sensitivity to sampling noise.

the curse of dimensionality. Distances and density estimates can become less informative as dimension increases.

the difference between variance and standard deviation. Standard deviation is the square root of variance, so it uses the original units.

the gradient. The vector of partial derivatives of a scalar-valued function.

the Hessian. The matrix of second partial derivatives, describing local curvature.

the kernel trick. Computing inner products in an implicit feature space without explicitly mapping all coordinates.

time-series stationarity. A property describing time-invariant relevant distributional characteristics, depending on its precise definition.

TinyML. Machine learning designed to run on very resource-constrained embedded devices.

transfer learning. Adapting representations or parameters learned from one task to another.

underfitting. A model lacks capacity or sufficient training to capture important structure.

unsupervised learning. Discovering patterns without target labels, such as clusters or lower-dimensional structure.

Appendix G — 128 interview questions and answers

The questions progress broadly from fundamentals to advanced algorithms and systems. Each answer is deliberately concise; rehearse with a derivation or example for technical interviews.

001. How does AI differ from ML?

AI includes reasoning, search and planning; ML is the subset that estimates behavior from data.

002. How does ML differ from deep learning?

Deep learning uses multilayer neural networks; ML also includes trees, regression, kernels, and many other methods.

003. What is supervised learning?

Learning a mapping from input features to observed targets.

004. What is unsupervised learning?

Discovering patterns without target labels, such as clusters or lower-dimensional structure.

005. What is self-supervised learning?

Constructing training signals from the data itself, for example predicting masked tokens.

006. What is reinforcement learning?

Learning sequential actions using rewards or costs from an environment.

007. What is transfer learning?

Adapting representations or parameters learned from one task to another.

008. What is domain shift?

A difference between training and deployment data distributions or relationships.

009. What is a training set?

Examples used to estimate fitted model parameters.

010. What is a validation set?

Held-out examples used to select and tune modeling decisions.

011. What is a test set?

Previously unused examples used for final evaluation after model selection.

012. What is leakage?

Using unavailable future, target, or holdout information during training or selection.

013. What is a baseline?

A simple reference method that a proposed solution should outperform.

014. Why establish a decision objective first?

Because prediction value depends on intended actions, constraints, and error costs.

015. What is a tensor?

A multidimensional array, often representing model inputs, weights, or activations.

016. What does a dot product calculate?

The sum of corresponding coordinate products; it quantifies alignment and linear scores.

017. What is a matrix rank?

The number of linearly independent columns or rows.

018. What does SVD decompose?

A matrix into left singular vectors, singular values, and right singular vectors.

019. What is an eigenvector?

A nonzero direction preserved up to scalar multiplication by a square matrix.

020. Why does matrix shape matter?

An operation can only be valid when relevant dimensions align.

021. What is the gradient?

The vector of partial derivatives of a scalar-valued function.

022. What is a partial derivative?

A derivative with one variable changing while others are held fixed.

023. Why is the chain rule important?

It differentiates compositions and makes backpropagation possible.

024. What is the Hessian?

The matrix of second partial derivatives, describing local curvature.

025. What is a saddle point?

A stationary point that is neither a local minimum nor a local maximum.

026. What is a convex objective?

An objective whose line segment between two points lies on or above the function, ensuring local minima are global.

027. What is Bayes' theorem?

A rule updating a prior probability using observation likelihood and evidence.

028. What is the difference between variance and standard deviation?

Standard deviation is the square root of variance, so it uses the original units.

029. What is covariance?

The expectation of centered variable products; it measures joint variation.

030. What is correlation?

A normalized measure of association, often Pearson linear association.

031. What is conditional independence?

Two variables do not provide extra dependence once a third variable is known.

032. What is maximum likelihood estimation?

Choosing parameter values that maximize the probability or density of the observed data.

033. What is MAP estimation?

Maximizing the posterior probability, combining likelihood with a prior.

034. What is entropy?

A measure of uncertainty in a probability distribution.

035. What is cross-entropy?

Expected negative log probability assigned by a model to a target distribution.

036. What is KL divergence?

A nonnegative measure of discrepancy between probability distributions; it is asymmetric.

037. What does a 95% confidence interval mean?

Across repeated samples, the procedure covers the fixed population parameter about 95% of the time under assumptions.

038. What is a p-value?

The probability, under the null model, of obtaining a test statistic at least as extreme as observed.

039. What is statistical power?

The probability of rejecting a false null under a specified alternative.

040. Why is correlation not causation?

Confounding, reverse direction, or selection can create associations without a direct causal effect.

041. What is gradient descent?

Iteratively subtracting a scaled loss gradient from the current parameters.

042. What is stochastic gradient descent?

An update using a randomly sampled training example or mini-batch instead of the full dataset.

043. What is a learning rate?

The scalar controlling the parameter update size.

044. What does momentum do?

Accumulates a direction from past gradients to smooth or accelerate updates.

045. What is Adam?

An optimizer that combines running gradient means and second moments into adaptive updates.

046. What is gradient clipping?

Limiting gradient magnitude to control unusually large updates.

047. What is overfitting?

A model learns training-specific patterns that do not transfer well to unseen data.

048. What is underfitting?

A model lacks capacity or sufficient training to capture important structure.

049. What is the bias-variance tradeoff?

The balance between systematic model restrictions and sensitivity to sampling noise.

050. What is regularization?

Constraints or penalties intended to favor solutions that generalize better.

051. How does L1 differ from L2 penalty?

L1 uses absolute coefficient magnitudes and may set some weights to zero; L2 uses squared magnitudes.

052. What is feature standardization?

Subtracting a reference mean and dividing by its reference standard deviation.

053. Why scale features inside CV folds?

Scaling on all data would reveal distribution statistics from validation folds.

054. What is one-hot encoding?

Representing each category using binary indicators, generally one per level.

055. What is imputation?

Filling missing entries using a specified rule or fitted estimator.

056. What does SMOTE do?

Constructs synthetic minority observations from nearby minority neighbors within training folds.

057. What does linear regression minimize?

In ordinary least squares, the sum or mean of squared residuals.

058. What is a residual?

The observed target minus the fitted prediction.

059. What is R-squared?

One minus residual sum of squares divided by target total sum of squares, under a standard definition.

060. What does logistic regression predict?

Probabilities via a linear log-odds score transformed by a sigmoid or softmax.

061. What is a logit?

The logarithm of probability odds, log(p/(1-p)).

062. What does kNN require?

A meaningful distance or similarity, a neighborhood size k, and stored reference examples.

063. What is the curse of dimensionality?

Distances and density estimates can become less informative as dimension increases.

064. What is Naive Bayes' main assumption?

Features are conditionally independent given the class.

065. What do support vectors determine?

They are influential training points that define a support vector machine boundary.

066. What is the kernel trick?

Computing inner products in an implicit feature space without explicitly mapping all coordinates.

067. What is Gini impurity?

One minus the sum of squared class proportions at a tree node.

068. How does bagging help?

Averaging models fitted on resampled data can reduce prediction variance.

069. How does boosting differ from bagging?

Boosting trains successive learners to correct errors; bagging trains resampled models more independently.

070. What is an out-of-bag estimate?

Evaluation on examples omitted from a bootstrap sample used to train a component model.

071. What does k-means optimize?

Within-cluster squared distances to assigned centroids.

072. How does DBSCAN define a cluster?

By density-connected groups of observations around sufficiently populated neighborhoods.

073. What is a Gaussian mixture?

A probabilistic combination of Gaussian components assigning soft component responsibilities.

074. What does PCA maximize?

Variance captured in each successive orthogonal component, given centered data.

075. How does t-SNE differ from PCA?

t-SNE is a nonlinear neighborhood visualization method; PCA is a linear projection with interpretable variance.

076. What is anomaly detection?

Identifying observations that deviate from a learned or assumed normal pattern.

077. What is Isolation Forest?

An ensemble that detects anomalies through unusually short random-partition paths.

078. What is support in association rules?

The fraction of transactions that contain an itemset.

079. What is confidence in association rules?

The share of antecedent-containing transactions that also contain the consequent.

080. What is lift?

Confidence divided by the baseline frequency of the consequent.

081. What is a confusion matrix?

A table comparing true and predicted classification labels.

082. What is precision?

TP/(TP+FP): correctness among predicted positives.

083. What is recall?

TP/(TP+FN): coverage of actual positives.

084. What is specificity?

TN/(TN+FP): coverage of actual negatives.

085. What is F1 score?

The harmonic mean of precision and recall.

086. When is PR-AUC especially informative?

When positive cases are rare and positive prediction quality matters.

087. What is calibration?

Agreement between stated probabilities and observed event frequencies.

088. What is cross-validation?

Repeatedly fitting on training folds and assessing on excluded folds.

089. What is nested cross-validation?

An outer evaluation loop with separate inner hyperparameter tuning.

090. Why avoid random CV in forecasting?

Random folds let future observations influence estimates for the past.

091. What is permutation importance?

A held-out performance change after disrupting the correspondence of one feature.

092. What does SHAP approximate or compute?

Feature attribution values defined relative to a specified cooperative-game background and model setup.

093. What is a perceptron?

A simple weighted-input decision unit, often with threshold activation.

094. What does backpropagation calculate?

Gradients of network outputs or loss with respect to model parameters using chain-rule composition.

095. Why use ReLU?

It provides simple nonlinearity and often avoids saturation in its positive regime.

096. Why use dropout?

To randomly omit activations during training as a regularization technique.

097. Why use residual connections?

They allow information and gradients to pass more directly across layers.

098. What does a CNN exploit?

Local spatial patterns and shared filter parameters.

099. What problem do LSTMs address?

They use gating to preserve or update recurrent information across longer sequences.

100. What is attention?

A mechanism that computes context-dependent weighted combinations of representations.

101. What is a Transformer?

A neural architecture built primarily around attention and feedforward processing of representations.

102. What is an embedding?

A learned or designed numeric vector representation of a discrete or complex entity.

103. What is fine-tuning?

Continuing training of a pretrained model on another task or domain.

104. What is LoRA?

Parameter-efficient tuning through learned low-rank updates to selected model weights.

105. What is a VAE?

A latent-variable generative model trained with reconstruction and regularized latent distribution objectives.

106. What is a GAN?

A generator and discriminator trained with an adversarial objective.

107. What is diffusion generation?

Learning to reverse a progressively noisy transformation to generate samples.

108. What is an autoregressive model?

A model that factorizes a joint distribution into sequential conditional predictions.

109. What is time-series stationarity?

A property describing time-invariant relevant distributional characteristics, depending on its precise definition.

110. What is ARIMA?

A classical autoregressive integrated moving-average forecasting family.

111. What does TF-IDF measure?

A document-specific term weighting incorporating term frequency and corpus rarity.

112. What is retrieval-augmented generation?

Retrieving external information to condition a generator's response.

113. What is an object detector?

A model that predicts object labels and locations in images.

114. What is intersection-over-union?

Area overlap divided by area union for predicted and reference regions.

115. What is an MDP?

A formalization of sequential decision-making using states, actions, transitions, and rewards.

116. What is a Bellman equation?

A recursive relationship between a value and immediate reward plus expected future value.

117. What is exploration versus exploitation?

Choosing uncertain actions to learn versus choosing current estimated best actions.

118. What is a graph neural network?

A model that learns from entities and their relational neighborhood structure.

119. What is message passing?

Updating node representations by combining information from connected nodes.

120. What is a Bayesian posterior?

The parameter or latent distribution after conditioning prior beliefs on observed data.

121. What is a Gaussian process?

A probabilistic distribution over functions specified through mean and covariance functions.

122. What is MLOps?

Practices for reproducible training, testing, packaging, deployment, and monitoring of ML systems.

123. What is concept drift?

A change in the relationship to be predicted, such as P(y|x), over time.

124. What is data drift?

A change in input-feature distributions between reference and current data.

125. What is quantization?

Representing computations or parameters at reduced numeric precision.

126. What is TinyML?

Machine learning designed to run on very resource-constrained embedded devices.

127. What is Nyquist frequency?

Half the sampling rate for a uniformly sampled signal.

128. What is a spiking neural network?

A network that represents computation with time-varying discrete spike events.

Appendix H — Derivations and worked checks

Derivation 1: Least-squares slope and gradient

For the one-parameter regression model y_hat_i = w x_i, choose J(w)=n^(-1) sum_i (y_i-w x_i)^2. The derivative of each squared residual is -2 x_i(y_i-w x_i) by the chain rule. Averaging gives the gradient. Setting it to zero gives w=sum_i x_i y_i / sum_i x_i^2 when the denominator is nonzero.

frac{dJ}{dw}=-frac2nsum_i x_i(y_i-wx_i)

$$w^* = \frac{\sum_i x_i y_i}{\sum_i x_i^2}$$

Derivation 2: Logistic cross-entropy gradient

For p_i=sigmoid(w^T x_i+b), the Bernoulli negative log-likelihood is -[y_i log p_i+(1-y_i)log(1-p_i)]. Because the sigmoid derivative is p_i(1-p_i), factors cancel in the chain rule and the derivative with respect to w is the average prediction error times input.

nabla_wJ=frac1nsum_i(p_i-y_i)x_i

Derivation 3: Normal equations

Write squared residual objective as ||y-Xw||^2. Differentiate to obtain -2X^T(y-Xw). The normal equations X^T X w=X^T y follow from the stationary condition. Prefer stable least-squares or QR/SVD solvers over explicit matrix inversion.

$$X^T X w = X^T y$$

Derivation 4: PCA variance maximization

For centered data with covariance S, a unit vector v gives projected variance v^T S v. A Lagrange multiplier enforces v^T v=1. Differentiating yields Sv=lambda v: optimal directions are covariance eigenvectors, and the leading eigenvalue gives maximum variance.

$$\max_{\|v\|_2 = 1} v^T S v$$

Derivation 5: Bayes posterior update

The product rule gives P(A,B)=P(A|B)P(B)=P(B|A)P(A). Rearranging for P(A|B) yields Bayes rule; the denominator sums the joint probability over possible causes. Without a base rate, positive-test probabilities can be badly misinterpreted.

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{\sum_{a} P(B \mid a)P(a)}$$

Exercise solution guide

Chapter 1. Level 1: articulate the assumption and principal use of foundations of ai and machine learning. Level 2: check the fully worked arithmetic in Chapter 1, Section 3. Level 3: execute scripts/chapter_01.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 2. Level 1: articulate the assumption and principal use of python for machine learning. Level 2: check the fully worked arithmetic in Chapter 2, Section 3. Level 3: execute scripts/chapter_02.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 3. Level 1: articulate the assumption and principal use of linear algebra. Level 2: check the fully worked arithmetic in Chapter 3, Section 3. Level 3: execute scripts/chapter_03.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 4. Level 1: articulate the assumption and principal use of calculus for machine learning. Level 2: check the fully worked arithmetic in Chapter 4, Section 3. Level 3: execute scripts/chapter_04.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 5. Level 1: articulate the assumption and principal use of probability theory. Level 2: check the fully worked arithmetic in Chapter 5, Section 3. Level 3: execute scripts/chapter_05.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 6. Level 1: articulate the assumption and principal use of statistics and statistical inference. Level 2: check the fully worked arithmetic in Chapter 6, Section 3. Level 3: execute scripts/chapter_06.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 7. Level 1: articulate the assumption and principal use of optimization. Level 2: check the fully worked arithmetic in Chapter 7, Section 3. Level 3: execute scripts/chapter_07.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 8. Level 1: articulate the assumption and principal use of understanding data. Level 2: check the fully worked arithmetic in Chapter 8, Section 3. Level 3: execute scripts/chapter_08.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 9. Level 1: articulate the assumption and principal use of exploratory data analysis. Level 2: check the fully worked arithmetic in Chapter 9, Section 3. Level 3: execute scripts/chapter_09.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 10. Level 1: articulate the assumption and principal use of preprocessing and feature engineering. Level 2: check the fully worked arithmetic in Chapter 10, Section 3. Level 3: execute scripts/chapter_10.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 11. Level 1: articulate the assumption and principal use of linear regression. Level 2: check the fully worked arithmetic in Chapter 11, Section 3. Level 3: execute scripts/chapter_11.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 12. Level 1: articulate the assumption and principal use of regularized and advanced regression. Level 2: check the fully worked arithmetic in Chapter 12, Section 3. Level 3: execute scripts/chapter_12.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 13. Level 1: articulate the assumption and principal use of classification fundamentals. Level 2: check the fully worked arithmetic in Chapter 13, Section 3. Level 3: execute scripts/chapter_13.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 14. Level 1: articulate the assumption and principal use of logistic regression. Level 2: check the fully worked arithmetic in Chapter 14, Section 3. Level 3: execute scripts/chapter_14.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 15. Level 1: articulate the assumption and principal use of k-nearest neighbors. Level 2: check the fully worked arithmetic in Chapter 15, Section 3. Level 3: execute scripts/chapter_15.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 16. Level 1: articulate the assumption and principal use of naive bayes. Level 2: check the fully worked arithmetic in Chapter 16, Section 3. Level 3: execute scripts/chapter_16.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 17. Level 1: articulate the assumption and principal use of support vector machines. Level 2: check the fully worked arithmetic in Chapter 17, Section 3. Level 3: execute scripts/chapter_17.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 18. Level 1: articulate the assumption and principal use of decision trees. Level 2: check the fully worked arithmetic in Chapter 18, Section 3. Level 3: execute scripts/chapter_18.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 19. Level 1: articulate the assumption and principal use of ensemble methods. Level 2: check the fully worked arithmetic in Chapter 19, Section 3. Level 3: execute scripts/chapter_19.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 20. Level 1: articulate the assumption and principal use of clustering. Level 2: check the fully worked arithmetic in Chapter 20, Section 3. Level 3: execute scripts/chapter_20.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 21. Level 1: articulate the assumption and principal use of dimensionality reduction. Level 2: check the fully worked arithmetic in Chapter 21, Section 3. Level 3: execute scripts/chapter_21.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 22. Level 1: articulate the assumption and principal use of anomaly detection. Level 2: check the fully worked arithmetic in Chapter 22, Section 3. Level 3: execute scripts/chapter_22.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 23. Level 1: articulate the assumption and principal use of association rules and recommender systems. Level 2: check the fully worked arithmetic in Chapter 23, Section 3. Level 3: execute scripts/chapter_23.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 24. Level 1: articulate the assumption and principal use of evaluating machine learning models. Level 2: check the fully worked arithmetic in Chapter 24, Section 3. Level 3: execute scripts/chapter_24.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 25. Level 1: articulate the assumption and principal use of model selection and hyperparameter tuning. Level 2: check the fully worked arithmetic in Chapter 25, Section 3. Level 3: execute scripts/chapter_25.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 26. Level 1: articulate the assumption and principal use of explainable and responsible machine learning. Level 2: check the fully worked arithmetic in Chapter 26, Section 3. Level 3: execute scripts/chapter_26.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 27. Level 1: articulate the assumption and principal use of neural network foundations. Level 2: check the fully worked arithmetic in Chapter 27, Section 3. Level 3: execute scripts/chapter_27.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 28. Level 1: articulate the assumption and principal use of deep learning architectures. Level 2: check the fully worked arithmetic in Chapter 28, Section 3. Level 3: execute scripts/chapter_28.py, reproduce the printed output, then justify any code

modifications and metric changes.

Chapter 29. Level 1: articulate the assumption and principal use of generative machine learning. Level 2: check the fully worked arithmetic in Chapter 29, Section 3. Level 3: execute scripts/chapter_29.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 30. Level 1: articulate the assumption and principal use of time-series machine learning. Level 2: check the fully worked arithmetic in Chapter 30, Section 3. Level 3: execute scripts/chapter_30.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 31. Level 1: articulate the assumption and principal use of natural language processing. Level 2: check the fully worked arithmetic in Chapter 31, Section 3. Level 3: execute scripts/chapter_31.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 32. Level 1: articulate the assumption and principal use of computer vision. Level 2: check the fully worked arithmetic in Chapter 32, Section 3. Level 3: execute scripts/chapter_32.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 33. Level 1: articulate the assumption and principal use of reinforcement learning. Level 2: check the fully worked arithmetic in Chapter 33, Section 3. Level 3: execute scripts/chapter_33.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 34. Level 1: articulate the assumption and principal use of graph machine learning. Level 2: check the fully worked arithmetic in Chapter 34, Section 3. Level 3: execute scripts/chapter_34.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 35. Level 1: articulate the assumption and principal use of probabilistic machine learning. Level 2: check the fully worked arithmetic in Chapter 35, Section 3. Level 3: execute scripts/chapter_35.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 36. Level 1: articulate the assumption and principal use of end-to-end machine learning projects. Level 2: check the fully worked arithmetic in Chapter 36, Section 3. Level 3: execute scripts/chapter_36.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 37. Level 1: articulate the assumption and principal use of machine learning engineering and mlops. Level 2: check the fully worked arithmetic in Chapter 37, Section 3. Level 3: execute scripts/chapter_37.py, reproduce the printed output, then justify any code modifications and metric changes.

Chapter 38. Level 1: articulate the assumption and principal use of machine learning in electrical engineering and embedded systems. Level 2: check the fully worked arithmetic in Chapter 38, Section 3. Level 3: execute scripts/chapter_38.py, reproduce the printed output, then justify any code modifications and metric changes.

Appendix I — Debugging and troubleshooting

Shape mismatch. Print array dimensions; verify (n,p) multiplied by (p,).

Model trains but validation fails. Check leakage, split, label quality, drift and baseline.

Unexpectedly high accuracy. Audit duplicates, target proxies, time ordering and leaked labels.

Exploding loss. Lower learning rate; check scaling, gradients, targets and optimizer.

NaN gradients. Check zero divisions, log of zero, overflows and invalid inputs.

No learning. Check labels, gradient flow, loss, frozen parameters and step order.

Unbalanced classes. Report PR metrics, stratify appropriately, and consider costs.

Unstable randomness. Set seeds and disclose nondeterministic computation.

KNN model slow. Use scaled features, indexing or approximate nearest-neighbor methods.

Out-of-memory error. Reduce batch size, dimensionality, precision or intermediate copies.

Plots not interpretable. Label units and axes; plot distributions as well as averages.

Counterintuitive SHAP. Inspect correlated predictors and background distribution assumptions.

Transform changes inference columns. Save one fitted pipeline and validate feature names/order.

Tiny text classifier fails. Expand representative data; check stratification and vocabulary leakage.

Time-series leak. Construct lags and transformations from past-only information.

Dataset/licensing unclear. Document origin, permissions, permitted usage, and retention.

Model serialization unsafe. Only load artifacts from trustworthy controlled sources.

Edge latency too high. Measure worst-case runtime, quantize, prune, or choose a smaller model.

FFT peak wrong. Check sampling rate, anti-aliasing, signal duration and window.

Evaluation not reproducible. Record exact data split, package versions, random seed and code commit.

Appendix J — Further learning and references

AIMA, Russell and Norvig — Artificial intelligence breadth: logic, search, reasoning and agents.
https://aima.cs.berkeley.edu/

Bishop, Pattern Recognition and Machine Learning — Statistical learning and probabilistic models.
https://www.microsoft.com/en-us/research/people/cmbishop/

Murphy, Probabilistic Machine Learning — Modern probabilistic modeling with mathematical rigor.
https://probml.github.io/pml-book/

Goodfellow, Bengio, Courville, Deep Learning — Theory and practice of neural networks.
https://www.deeplearningbook.org/

Stanford CS229 — Machine learning course materials.
https://cs229.stanford.edu/

Stanford CS231n — Deep learning and computer vision.
https://cs231n.stanford.edu/

Stanford CS224N — Neural natural language processing.
https://web.stanford.edu/class/cs224n/

scikit-learn user guide — Official classical ML library documentation.
https://scikit-learn.org/stable/user_guide.html

PyTorch documentation — Official tensor and deep learning documentation.
https://pytorch.org/docs/stable/

NumPy documentation — Official numerical array reference.
https://numpy.org/doc/stable/

pandas documentation — Official DataFrame documentation.
https://pandas.pydata.org/docs/

SciPy documentation — Numerical algorithms and statistical functions.
https://docs.scipy.org/doc/scipy/

UCI Machine Learning Repository — Public datasets with individual data-use notes.
https://archive.ics.uci.edu/

OpenML — Benchmarks and public datasets.
https://www.openml.org/

Papers with Code — Research papers linked to code and benchmarks.
https://paperswithcode.com/

Datasets in the companion scripts

Datasets used in examples include the scikit-learn Iris, Wine, breast cancer, diabetes, and handwritten digits datasets, each loaded from scikit-learn. Other numerical and text datasets are constructed synthetically in code. Read the original dataset information and licenses before redistribution or real-world reuse.

Appendix K — Reproducibility and chapter map

Setup

```
python -m venv .venv
# On Windows: .venv\Scripts\activate
# On macOS/Linux: source .venv/bin/activate
python -m pip install -r requirements.txt
python scripts/chapter_11.py
python scripts/project_05.py
```

Executed program index

Chapter 01. scripts/chapter_01.py; executed output in outputs/chapter_01.txt; formula and visualization in Chapter 1.

Chapter 02. scripts/chapter_02.py; executed output in outputs/chapter_02.txt; formula and visualization in Chapter 2.

Chapter 03. scripts/chapter_03.py; executed output in outputs/chapter_03.txt; formula and visualization in Chapter 3.

Chapter 04. scripts/chapter_04.py; executed output in outputs/chapter_04.txt; formula and visualization in Chapter 4.

Chapter 05. scripts/chapter_05.py; executed output in outputs/chapter_05.txt; formula and visualization in Chapter 5.

Chapter 06. scripts/chapter_06.py; executed output in outputs/chapter_06.txt; formula and visualization in Chapter 6.

Chapter 07. scripts/chapter_07.py; executed output in outputs/chapter_07.txt; formula and visualization in Chapter 7.

Chapter 08. scripts/chapter_08.py; executed output in outputs/chapter_08.txt; formula and visualization in Chapter 8.

Chapter 09. scripts/chapter_09.py; executed output in outputs/chapter_09.txt; formula and visualization in Chapter 9.

Chapter 10. scripts/chapter_10.py; executed output in outputs/chapter_10.txt; formula and visualization in Chapter 10.

Chapter 11. scripts/chapter_11.py; executed output in outputs/chapter_11.txt; formula and visualization in Chapter 11.

Chapter 12. scripts/chapter_12.py; executed output in outputs/chapter_12.txt; formula and visualization in Chapter 12.

Chapter 13. scripts/chapter_13.py; executed output in outputs/chapter_13.txt; formula and visualization in Chapter 13.

Chapter 14. scripts/chapter_14.py; executed output in outputs/chapter_14.txt; formula and visualization in Chapter 14.

Chapter 15. scripts/chapter_15.py; executed output in outputs/chapter_15.txt; formula and visualization in Chapter 15.

Chapter 16. scripts/chapter_16.py; executed output in outputs/chapter_16.txt; formula and visualization in Chapter 16.

Chapter 17. scripts/chapter_17.py; executed output in outputs/chapter_17.txt; formula and visualization in Chapter 17.

Chapter 18. scripts/chapter_18.py; executed output in outputs/chapter_18.txt; formula and visualization in Chapter 18.

Chapter 19. scripts/chapter_19.py; executed output in outputs/chapter_19.txt; formula and visualization in Chapter 19.

Chapter 20. scripts/chapter_20.py; executed output in outputs/chapter_20.txt; formula and visualization in Chapter 20.

Chapter 21. scripts/chapter_21.py; executed output in outputs/chapter_21.txt; formula and visualization in Chapter 21.

Chapter 22. scripts/chapter_22.py; executed output in outputs/chapter_22.txt; formula and visualization in Chapter 22.

Chapter 23. scripts/chapter_23.py; executed output in outputs/chapter_23.txt; formula and visualization in Chapter 23.

Chapter 24. scripts/chapter_24.py; executed output in outputs/chapter_24.txt; formula and visualization in Chapter 24.

Chapter 25. scripts/chapter_25.py; executed output in outputs/chapter_25.txt; formula and visualization in Chapter 25.

Chapter 26. scripts/chapter_26.py; executed output in outputs/chapter_26.txt; formula and visualization in Chapter 26.

Chapter 27. scripts/chapter_27.py; executed output in outputs/chapter_27.txt; formula and visualization in Chapter 27.

Chapter 28. scripts/chapter_28.py; executed output in outputs/chapter_28.txt; formula and visualization in Chapter 28.

Chapter 29. scripts/chapter_29.py; executed output in outputs/chapter_29.txt; formula and visualization in Chapter 29.

Chapter 30. scripts/chapter_30.py; executed output in outputs/chapter_30.txt; formula and visualization in Chapter 30.

Chapter 31. scripts/chapter_31.py; executed output in outputs/chapter_31.txt; formula and visualization in Chapter 31.

Chapter 32. scripts/chapter_32.py; executed output in outputs/chapter_32.txt; formula and visualization in Chapter 32.

Chapter 33. scripts/chapter_33.py; executed output in outputs/chapter_33.txt; formula and visualization in Chapter 33.

Chapter 34. scripts/chapter_34.py; executed output in outputs/chapter_34.txt; formula and visualization in Chapter 34.

Chapter 35. scripts/chapter_35.py; executed output in outputs/chapter_35.txt; formula and visualization in Chapter 35.

Chapter 36. scripts/chapter_36.py; executed output in outputs/chapter_36.txt; formula and visualization in Chapter 36.

Chapter 37. scripts/chapter_37.py; executed output in outputs/chapter_37.txt; formula and visualization in Chapter 37.

Chapter 38. scripts/chapter_38.py; executed output in outputs/chapter_38.txt; formula and visualization in Chapter 38.

Project scripts

Project 01. House-price baseline; scripts/project_01.py and outputs/project_01.txt.

Project 02. Diabetes-outcome regression; scripts/project_02.py and outputs/project_02.txt.

Project 03. Spam detection; scripts/project_03.py and outputs/project_03.txt.

Project 04. Customer segmentation; scripts/project_04.py and outputs/project_04.txt.

Project 05. Handwritten digits; scripts/project_05.py and outputs/project_05.txt.

Project 06. Image classification in PyTorch; scripts/project_06.py and outputs/project_06.txt.

Project 07. Chronological time-series forecasting; scripts/project_07.py and outputs/project_07.txt.

Project 08. Anomaly detection; scripts/project_08.py and outputs/project_08.txt.

Project 09. Recommendation baseline; scripts/project_09.py and outputs/project_09.txt.

Project 10. Predictive maintenance; scripts/project_10.py and outputs/project_10.txt.

Project 11. Text sentiment classification; scripts/project_11.py and outputs/project_11.txt.

Project 12. Minimal Transformer model; scripts/project_12.py and outputs/project_12.txt.

Known scope limitations

- The handbook develops one principal equation and one complete, small Python demonstration per chapter; it does not provide a separate proof and implementation for every topic in the roadmaps.
- Some chapter examples illustrate mechanisms on training data; those are not reliable measurements of generalization unless a held-out or cross-validated evaluation is explicitly present.
- The toy text examples are too small for reliable accuracy claims. Synthetic equipment examples cannot validate real equipment safety or performance.
- Package and hardware differences can affect stochastic outputs. 38 chapter scripts and 12 projects were verified in the creation environment; the exact runtime versions are in companion environment.txt.
- Figures are scientifically labeled teaching illustrations generated from synthetic/reference numerical examples. They are not empirical research results.